



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Luis Miguel Casado Díaz
Mayo 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary (Methodologies)

Data Collection formed the foundation of the pipeline through two parallel acquisition strategies. Historical launch records were retrieved programmatically via the **SpaceX REST API v4**, making sequential calls to four endpoints — /rockets, /launchpads, /payloads, and /cores — to enrich each launch record with booster version, launch site coordinates, payload mass, and core reuse details. In parallel, a **web scraping** pipeline was built using BeautifulSoup and requests to extract and parse launch tables directly from a Wikipedia snapshot, handling raw HTML structure, irregular cells, and encoded characters through a set of custom helper functions. Both pipelines converged into a unified Pandas DataFrame exported as a reproducible CSV.

Data Wrangling prepared the raw dataset for modelling by auditing missing values, imputing PayloadMass nulls with the column mean, and — critically — engineering the binary target variable Class (1 = successful landing, 0 = failure or no attempt) by mapping the ten distinct outcome strings into two categories. This label became the ground truth for all downstream supervised learning.

Exploratory Data Analysis was conducted in two complementary layers. A **visual EDA** using matplotlib and seaborn examined relationships between flight number, payload mass, orbit type, launch site, and landing outcome across six structured plots, revealing that success rates increased with mission experience, varied significantly by orbit type, and showed site-specific payload constraints. A **geospatial EDA** using Folium mapped all four launch sites as interactive markers, overlaid individual launch outcomes as colour-coded clustered icons.

Feature Engineering transformed the cleaned dataset into a machine-learning-ready matrix by applying one-hot encoding (pd.get_dummies) to all categorical variables — Orbit, LaunchSite, LandingPad, and Serial — and casting the full feature matrix to float64. Features were then standardised using StandardScaler to ensure zero mean and unit variance, preventing scale-sensitive algorithms from being distorted by variables of differing magnitude.

Model Training & Evaluation compared four supervised classification algorithms — **Logistic Regression**, **Support Vector Machine**, **Decision Tree**, and **K-Nearest Neighbours** — each tuned via GridSearchCV with 10-fold cross-validation over algorithm-specific hyperparameter grids. Models were evaluated on a stratified 80/20 train-test split using five metrics — Accuracy, Precision, Recall, F1 Score, and Jaccard Index — to account for class imbalance and surface precision-recall trade-offs beyond what accuracy alone reveals. Logistic Regression and SVM tied across all test metrics (F1 = 0.889, Jaccard = 0.800), with Logistic Regression selected as the preferred model on grounds of interpretability and computational efficiency. The Decision Tree achieved the highest cross-validation score (0.920) but showed signs of overfitting, while KNN underperformed on the held-out test set relative to the other three methods.

Executive Summary (Results)

- SpaceX's competitive pricing advantage over rival launch providers stems primarily from its ability to recover and reuse the Falcon 9 first stage booster, reducing per-launch costs from an industry average of \$165M to \$62M. The ability to predict whether a given launch will result in a successful booster recovery therefore carries direct commercial intelligence value — both for SpaceX's operational planning and for competing providers seeking to assess bid viability. This project developed and validated a machine learning pipeline to make that prediction from publicly available launch data.
- **Data & Feature Landscape.** The dataset compiled 90 Falcon 9 launches through November 2020, sourced from the SpaceX REST API and supplemented with Wikipedia records. After wrangling and feature engineering, each launch was described by 83 variables covering payload mass, orbit type, launch site, booster block version, reuse count, grid fin and leg configuration, and landing pad assignment. The overall historical landing success rate across the dataset stands at approximately **66.7%**, rising sharply from near zero in 2013 to above 80% by 2020 — reflecting SpaceX's accelerating operational maturity.
- **Key Analytical Findings.** Geospatial analysis confirmed that all four active launch sites share a deliberate siting strategy: low latitude for orbital efficiency, immediate coastal access to ensure spent stages fall over open ocean, and proximity to transport infrastructure while maintaining a safety buffer from populated areas. EDA revealed that success rates vary substantially by orbit type — ES-L1, GEO, HEO, and SSO missions achieved near-perfect landing records, while GTO missions remained mixed — and that heavier payloads did not systematically reduce success probability within LEO and ISS-bound missions.
- **Model Performance.** All four classifiers delivered competitive results on the 18-sample held-out test set. Logistic Regression and SVM achieved the strongest and equal performance, with a **test accuracy of 83.3%, and F1 Score of 0.889**, correctly predicting every successful landing in the test set (Recall = 1.0) at the cost of three false positives. The Decision Tree showed the highest cross-validation score (92.0%) but did not generalise as cleanly to unseen data, suggesting overfitting. KNN underperformed across all metrics. **Logistic Regression was selected as the production model** on the basis of tied predictive performance with SVM, superior interpretability of its decision coefficients, and significantly lower computational overhead.
- **Business Implication.** A model achieving 83% accuracy on unseen launches — with zero missed successful recoveries in the test set — provides a reliable probabilistic signal for estimating whether the \$100M+ cost differential between a reused and expendable Falcon 9 is factored into a given launch price. For a competing provider preparing a bid, this translates directly into a more informed and defensible pricing strategy

- [GitHub Repo](#)

Introduction

The commercial space launch industry underwent a fundamental economic disruption when SpaceX introduced the concept of orbital-class booster recovery and reuse at scale. Prior to this, rocket first stages were expendable — discarded into the ocean after a single flight — making the per-launch cost of a Falcon 9 competitor prohibitively high, typically exceeding \$165 million. SpaceX's ability to land, refurbish, and re-fly the Falcon 9 first stage compresses that cost to approximately \$62 million per mission, a saving of over 60% that no rival provider has been able to consistently match. This cost structure has progressively reshaped the competitive dynamics of the global launch market, allowing SpaceX to underbid incumbents on commercial satellite contracts, NASA resupply missions, and crewed spaceflight programs simultaneously.

The financial hinge of this advantage is the booster recovery itself. When a first stage is successfully landed — whether on an autonomous drone ship at sea or a ground-based return-to-launch-site manoeuvre — it can be inspected, refurbished, and relaunched, amortizing its manufacturing cost across multiple missions. When recovery fails, that cost is written off entirely. For any organization seeking to compete with or procure from SpaceX, knowing in advance whether a given mission is likely to result in a successful recovery carries direct commercial relevance: it determines whether the advertised price reflects a reused booster or an expendable one, and therefore whether a competing bid can be priced competitively or not. This project was conceived precisely at that intersection — applying supervised machine learning to publicly available launch records to build a predictive model capable of estimating first stage landing success before a mission takes place, transforming an operational outcome into a quantifiable, data-driven business signal.

Introduction

- Problems you want to find answers

The central question driving this project is deceptively simple: **can the outcome of a Falcon 9 first stage landing be predicted before the mission takes place, using only information that is publicly available at the time of launch?**

Answering it, however, requires resolving a chain of interconnected analytical challenges.

First, launch data is not consolidated in any single structured source — it must be retrieved, reconciled, and integrated across the SpaceX API and third-party records, introducing questions of completeness, consistency, and reproducibility.

Second, the factors that influence landing success are not immediately obvious: payload mass, orbit type, launch site, booster reuse history, and hardware configuration all interact in ways that require systematic exploration before any modelling can begin.

Third, the dataset is relatively small and historically skewed — early Falcon 9 missions had near-zero recovery rates, while recent missions succeed routinely — creating a class imbalance that naive accuracy metrics cannot adequately capture.

Fourth, no single machine learning algorithm is guaranteed to handle this combination of binary classification, mixed feature types, and limited sample size optimally, making algorithm selection and hyperparameter tuning a non-trivial decision with measurable consequences.

Finally, and most importantly for commercial applicability, a prediction is only useful if it is trustworthy: the model must demonstrably generalise to launches it has never seen, distinguish genuine predictive signal from overfitted noise, and be interpretable enough for a non-technical stakeholder — a procurement officer, a rival launch provider, a mission planner — to act on its output with confidence.

¿can determine if the first stage will land? I can determine the cost of a launch so the sale price

Since the first stage is often sacrificed due to mission parameters like payload weight or specific orbits, its recovery is never guaranteed. Instead of relying on complex aerospace engineering simulations, I will use a data-driven approach to predict these outcomes.

By training a machine learning model on public information, I will determine whether the first stage will land successfully, allowing your team to estimate SpaceX's costs and set competitive pricing for Space Y.

Section 1

Methodology

Data Collection – SpaceX API (1/6)

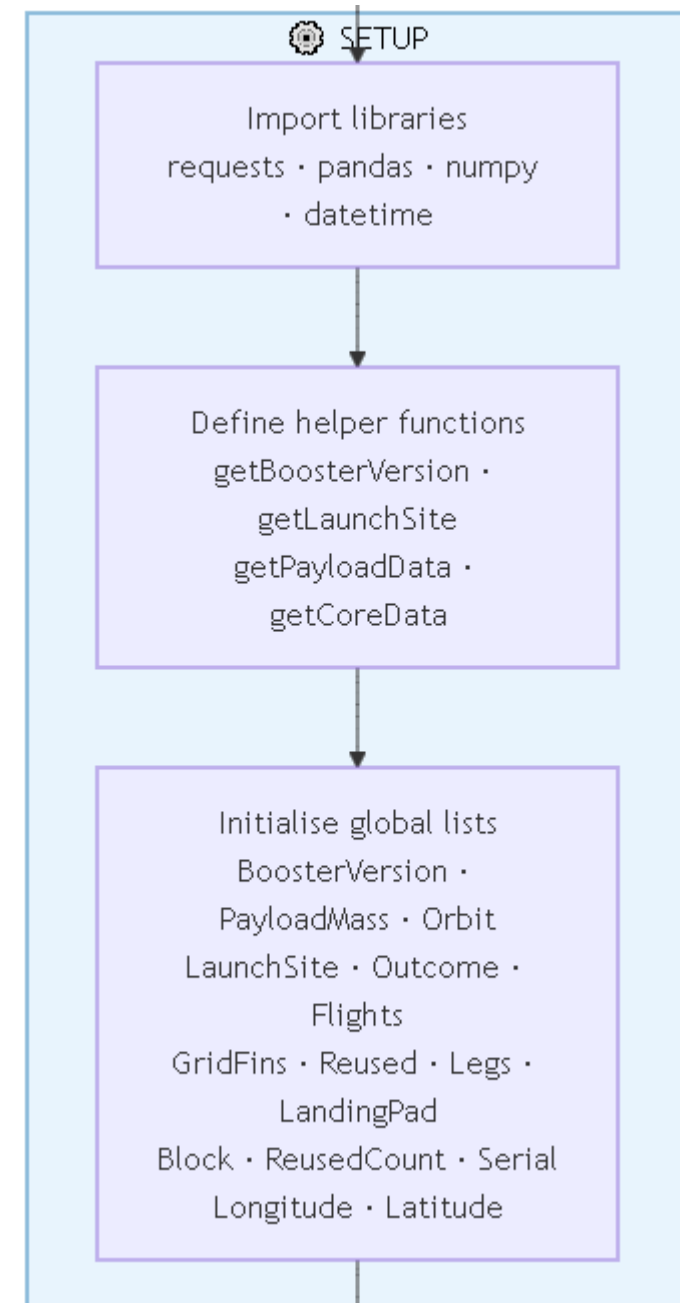
- The data-collection process using SpaceX REST API calls is shown in the next five slides.
- [API data-collection notebook](#), is included as an external reference for verification and peer-review purpose.

Data Collection – SpaceX API (2/6)

- Install the required packages, Import the necessary libraries, and define helper functions that extract strings from table cells passed as arguments:

Helper Function	API Access Point
getBoosterVersion	https://api.spacexdata.com/v4/rockets/
getLaunchSite	https://api.spacexdata.com/v4/launchpads/
getPayloadData	https://api.spacexdata.com/v4/payloads/
getCoreData	https://api.spacexdata.com/v4/cores/

- Helper functions iterate over the data rows, query the API access point for each entry, and append the retrieved data to global list variables.



Data Collection – SpaceX API (3/6)

- Retrieve JSON data, normalize it into a Pandas DataFrame, get a working subset of the data, remove multiple cores launch, and launch after one day.

```
response=requests.get(static_json_url)
launches = response.json()
```

```
data = pd.json_normalize(launches)
```

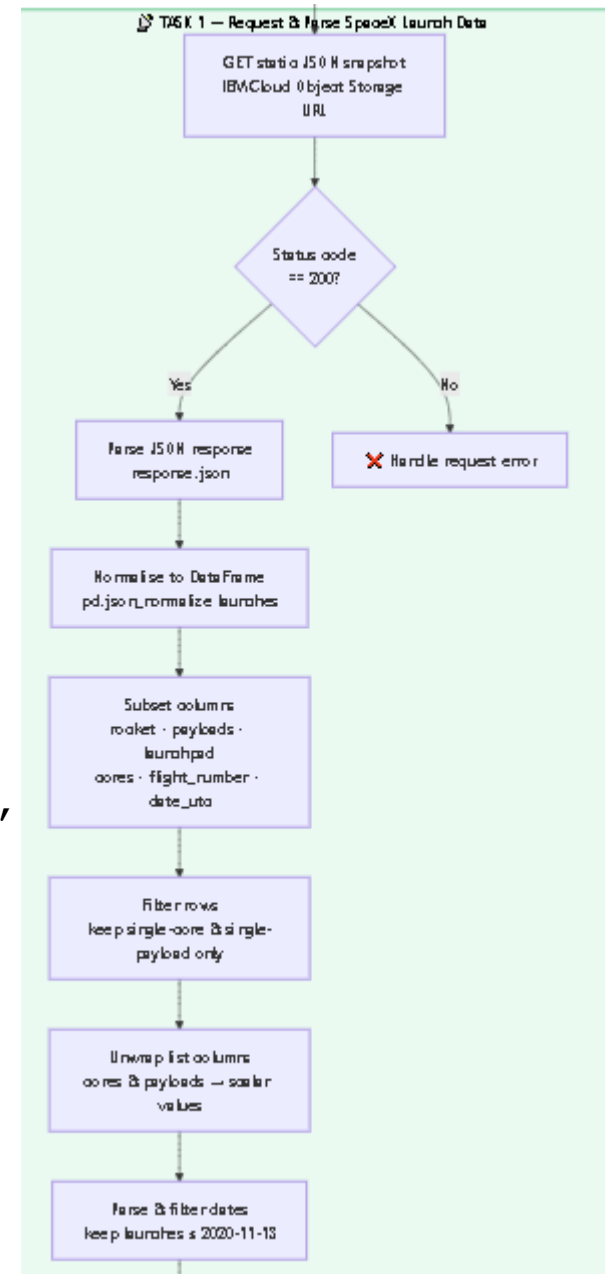
```
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number',
'date_utc']]
```

```
data = data[data['cores'].map(len)==1]
data = data[data['payloads'].map(len)==1]
```

```
data['cores'] = data['cores'].map(lambda x : x[0])
data['payloads'] = data['payloads'].map(lambda x : x[0])
```

```
data['date'] = pd.to_datetime(data['date_utc']).dt.date
```

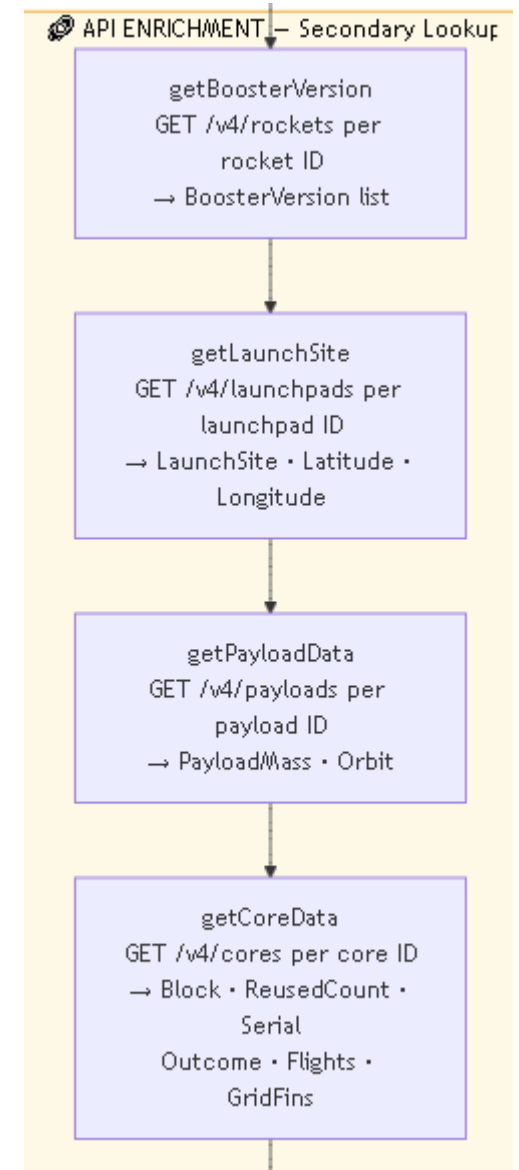
```
data = data[data['date'] <= datetime.date(2020, 11, 13)]
```



Data Collection – SpaceX API (4/6)

- Iterate over the data rows, retrieving additional information through secondary lookups. This is achieved by sequentially invoking all helper functions, each of which appends values to the global list variables.

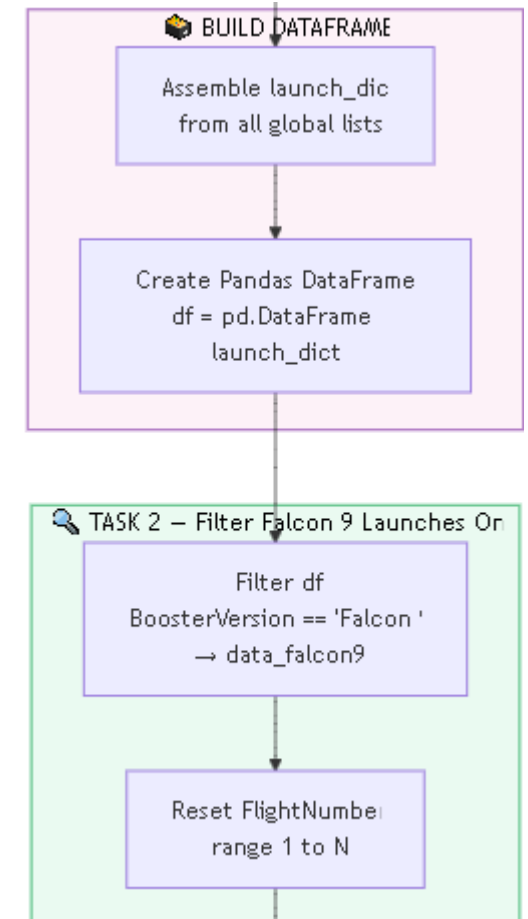
```
# Takes the dataset and uses the payloads column to call the API and append the data to the lists
def getPayloadData(data):
    for load in data['payloads']:
        if load:
            response =
requests.get("https://api.spacexdata.com/v4/payloads/"+load).json()
            PayloadMass.append(response['mass_kg'])
            Orbit.append(response['orbit'])
```



Data Collection – SpaceX API (5/6)

- Construct a dictionary from the global list variables to serve as the input to the DataFrame constructor, and filter it so that includes only falcon 9 launches.

```
launch_dict = {'FlightNumber':  
list(data['flight_number']), 'Date':  
list(data['date']),  
'BoosterVersion':BoosterVersion,  
'PayloadMass':PayloadMass, 'Orbit':Orbit,  
'LaunchSite':LaunchSite, 'Outcome':Outcome,  
'Flights':Flights, 'GridFins':GridFins,  
'Reused':Reused, 'Legs':Legs, 'LandingPad':LandingPad,  
'Block':Block, 'ReusedCount':ReusedCount,  
'Serial':Serial, 'Longitude': Longitude, 'Latitude':  
Latitude}  
  
# Create a data from launch dict  
df = pd.DataFrame(launch_dict)  
data_falcon9 = df[df['BoosterVersion'] == 'Falcon 9'].copy()  
  
# Reset the flight number column  
data_falcon9.loc[:, 'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))
```

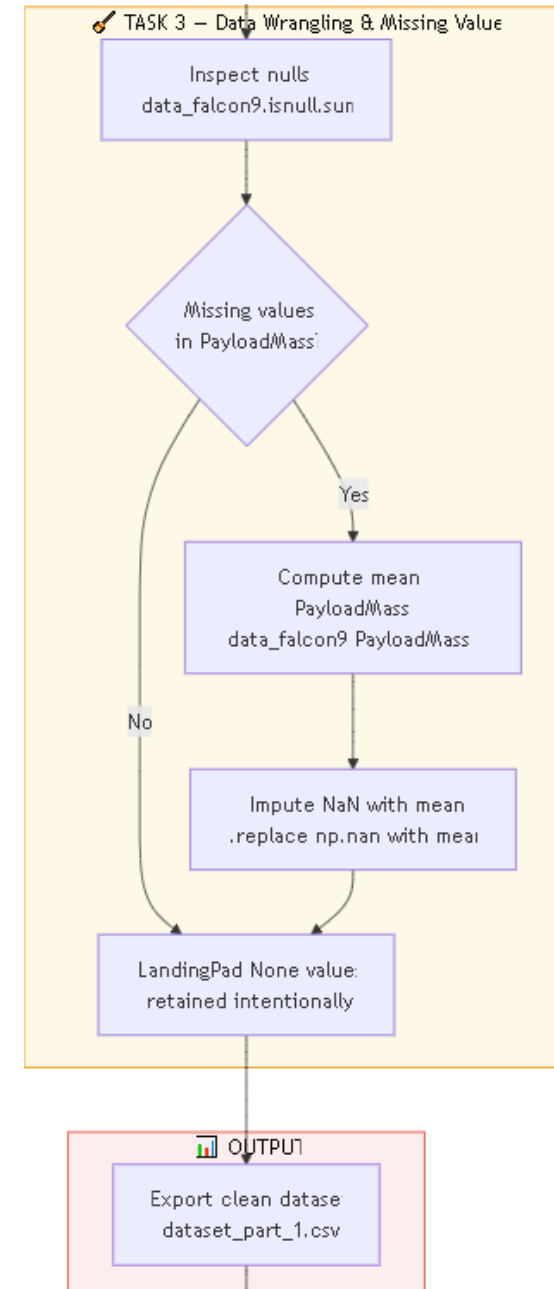


Data Collection – SpaceX API (6/6)

- Replace the PayloadMass NULL values with the mean and save the dataset

```
# Calculate the mean value of PayloadMass
column
PayloadMass_mean =
data_falcon9['PayloadMass'].mean(axis=0)
# Replace the np.nan values with its mean
value
data_falcon9['PayloadMass'].replace(np.nan,
PayloadMass_mean)
```

```
data_falcon9.to_csv('dataset_part_1.csv', index=False)
```

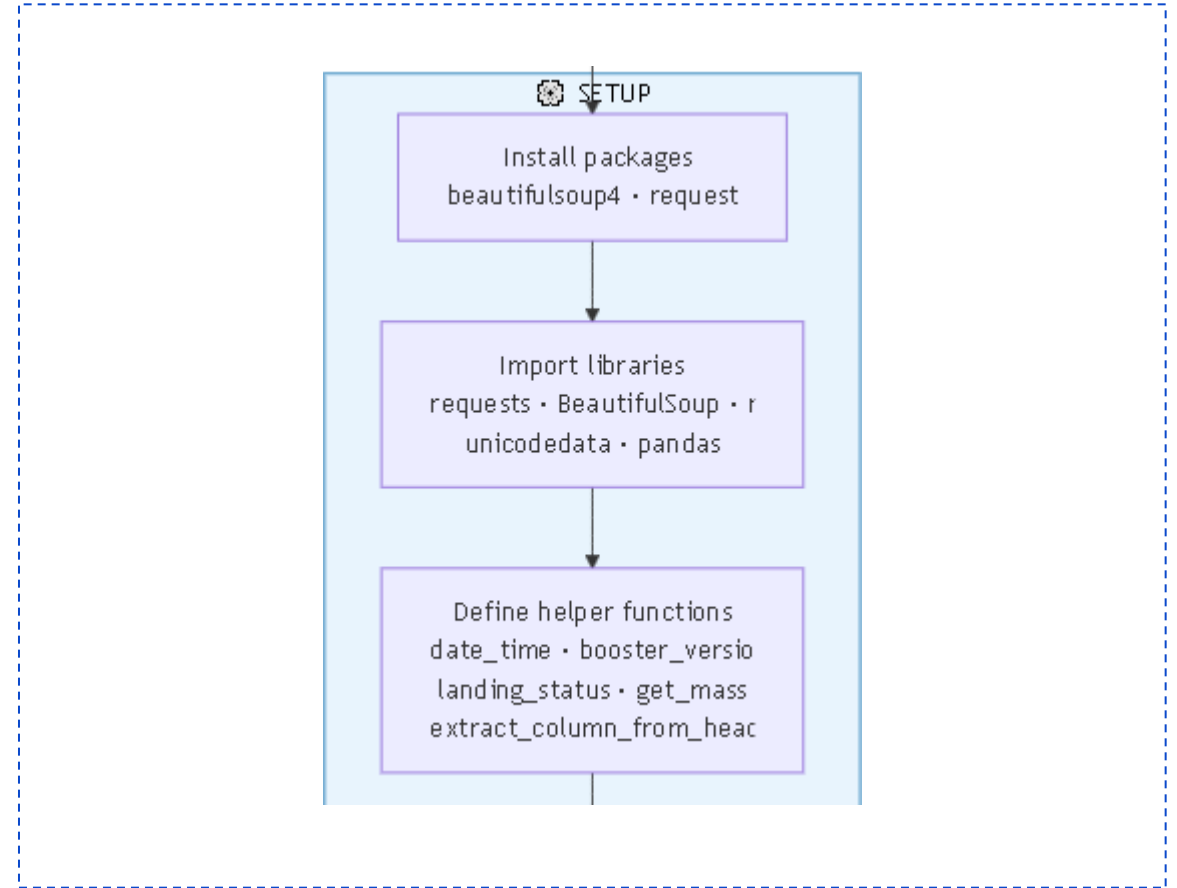


Data Collection – Scraping (1/6)

- The web-scraping process is presented in the following five slides.
- [The web-scraping notebook](#) is provided as an external reference for transparency and peer-review purposes

Data Collection - Scraping (2/6)

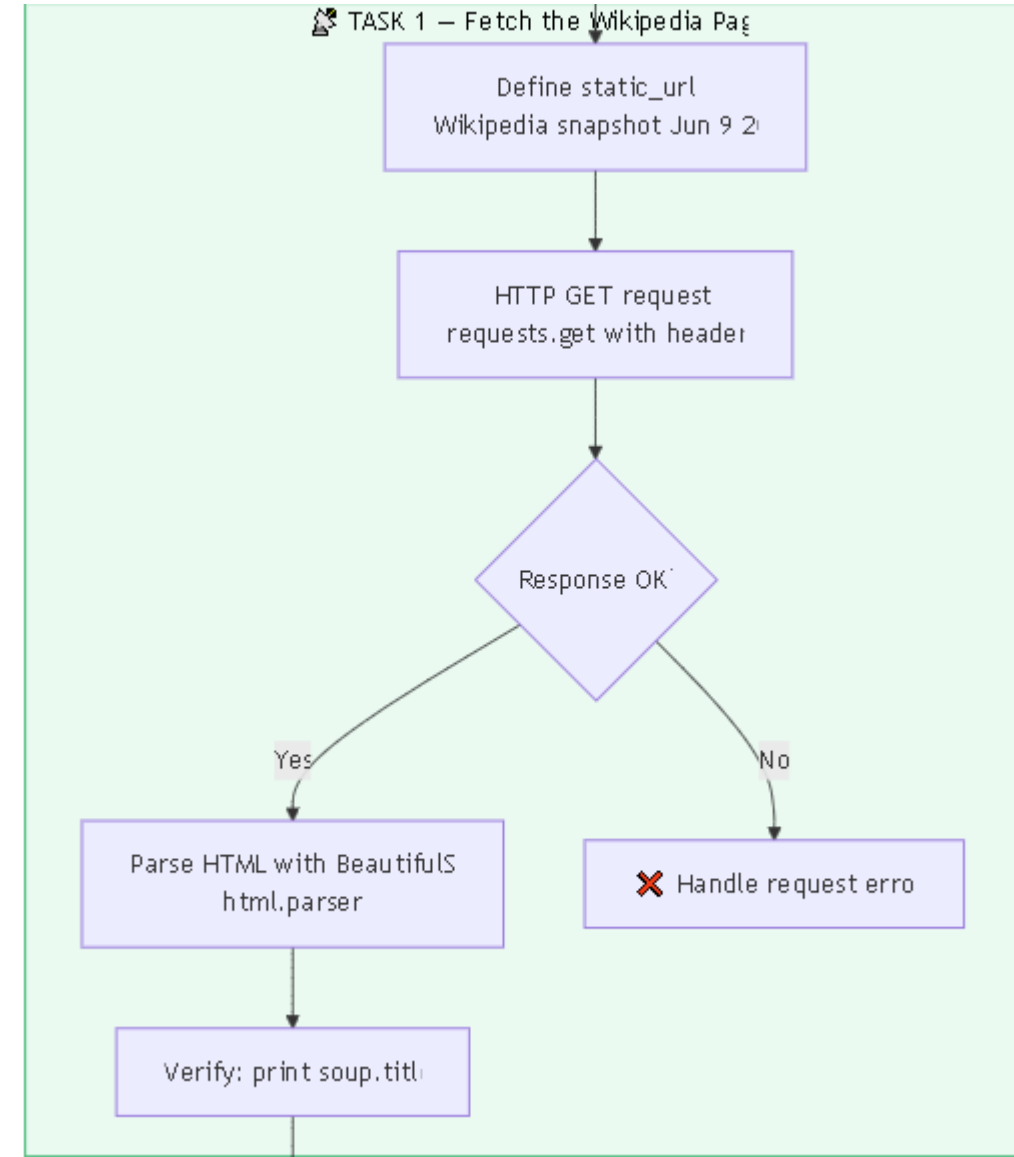
- Install the required packages, Import the necessary libraries, and define helper functions that extract strings from table cells passed as arguments:
 - `data_time`
 - `booster_version`
 - `landing_status`
 - `get_mass`



Data Collection - Scraping (3/6)

- Retrieve the data source by issuing an HTTP request to a static URL containing a fixed data subset, ensuring that all students can replicate the results

```
static_url =  
"https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and  
_Falcon_Heavy_launches&oldid=1027686922"  
  
headers = {  
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "  
                "AppleWebKit/537.36 (KHTML, like Gecko) "  
                "Chrome/91.0.4472.124 Safari/537.36"  
}  
  
# use requests.get() method with the provided static_url and  
headers  
# assign the response to a object  
response=requests.get(static_url, headers=headers)
```



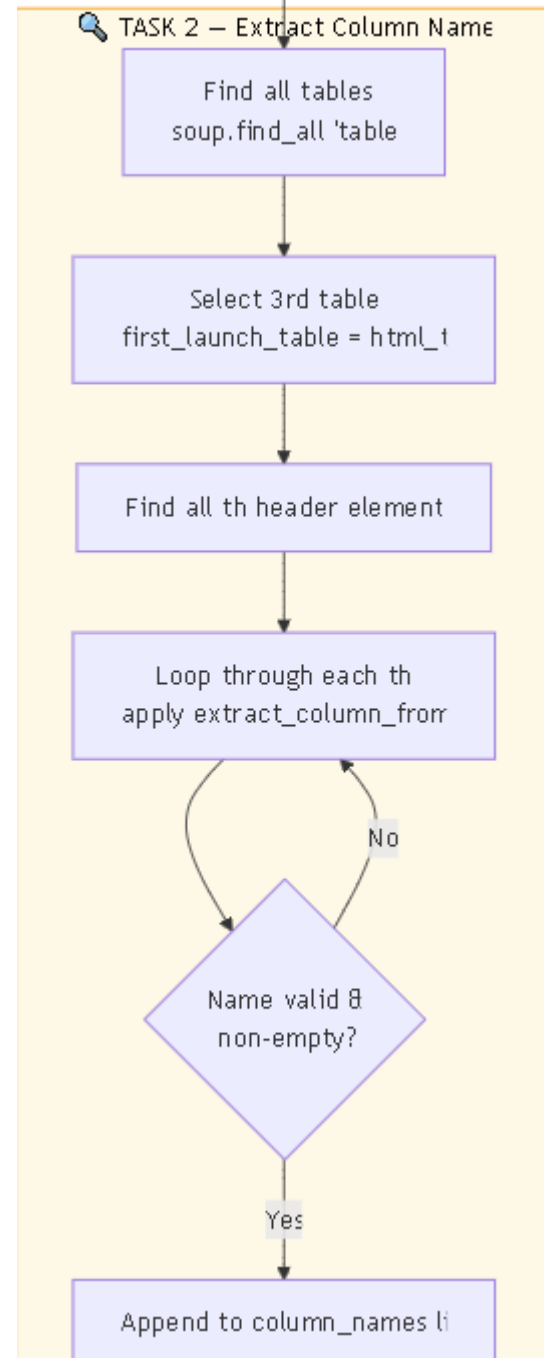
Data Collection - Scraping (4/6)

- Extract then column names from the third table by iterating over each <th> HTML tag and obtaining the corresponding header text

```
column_names = []
```

```
# Apply find_all() function with `th` element on
first_launch_table
# Iterate each th element and apply the provided
extract_column_from_header() to get a column name
# Append the Non-empty column name (`if name is not None
and len(name) > 0`) into a list called column_names
columns = first_launch_table.find_all('th')
for col in columns:
    name = extract_column_from_header(col)
    if name != None and name != '':
        column_names.append(name)
```

```
['Flight No.', 'Date and time ( )', 'Launch site', 'Payload', 'Payload mass', 'Orbit',
'Customer', 'Launch outcome']
```

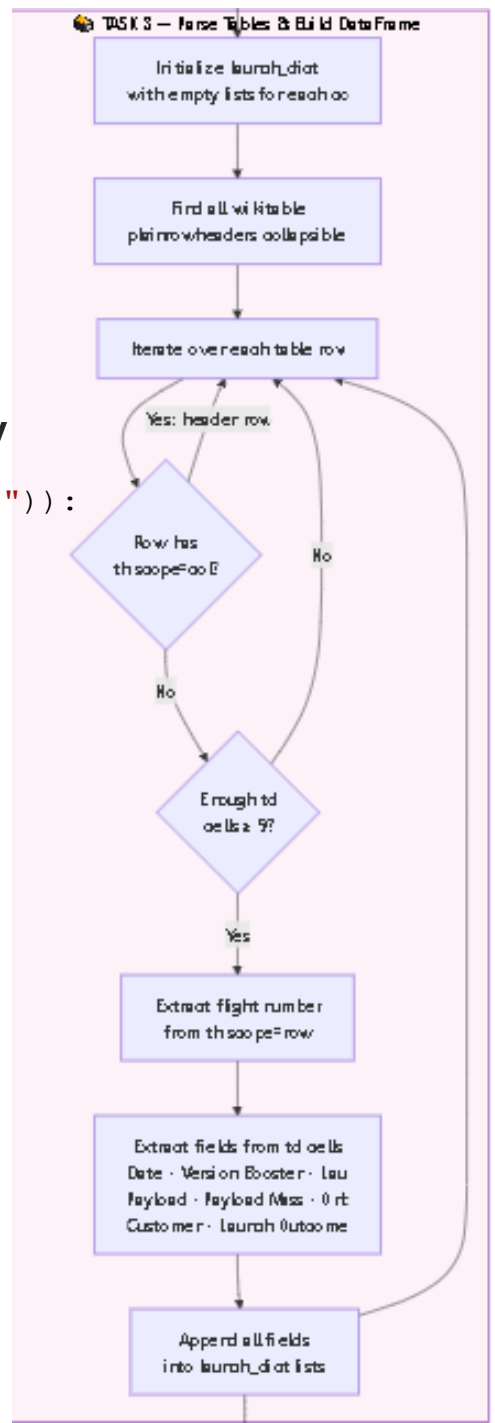


Data Collection - Scraping (5/6)

- Iterate through all rows in all tables, selecting only standard rows containing more than with more than nine columns.

From each valid row, extract the cell text and store it in a dictionary

```
for table_number, table in enumerate(soup.find_all('table', "wikitable plainrowheaders collapsible")):
    # get table row
    for row in table.find_all("tr"):
        # Skip header rows (they contain <th scope="col">)
        if row.find('th', attrs={"scope": "col"}):
            continue
        # Data rows contain <td> tags
        cols = row.find_all('td')
        # Skip empty rows or rows without enough columns
        if len(cols) < 9:
            continue
        # Extract flight number from <th scope="row">
        flight_no = row.find('th', attrs={"scope": "row"})
        if flight_no:
            launch_dict['Flight No.'].append(flight_no.get_text(strip=True))
            launch_dict['Date'].append(cols[0].get_text(strip=True))
            launch_dict['Version booster'].append(cols[1].get_text(strip=True))
            launch_dict['Launch site'].append(cols[2].get_text(strip=True))
            launch_dict['Payload'].append(cols[3].get_text(strip=True))
            launch_dict['Payload mass'].append(extract_kg(cols[4].get_text(strip=True)))
            launch_dict['Orbit'].append(cols[5].get_text(strip=True)[:3]) #keep only 3 first letters
            launch_dict['Customer'].append(cols[6].get_text(strip=True))
            launch_dict['Launch outcome'].append(cols[7].get_text(strip=True))
            launch_dict['Booster landing'].append(cols[8].get_text(strip=True))
```

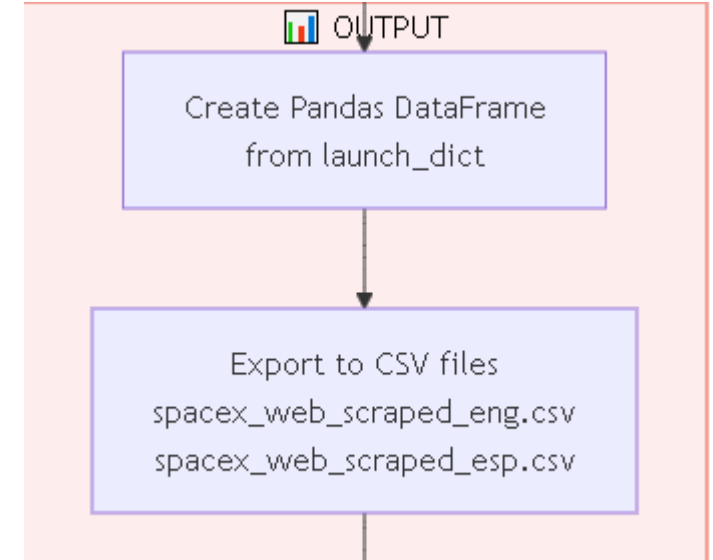


Data Collection - Scraping (5/6)

[illegible]

Data Collection - Scraping (6/6)

- Create the DataFrame from the dictionary.



```
df= pd.DataFrame({ key:pd.Series(value) for key, value in launch_dict.items() })
df.to_csv('spacex_web_scraped_eng.csv', index=False)
df.to_csv('spacex_web_scraped_esp.csv', index=False, decimal=',')
```

Data Wrangling

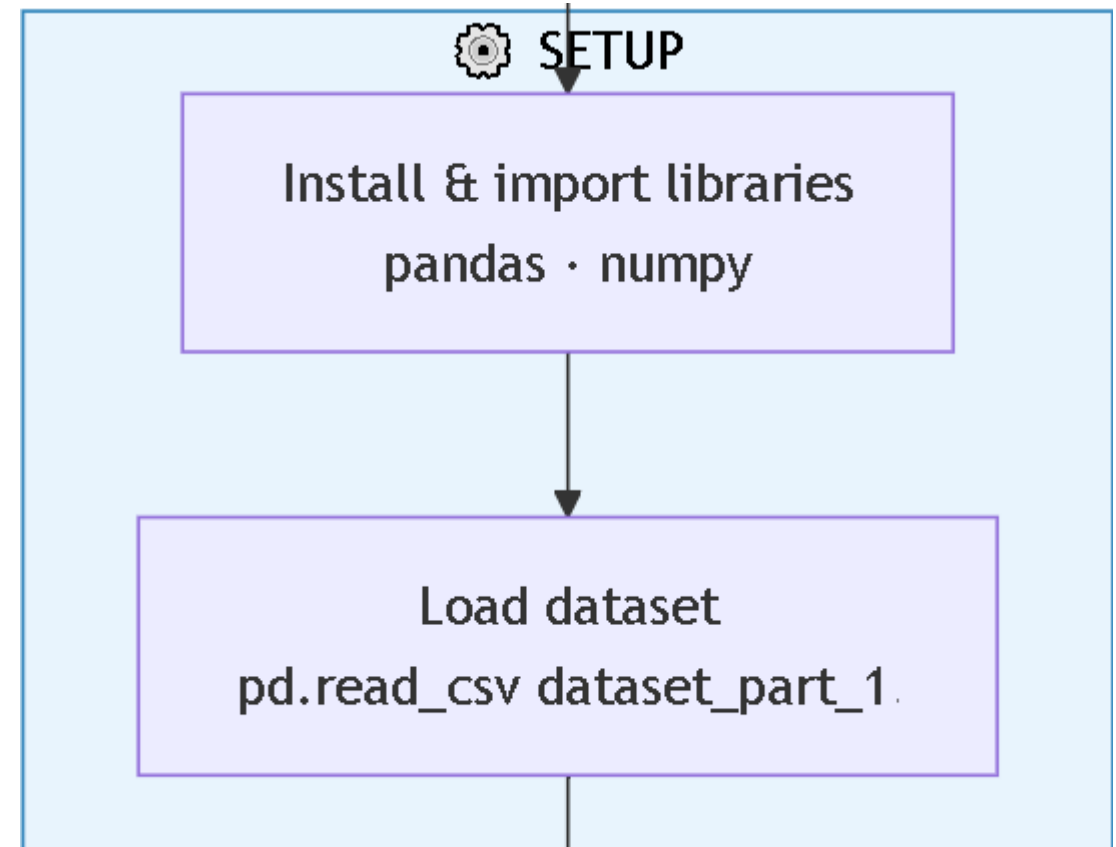
- Describe how data were processed
- You need to present your data wrangling process using key phrases and flowcharts
- Add the GitHub URL of your completed data wrangling related notebooks, as an external reference and peer-review purpose

Data Wrangling (1/6)

- The data-wrangling process is presented in the following five slides.
- [The data-wrangling notebook](#) is provided as an external reference for transparency and peer-review purposes

Data Wrangling (2/6)

- Install the required packages, Import the necessary libraries, and load an IBM-curated dataset that enables all students to replicate the results.

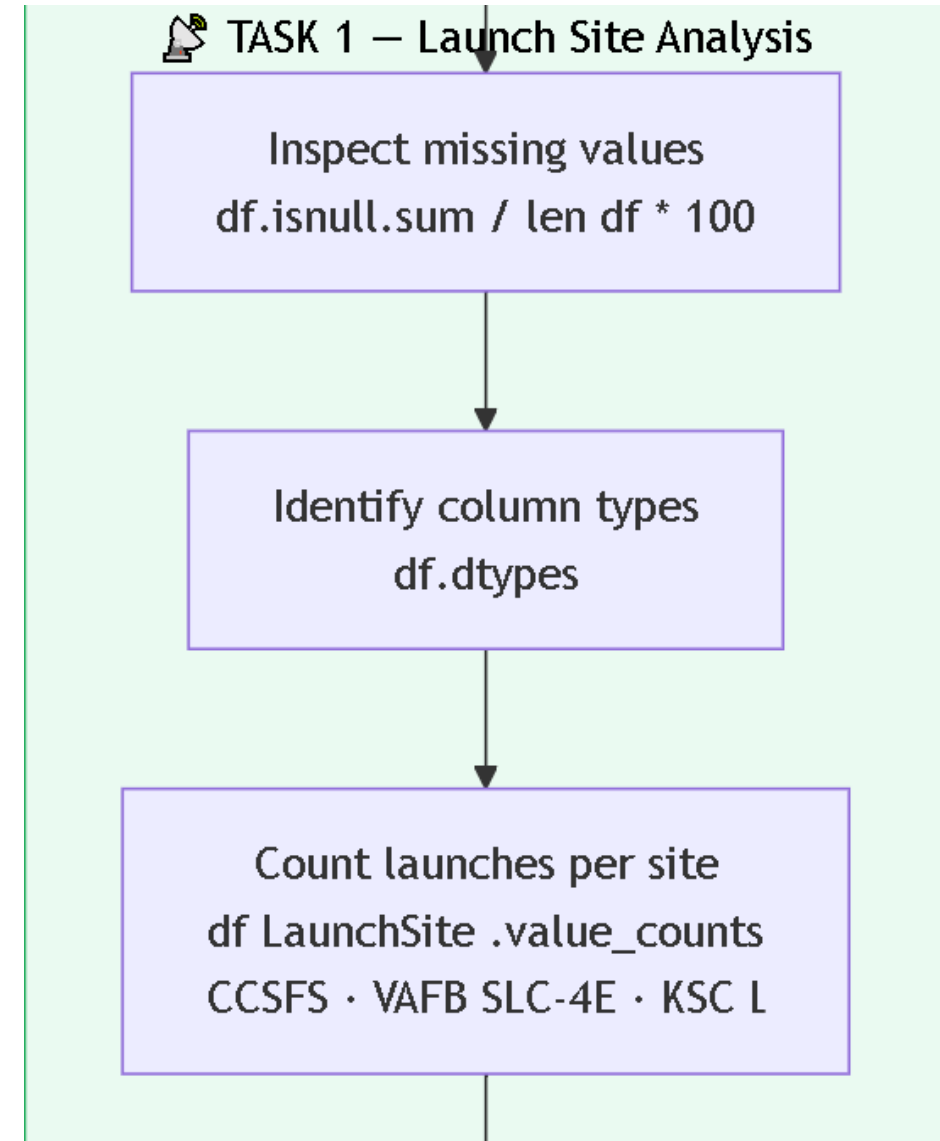


Data Wrangling (3/6)

- Check data types and missing values and count launches per LaunchSite

FlightNumber	0.000000	int64
Date	0.000000	str
BoosterVersion	0.000000	str
PayloadMass	0.000000	float64
Orbit	0.000000	str
LaunchSite	0.000000	str
Outcome	0.000000	str
Flights	0.000000	int64
GridFins	0.000000	bool
Reused	0.000000	bool
Legs	0.000000	bool
LandingPad	28.888889	str
Block	0.000000	float64
ReusedCount	0.000000	int64
Serial	0.000000	str
Longitude	0.000000	float64
Latitude	0.000000	float64
dtype:	float64	

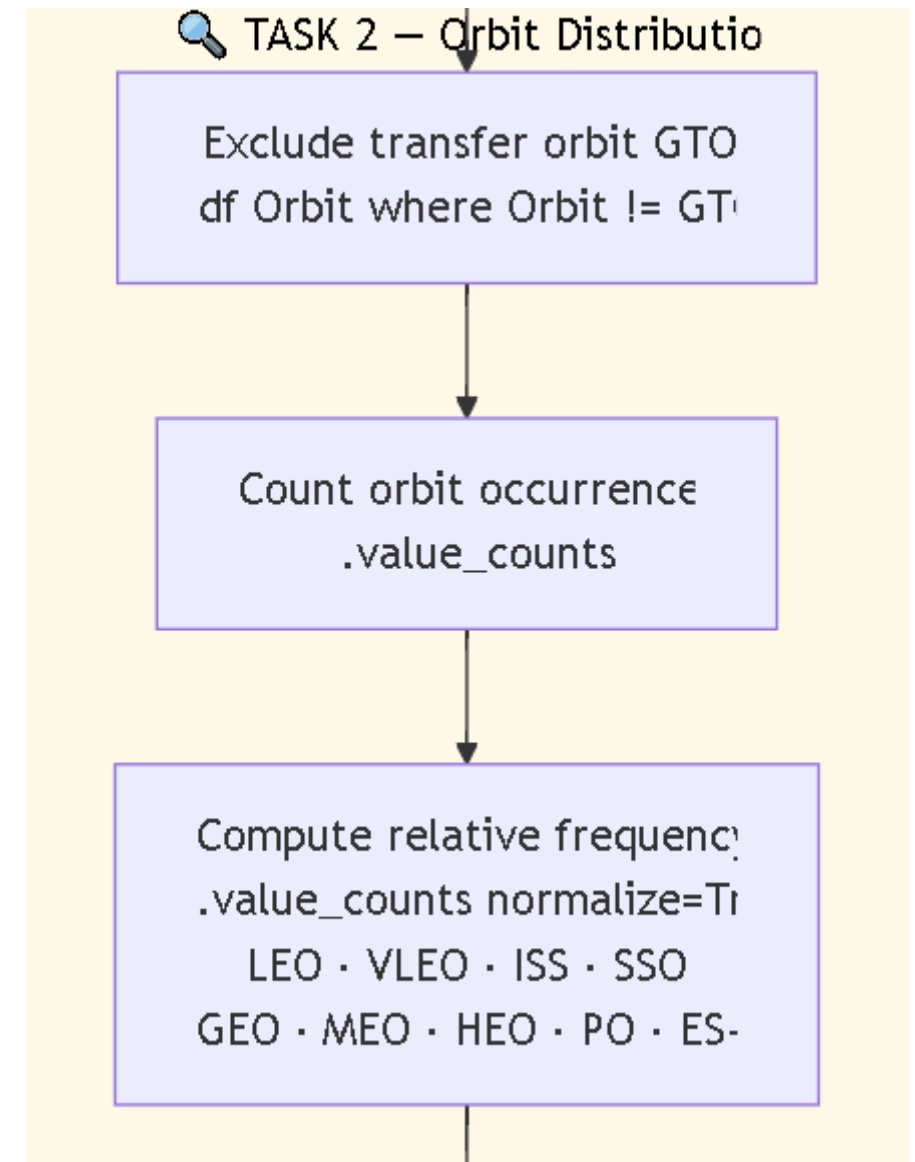
LaunchSite	Launches
CCAFS SLC 40	55
KSC LC 39 ^a	22
VAFB SLC 4E	13



Data Wrangling (4/6)

- Analysis of Launches by Orbit Type

Orbit	Launches	%
ISS	21	33.33
VLEO	14	22.22
PO	9	14.29
LEO	7	11.11
SSO	5	7.94
MEO	3	4.76
ES-L1	1	1.59
HEO	1	1.59
SO	1	1.59
GEO	1	1.59



Data Wrangling (5/6)

Index	Outcome	Launches	%
0	True ASDS	41	45.56
1	None None	19	21.11
2	True RTLS	14	15.56
3	False ASDS	6	6.67
4	True Ocean	5	5.56
5	False Ocean	2	2.22
6	None ASDS	2	2.22
7	False RTLS	1	1.11

```
bad_outcomes=set(landing_outcomes.keys()[[1,3,5,6,7]])
```

TASK 3 – Mission Outcome Analysis

Count outcome occurrences
`landing_outcomes = df Outcome`
`.value_counts`

Compute outcome percenta
`.value_counts normalize=True`

Enumerate outcome keys
True·False ASDS
True·False RTLS
True·False Ocean
None ASDS · None None

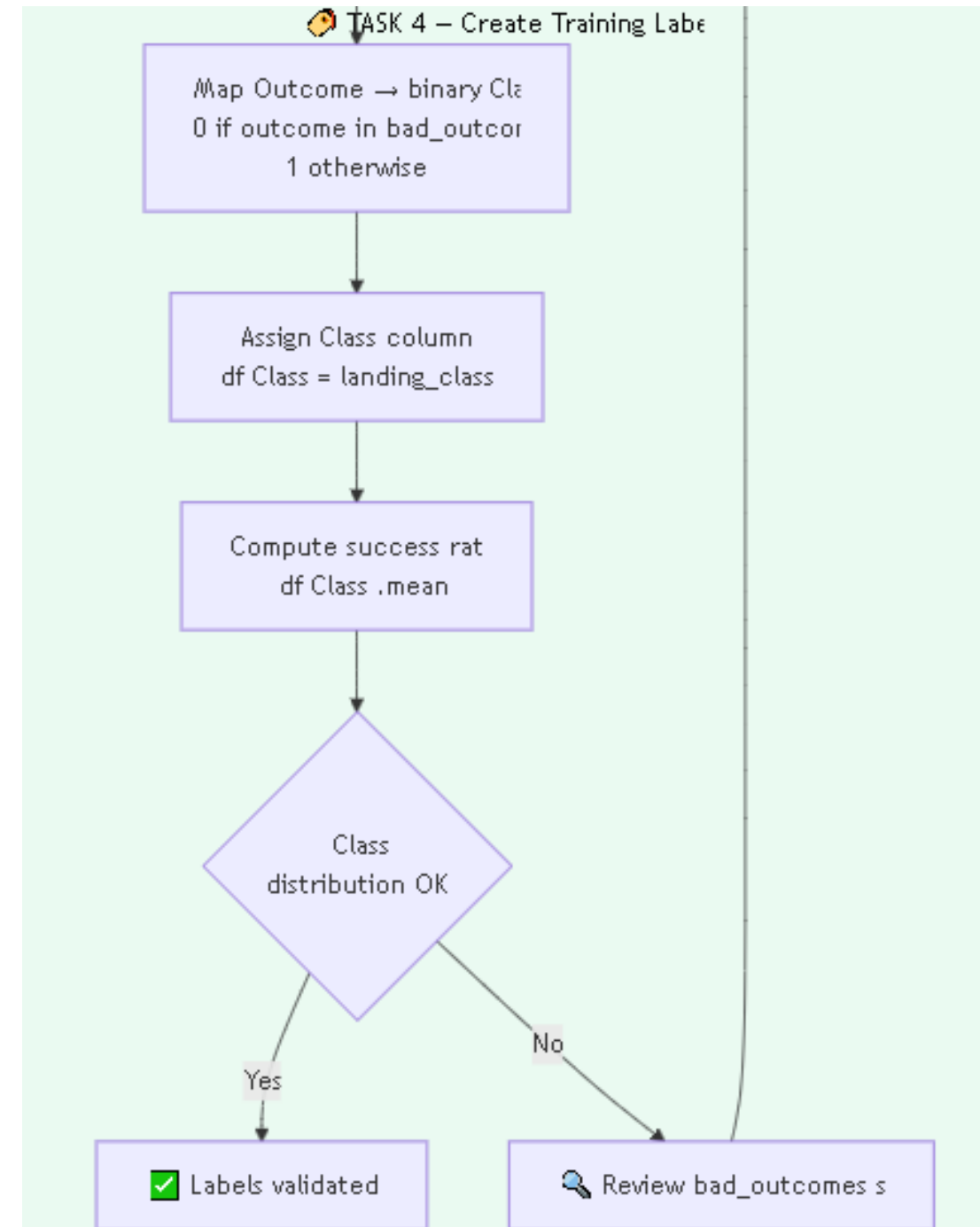
Define bad_outcomes set
indices 1 · 3 · 5 · 6 · 7
unsuccessful or no landing

Data Wrangling (6/6)

```
# landing_class = 0 if bad outcome  
# landing_class = 1 otherwise  
landing_class = [0 if outcome in  
bad_outcomes else 1 for outcome in  
df['Outcome']]
```

```
df['Class']=landing_class
```

```
df["Class"].mean() #0,66% success rate
```



EDA with Data Visualization

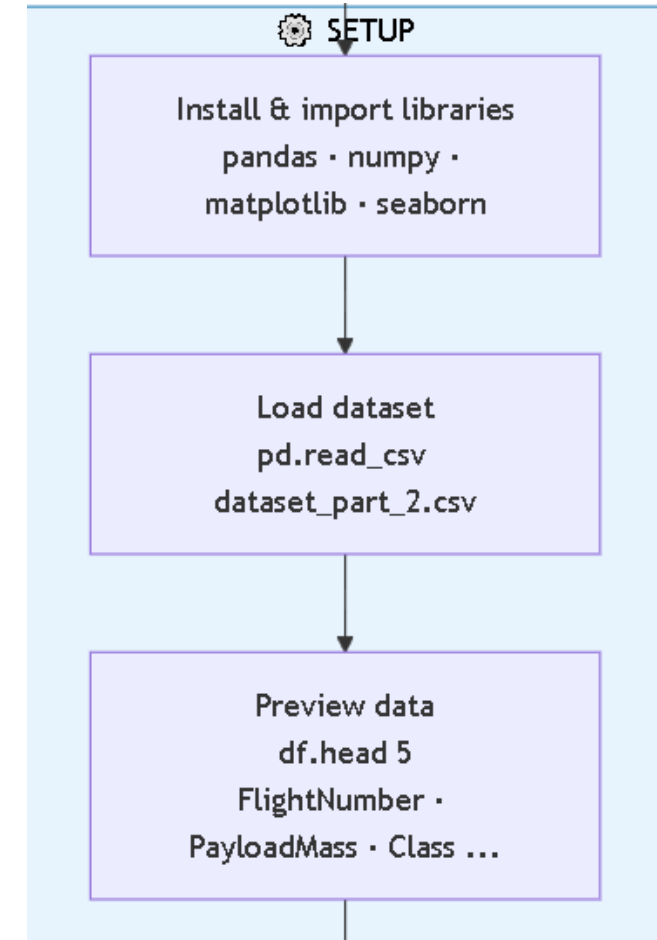
- Summarize what charts were plotted and why you used those charts
- Add the GitHub URL of your completed EDA with data visualization notebook, as an external reference and peer-review purpose

EDA with Data Visualization (1/9)

- The Exploratory data analysis with data visualization process is presented in the following twelve slides.
- [The Visualization Exploratory-data-analysis notebook](#) is provided as an external reference for transparency and peer-review purposes

EDA with Data Visualization (1/3)

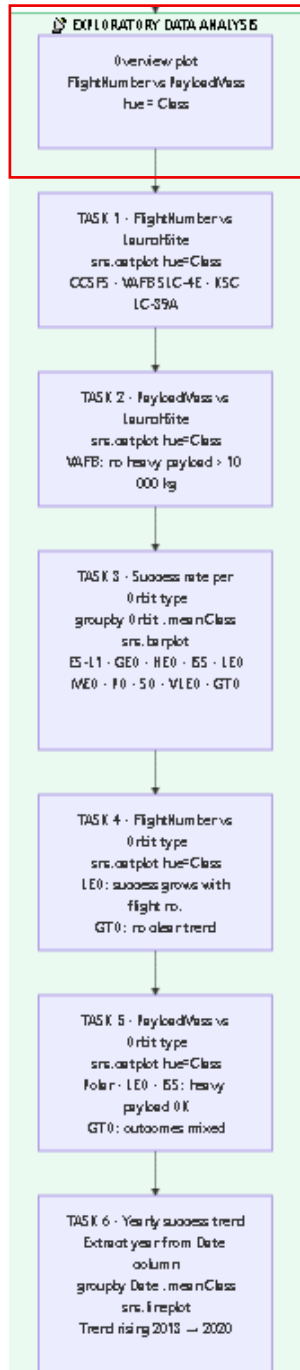
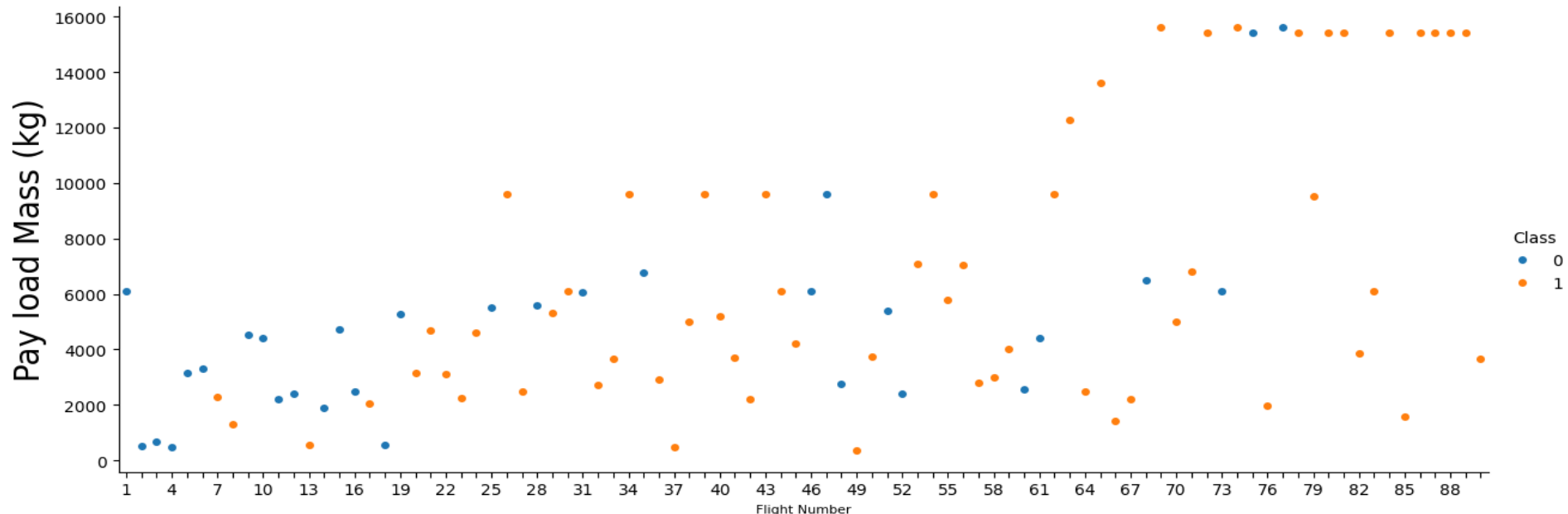
- Install the required packages, Import the necessary libraries, and load an IBM-curated dataset that enables all students to replicate the results.



EDA with Data Visualization (2a/3)

```
g= sns.catplot(y="PayloadMass", x="FlightNumber", hue="Class", data=df, aspect = 2.5)
plt.xlabel("Flight Number",fontsize=8)
plt.ylabel("Pay load Mass (kg)",fontsize=20)
ax = g.axes[0, 0]
for i, label in enumerate(ax.get_xticklabels()):
    if i % 3 != 0:
        label.set_visible(False)
plt.show()
```

As the flight number increases (more experience) , the first stage is more likely to land successfully. The payload mass also appears to be a factor; even with more massive payloads, the first stage often returns successfully.



EDA with Data Visualization (2b/3)

```
# Plot a scatter point chart with x axis to be Flight Number and y axis to be the launch site
# (Categorical), and hue to be the class value
```

```
T=sns.catplot(y="LaunchSite", x="FlightNumber", hue="Class", data=df, aspect = 2.5)
```

```
plt.xlabel("Flight Number",fontsize=8)
```

```
plt.ylabel("Launch Site",fontsize=20)
```

```
ax = T.axes[0, 0]
```

```
# Manually set one tick per flightnumber :Y is categorical
```

```
flight_numbers = sorted(df["FlightNumber"].unique())
```

```
ax.set_xticks(flight_numbers)
```

```
ax.set_xticklabels(flight_numbers, fontsize=7)
```

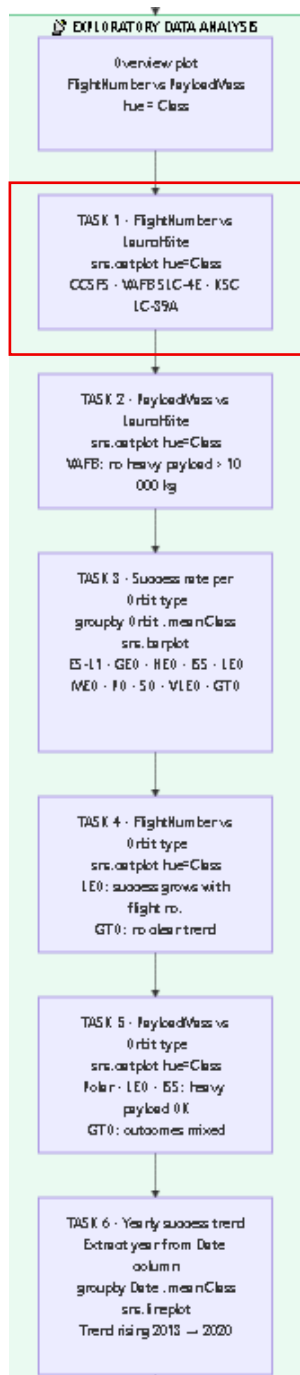
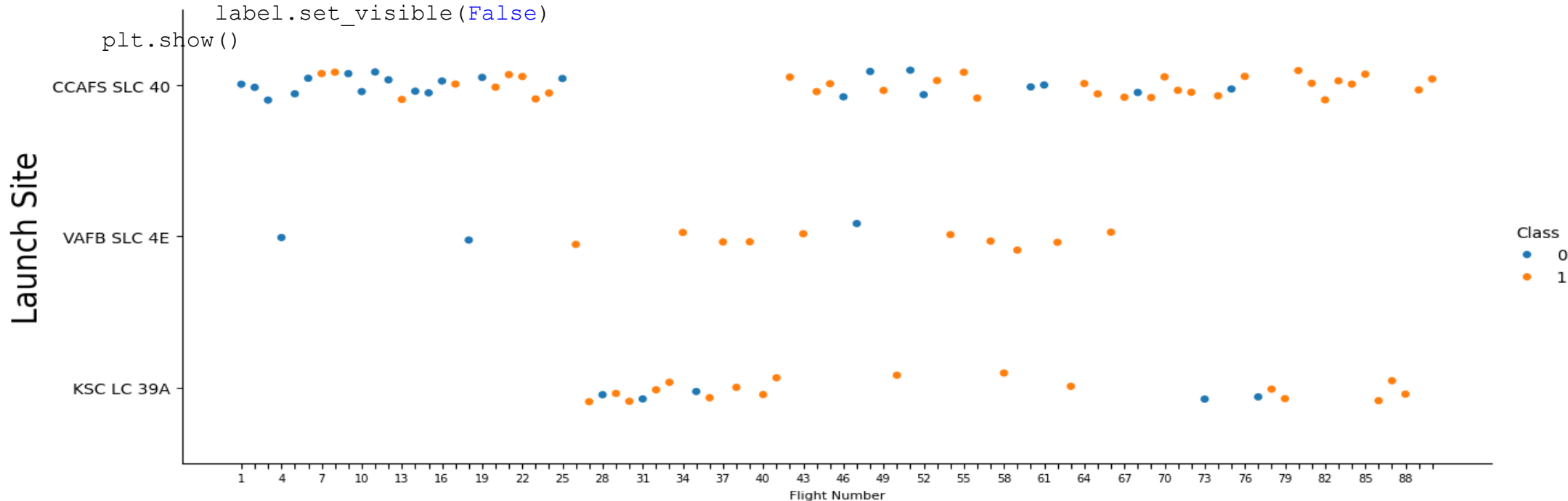
```
# Now filter to show every 3rd label
```

```
for i, label in enumerate(ax.get_xticklabels()):
```

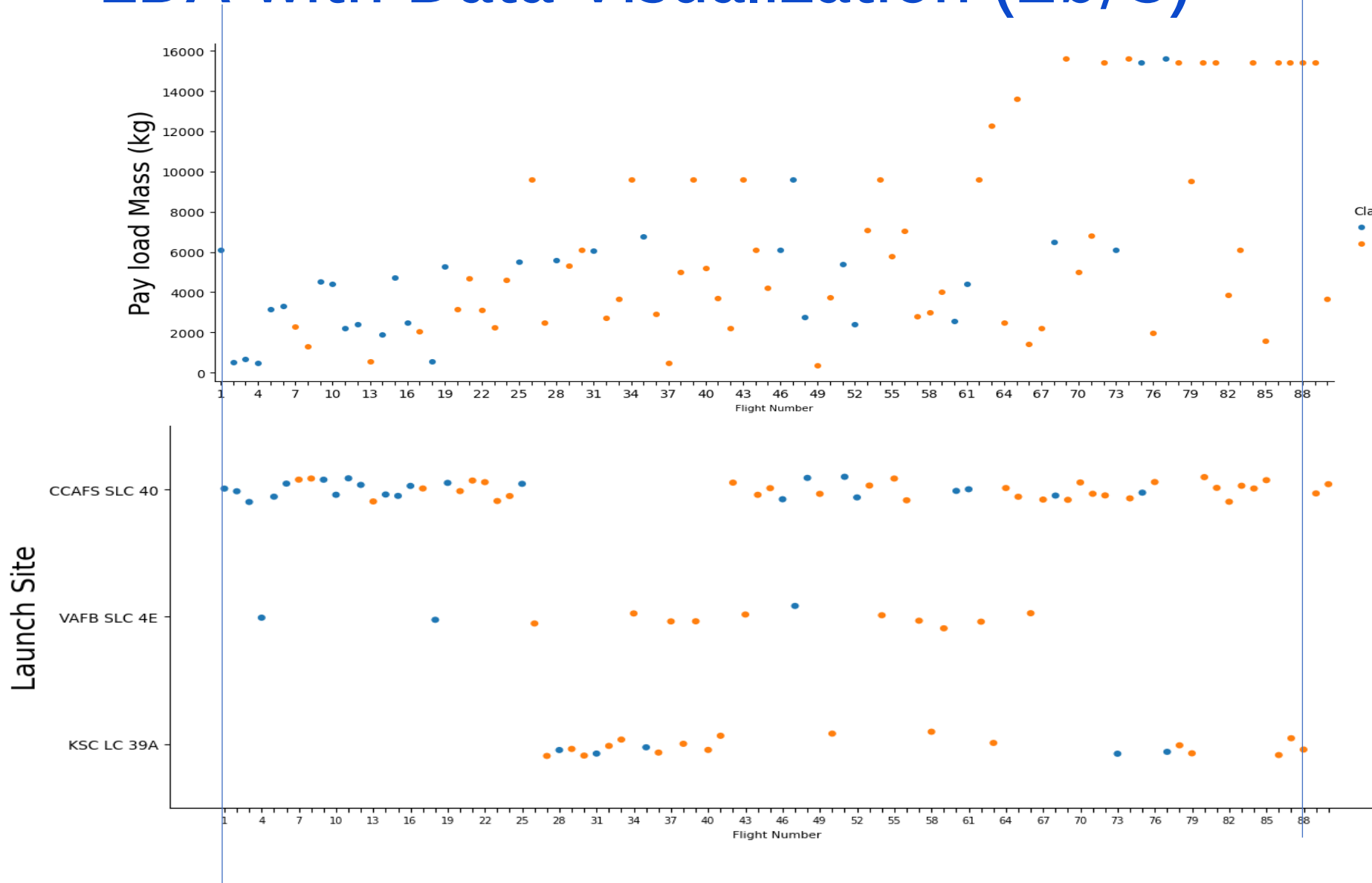
```
    if i % 3 != 0:
```

```
        label.set_visible(False)
```

```
plt.show()
```

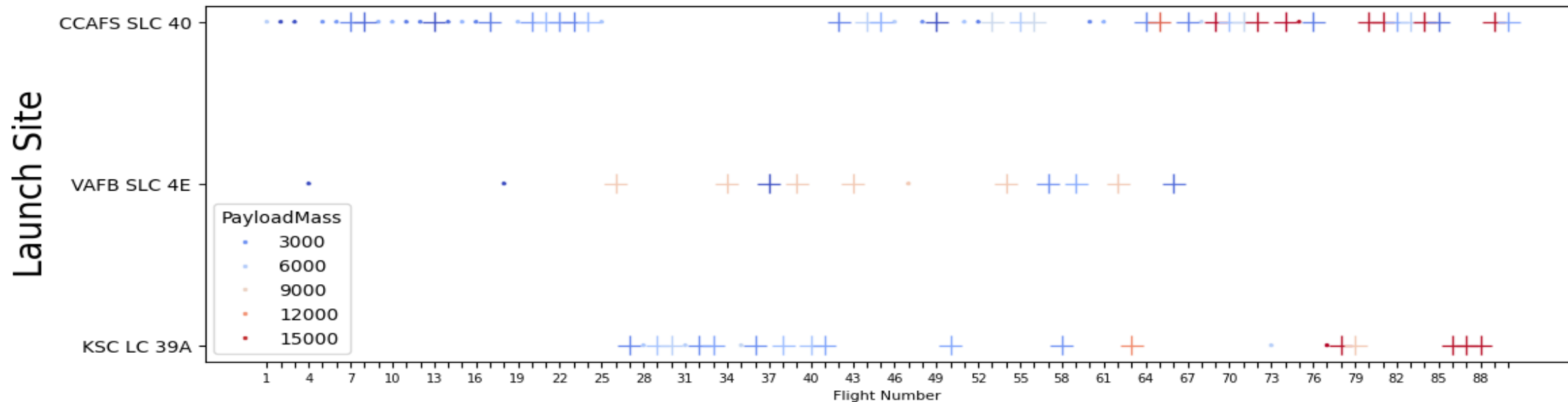


EDA with Data Visualization (2b/3)



EDA with Data Visualization (2b/3)

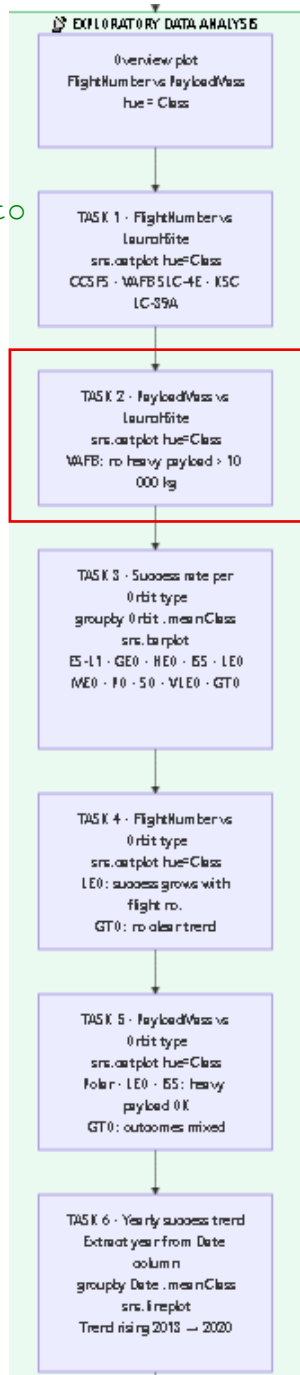
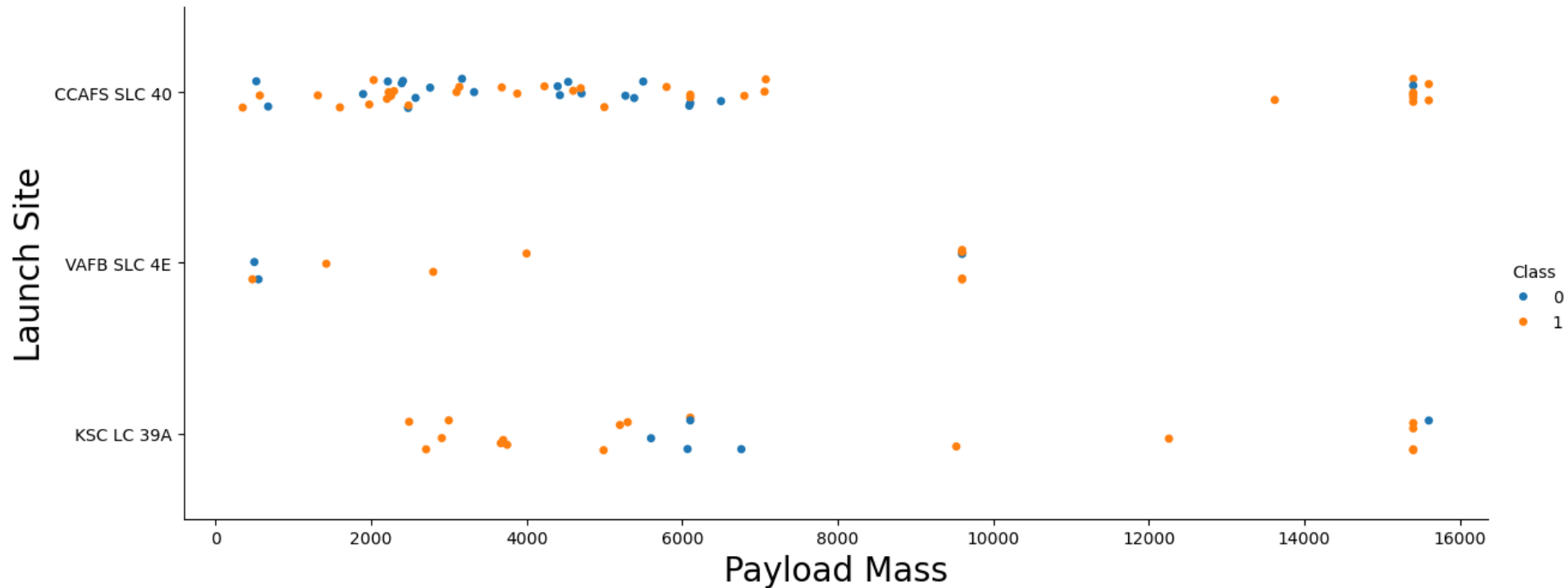
```
fig, ax = plt.subplots(figsize=(13, 4))
# Class 0 - circle marker
sns.scatterplot(data=df[df["Class"] == 0], x="FlightNumber", y="LaunchSite", hue="PayloadMass", marker=".", palette="coolwarm", legend="brief", ax=ax)
# Class 1 - star marker
sns.scatterplot(data=df[df["Class"] == 1], x="FlightNumber", y="LaunchSite", hue="PayloadMass", marker="+", palette="coolwarm", legend=False, ax=ax, s=150)
plt.xlabel("Flight Number", fontsize=8)
plt.ylabel("Launch Site", fontsize=20)
# One tick per flight number, every 3rd visible
flight_numbers = sorted(df["FlightNumber"].unique())
ax.set_xticks(flight_numbers)
ax.set_xticklabels(flight_numbers, fontsize=7)
for i, label in enumerate(ax.get_xticklabels()):
    if i % 3 != 0:
        label.set_visible(False)
plt.show()
```



EDA with Data Visualization (2c/3)

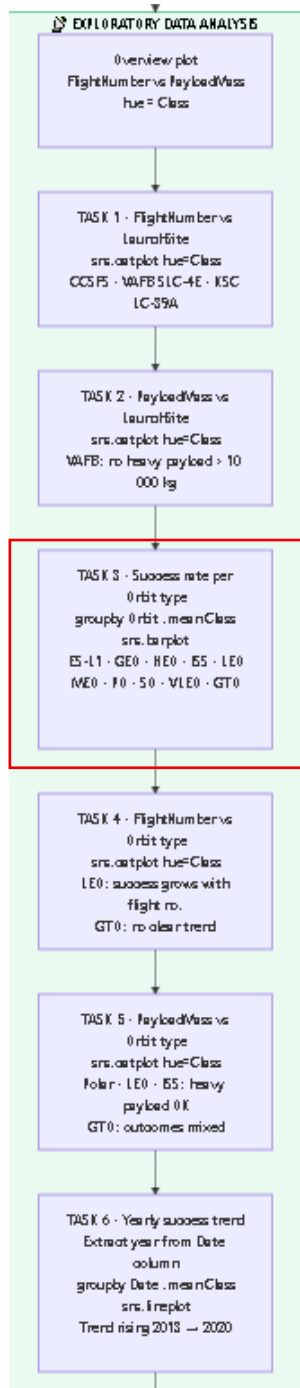
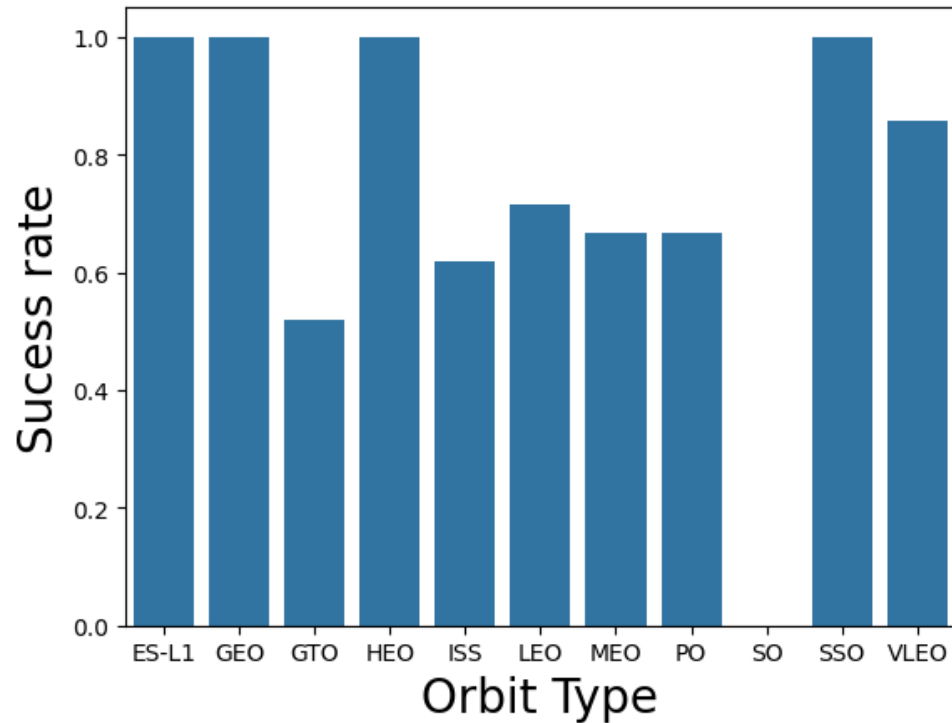
Plot a scatter point chart with x axis to be Pay Load Mass (kg) and y axis to be the launch site, and hue to be the class value

```
sns.catplot(y="LaunchSite", x="PayloadMass", hue="Class", data=df, aspect = 2.5)  
plt.xlabel("Payload Mass", fontsize=20)  
plt.ylabel("Launch Site", fontsize=20)  
plt.show()
```



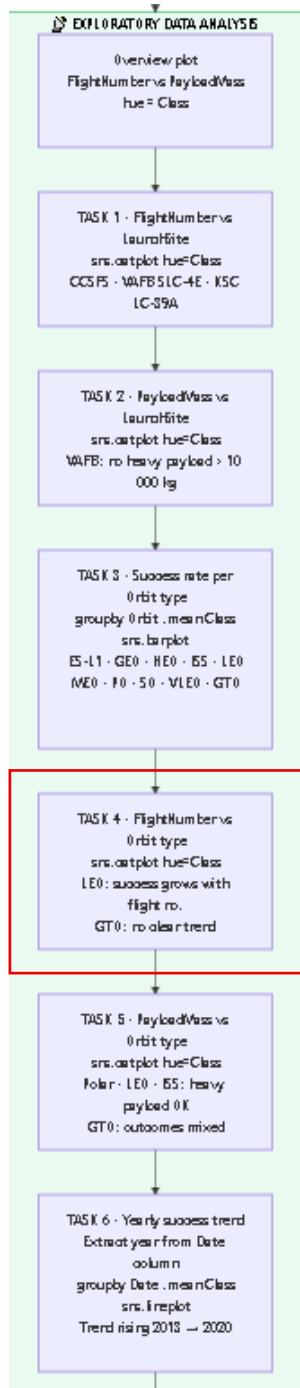
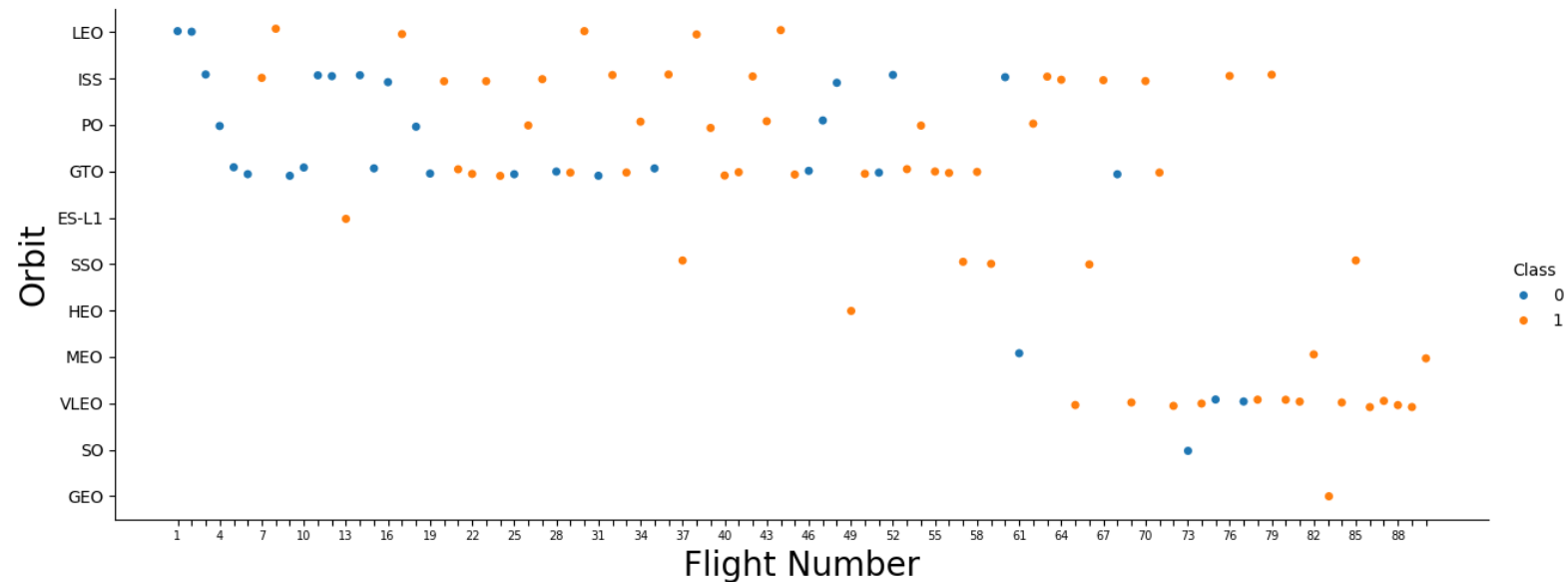
EDA with Data Visualization (2d/3)

```
# HINT use groupby method on Orbit column and get the mean of Class column
df_temp = df[['Orbit', 'Class']]
df_plot = df_temp.groupby(['Orbit']).mean(['Class'])
sns.barplot(x='Orbit', y='Class', data = df_plot)
plt.xlabel("Orbit Type", fontsize=20)
plt.ylabel("Sucess rate", fontsize=20)
plt.show()
```



EDA with Data Visualization (2e/3)

```
# Plot a scatter point chart with x axis to be FlightNumber and y axis to be the Orbit, and hue to be the class value
T=sns.catplot(y="Orbit", x="FlightNumber", hue="Class", data=df, aspect = 2.5)
plt.xlabel("Flight Number", fontsize=20)
plt.ylabel("Orbit", fontsize=20)
ax = T.axes[0, 0]
# Manually set one tick per flight number
flight_numbers = sorted(df["FlightNumber"].unique())
ax.set_xticks(flight_numbers)
ax.set_xticklabels(flight_numbers, fontsize=7)
# Now filter to show every 3rd label
for i, label in enumerate(ax.get_xticklabels()):
    if i % 3 != 0:
        label.set_visible(False)
plt.show()
```

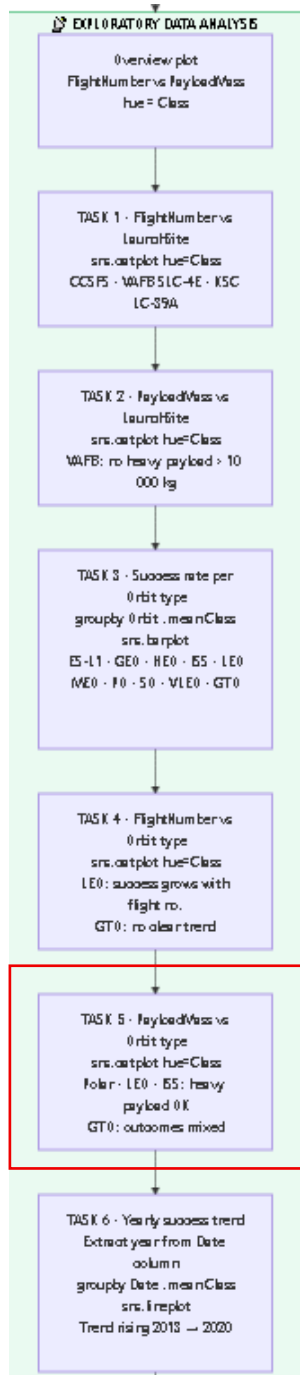
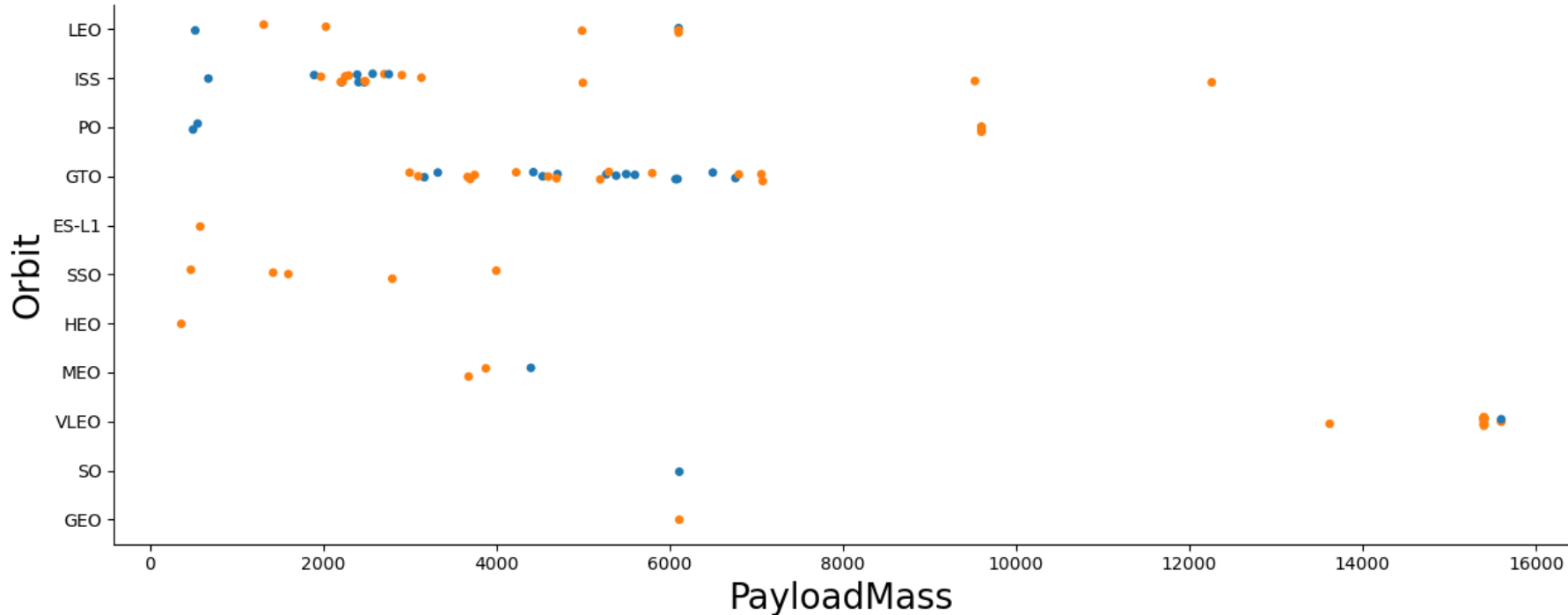


EDA with Data Visualization (2f/3)

```
# Plot a scatter point chart with x axis to be Payload Mass and y axis to be the Orbit, and hue to be the class value
```

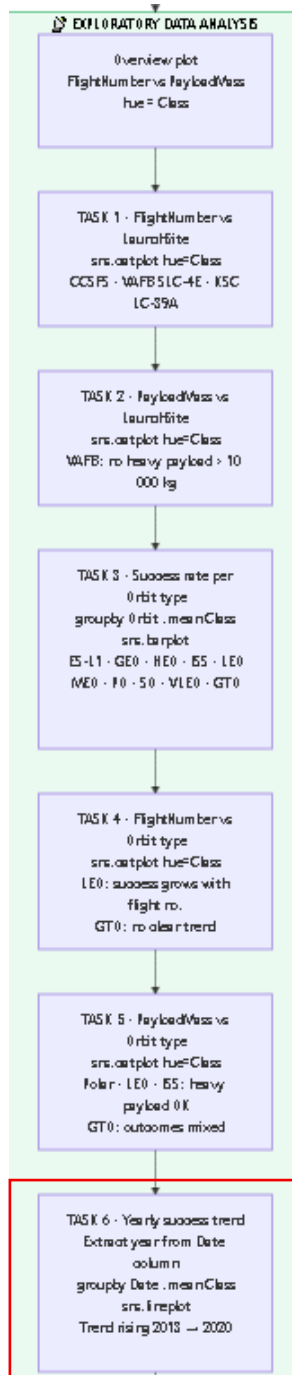
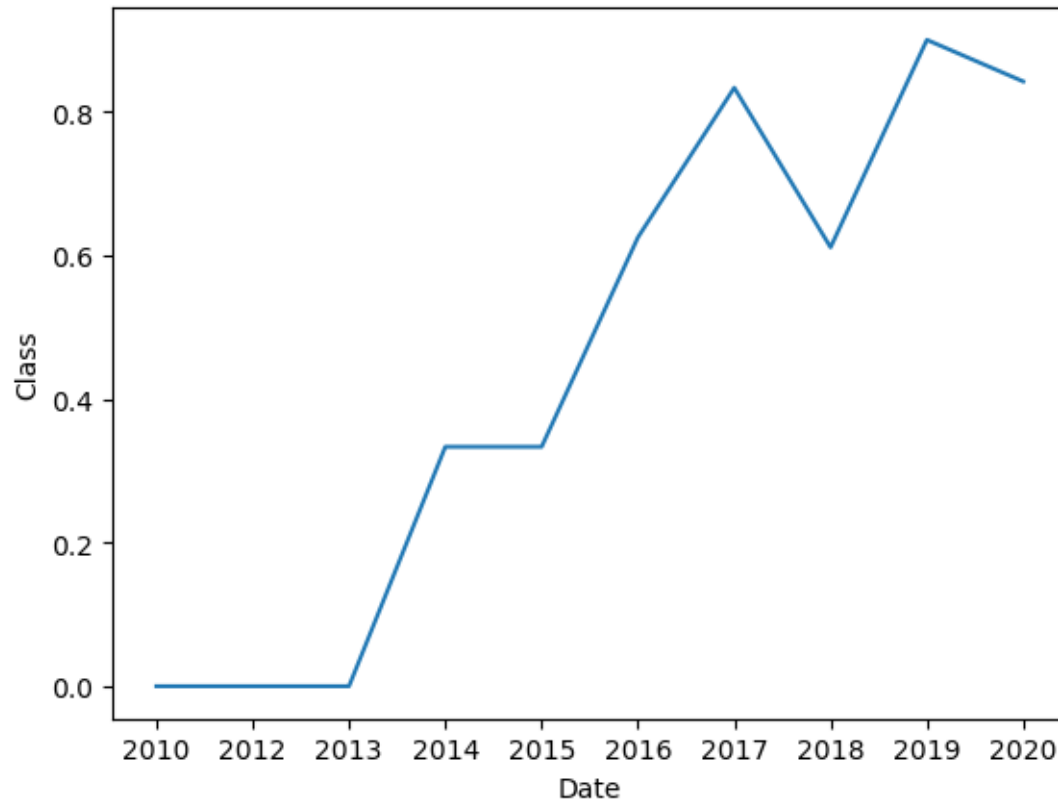
```
# Plot a scatter point chart with x axis to be FlightNumber and y axis to be the Orbit, and hue to be the class value
```

```
sns.catplot(y="Orbit", x="PayloadMass", hue="Class", data=df, aspect = 2.5)
plt.xlabel("PayloadMass", fontsize=20)
plt.ylabel("Orbit", fontsize=20)
plt.show()
```



EDA with Data Visualization (2g/3)

```
# Plot a line chart with x axis to be the extracted year and y axis to be the success rate
df_temp = df[['Date', 'Class']]
df_plot = df_temp.groupby('Date').mean('Class')
sns.lineplot(df_plot, x='Date', y='Class')
```



EDA with Data Visualization (3/3)

```
features = df[['FlightNumber', 'PayloadMass', 'Orbit', 'LaunchSite', 'Flights', 'GridFins',  
'Reused', 'Legs', 'LandingPad', 'Block', 'ReusedCount', 'Serial']]  
features.head()
```

```
# HINT: Use get_dummies() function on the categorical columns  
features_one_hot = pd.get_dummies(features, columns=['Orbit', 'LaunchSite', 'LandingPad',  
'Serial'])  
features_one_hot.head()
```

```
# HINT: use astype function  
features_one_hot = features_one_hot.astype(np.float64)  
features_one_hot.info()
```

🔧 FEATURE ENGINEERING

Select feature columns
FlightNumber ·
PayloadMass · Orbit
LaunchSite · Flights ·
GridFins
Reused · Legs · LandingPad
Block · ReusedCount · Serial

TASK 7 · One-Hot Encode
categoricals
pd.get_dummies on
Orbit · LaunchSite ·
LandingPad · Serial
→ features_one_hot

TASK 8 · Cast to float64
features_one_hot.astype
np.float64
ensures ML-ready numeric
matrix

EDA with Data Visualization (3/3)

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 90 entries, 0 to 89

Data columns (total 80 columns):

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

0	FlightNumber	90 non-null	float64	
1	PayloadMass	90 non-null	float64	
2	Flights	90 non-null	float64	
3	GridFins	90 non-null	float64	
4	Reused	90 non-null	float64	
5	Legs	90 non-null	float64	
6	Block	90 non-null	float64	
7	ReusedCount	90 non-null	float64	
8	Orbit_ES-L1	90 non-null	float64	
9	Orbit_GEO	90 non-null	float64	
10	Orbit_GTO	90 non-null	float64	
11	Orbit_HEO	90 non-null	float64	
12	Orbit_ISS	90 non-null	float64	
13	Orbit_LEO	90 non-null	float64	
14	Orbit_MEO	90 non-null	float64	
15	Orbit_PO	90 non-null	float64	
16	Orbit_SO	90 non-null	float64	
17	Orbit_SSO	90 non-null	float64	
18	Orbit_VLEO	90 non-null	float64	
19	LaunchSite_CCAFS SLC 40	90 non-null	float64	
20	LaunchSite_KSC LC 39A	90 non-null	float64	
21	LaunchSite_VAFB SLC 4E	90 non-null	float64	
22	LandingPad_5e9e3032383ecb267a34e7c7	90 non-null	float64	
23	LandingPad_5e9e3032383ecb554034e7c9	90 non-null	float64	
24	LandingPad_5e9e3032383ecb6bb234e7ca	90 non-null	float64	
25	LandingPad_5e9e3032383ecb761634e7cb	90 non-null	float64	
26	LandingPad_5e9e3033383ecbb9e534e7cc	90 non-null	float64	
27	Serial_B0003	90 non-null	float64	
28	Serial_B0005	90 non-null	float64	
29	Serial_B0007	90 non-null	float64	
30	Serial_B1003	90 non-null	float64	

31 Serial_B1004
32 Serial_B1005
33 Serial_B1006
34 Serial_B1007
35 Serial_B1008
36 Serial_B1010
37 Serial_B1011
38 Serial_B1012
39 Serial_B1013
40 Serial_B1015
41 Serial_B1016
42 Serial_B1017
43 Serial_B1018
44 Serial_B1019
45 Serial_B1020
46 Serial_B1021
47 Serial_B1022
48 Serial_B1023
49 Serial_B1025
50 Serial_B1026
51 Serial_B1028
52 Serial_B1029
53 Serial_B1030
54 Serial_B1031
55 Serial_B1032
56 Serial_B1034
57 Serial_B1035
58 Serial_B1036
59 Serial_B1037
60 Serial_B1038
61 Serial_B1039
62 Serial_B1040
63 Serial_B1041
64 Serial_B1042
65 Serial_B1043
66 Serial_B1044

[illegible]

67 Serial_B1045
68 Serial_B1046
69 Serial_B1047
70 Serial_B1048
71 Serial_B1049
72 Serial_B1050
73 Serial_B1051
74 Serial_B1054
75 Serial_B1056
76 Serial_B1058
77 Serial_B1059
78 Serial_B1060
79 Serial_B1062

```
dtypes: float64(80)
memory usage: 56.3 KB
```

[illegible]

EDA with SQL (1/4)

- The Exploratory data analysis with SQL process is presented in the following three slides.
- The [EDA with SQL notebook](#) is provided as an external reference for transparency and peer-review purposes

EDA with SQL (2/4)

- **%sql** select distinct Launch_Site from SPACEXTABLE

Launch_Site

CCAFS LC-40

VAFB SLC-4E

KSC LC-39A

CCAFS SLC-40

- **%sql** select * from SPACEXTABLE WHERE Launch_Site like "CCA%" LIMIT 5

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__K_G_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

- **%sql** select Customer , sum(PAYLOAD_MASS__KG_) from SPACEXTABLE WHERE Customer like "NASA (CRS)" group by Customer

Customer	sum(PAYLOAD_MASS__KG_)
NASA (CRS)	45596

EDA with SQL (3/4)

- **%sql** select Booster_Version , avg(PAYLOAD_MASS__KG_) from SPACEXTABLE WHERE Booster_Version like "F9 v1.1" group by Booster_Version

Booster_Version	avg(PAYLOAD_MASS__KG_)
F9 v1.1	2928.4

- **%sql** select Landing_Outcome , min(Date) from SPACEXTABLE where Landing_Outcome like "%ground%" group by Landing_Outcome

Landing_Outcome	min(Date)
Success (ground pad)	2015-12-22

- **%sql** select Booster_Version, PAYLOAD_MASS__KG_, Mission_Outcome, Landing_outcome from SPACEXTABLE where Landing_Outcome like 'Success (drone ship)' and 4000 < PAYLOAD_MASS__KG_ and PAYLOAD_MASS__KG_ < 6000

Booster_Version	PAYLOAD_MASS__KG_	Mission_Outcome	Landing_Outcome
F9 FT B1022	4696	Success	Success (drone ship)
F9 FT B1026	4600	Success	Success (drone ship)
F9 FT B1021.2	5300	Success	Success (drone ship)
F9 FT B1031.2	5200	Success	Success (drone ship)

- **%sql substr** Select substr(Mission_outcome,1 ,7) , count(Mission_Outcome) from SPACEXTABLE group by substr(Mission_Outcome,1,7)

substr(Mission_outcome,1 ,7)	count(Mission_Outcome)
Failure	1
Success	100

EDA with SQL (4/4)

- **%sql** select Booster_Version from SPACEXTABLE WHERE PAYLOAD_MASS__KG_ = (select max(PAYLOAD_MASS__KG_) from SPACEXTABLE) order by Booster_version
- **%sql** select CASE substring(Date, 6,2) WHEN '01' THEN 'January' WHEN '02' THEN 'February' WHEN '03' THEN 'March' WHEN '04' THEN 'April' WHEN '05' THEN 'May' WHEN '06' THEN 'June' WHEN '07' THEN 'July' WHEN '08' THEN 'August' WHEN '09' THEN 'September' WHEN '10' THEN 'October' WHEN '11' THEN 'November' WHEN '12' THEN 'December' END as Month, Booster_Version, Launch_site, Landing_Outcome from SPACEXTABLE where substr(Date,0,5)='2015' and Landing_outcome like 'Failure%'

Month	Booster_Version	Launch_Site	Landing_Outcome
January	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
April	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)

- **%sql** select Landing_Outcome, count(Landing_Outcome) as events from SPACEXTABLE where '2010-06-04' <= Date and Date < '2017-03-20' group by Landing_Outcome order by events desc

Landing_Outcome	events
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

Booster_Version

F9 B5 B1048.4
F9 B5 B1048.5
F9 B5 B1049.4
F9 B5 B1049.5
F9 B5 B1049.7
F9 B5 B1051.3
F9 B5 B1051.4
F9 B5 B1051.6
F9 B5 B1056.4
F9 B5 B1058.3
F9 B5 B1060.2
F9 B5 B1060.3

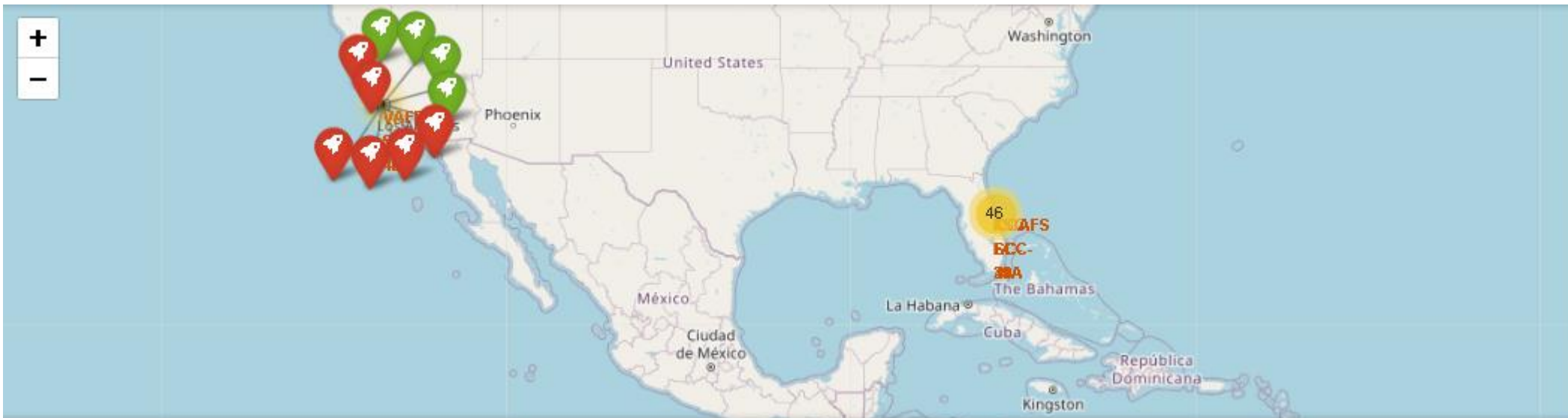
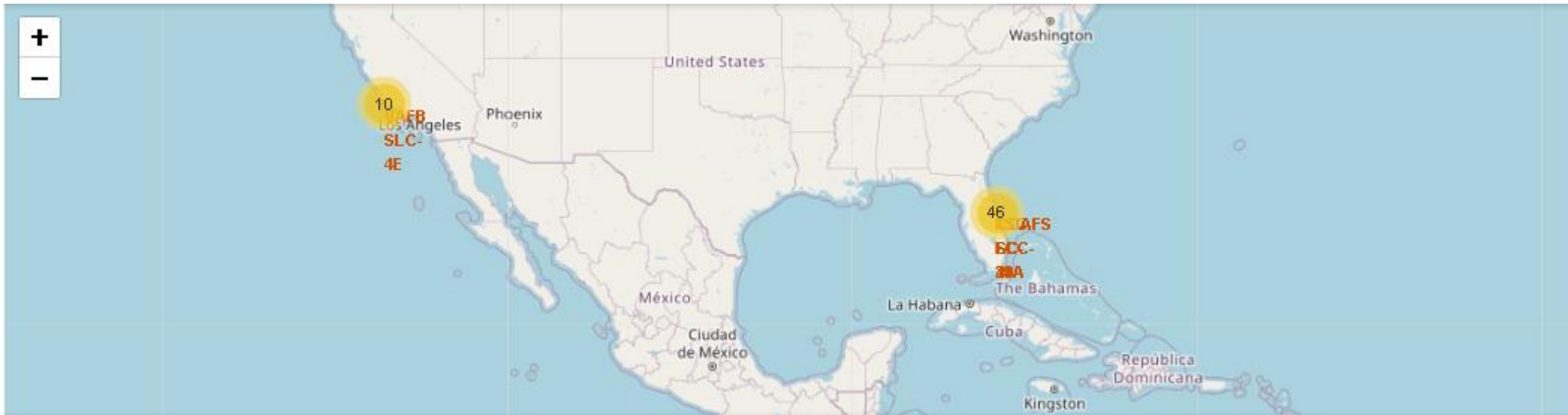
Build an Interactive Map with Folium (1/6)

- I used a Folium.Circle to add a highlighted circle area OF INFLUENCE with a text label on a specific coordinate
- I used a folium.Marker to pinpoint a specific coordinate.
- I used a folium.MarkerCluster() When markers are so close together that they overlap and hide each other in the same LaunchSite.
- I used a folium.PolyLine linking two related markers to show a relationship (distance).
- The [Visual Analytics with Folium notebook](#) is provided as an external reference for transparency and peer-review purposes

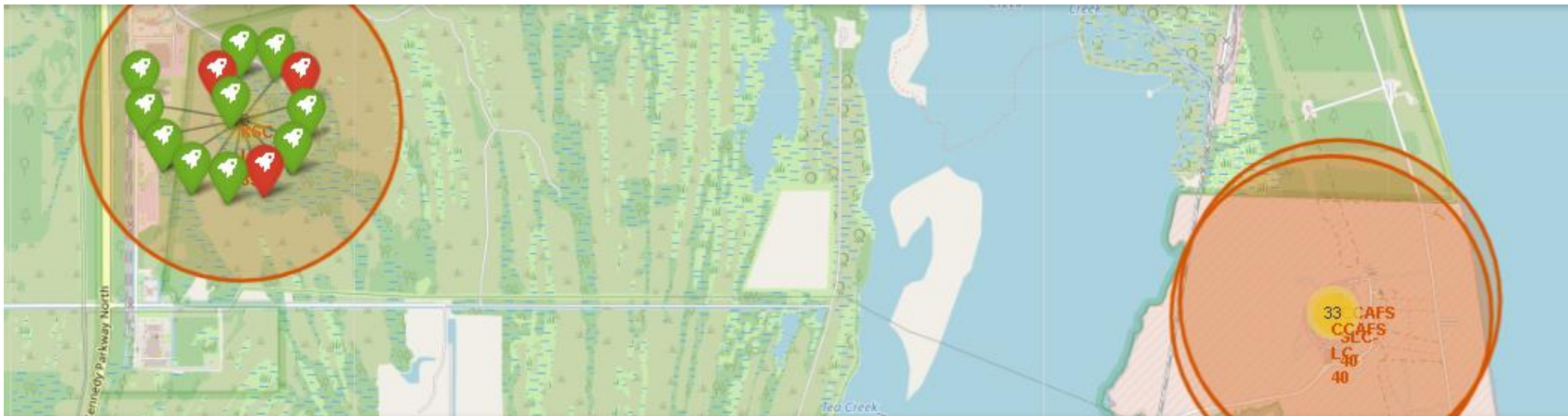
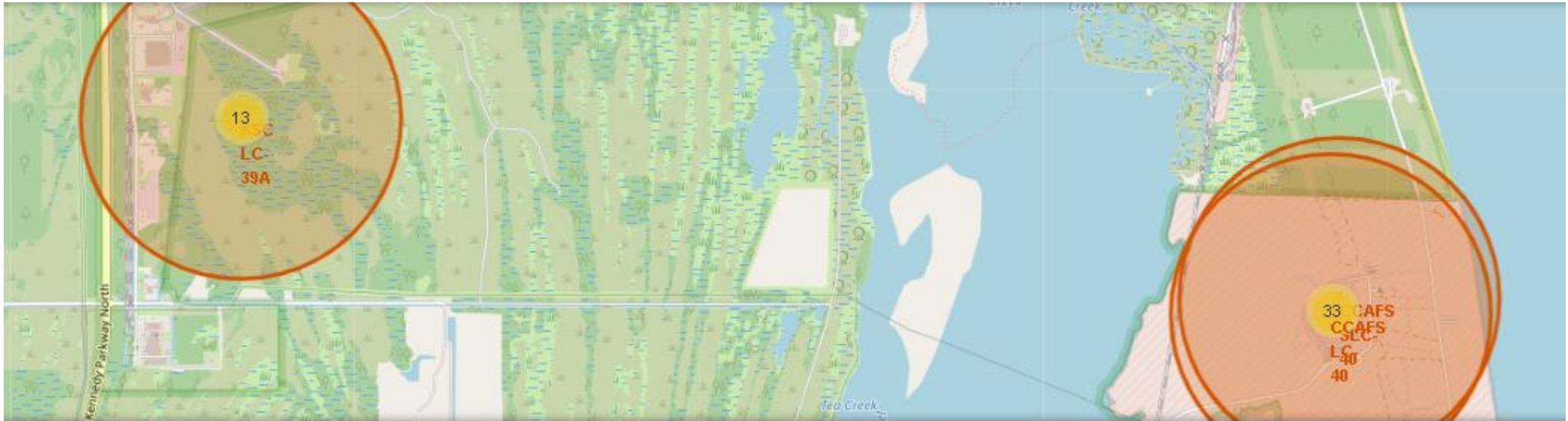
Build an Interactive Map with Folium (2/6)



Build an Interactive Map with Folium (3/6)



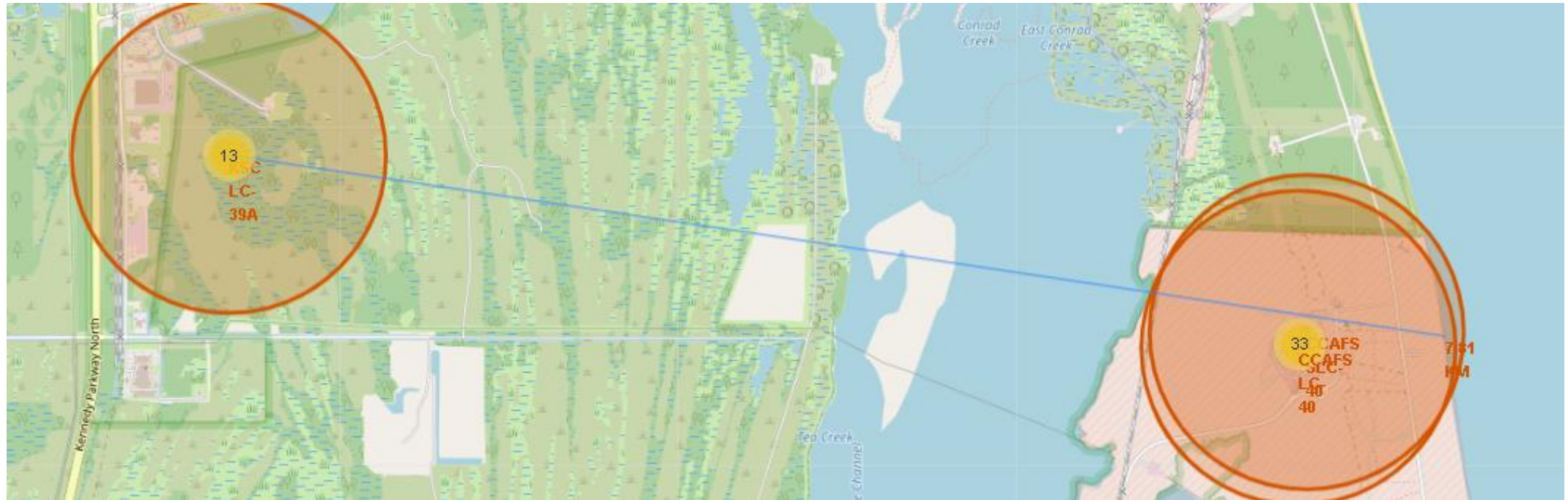
Build an Interactive Map with Folium (4/6)



Build an Interactive Map with Folium (5/6)



Build an Interactive Map with Folium (6/6)

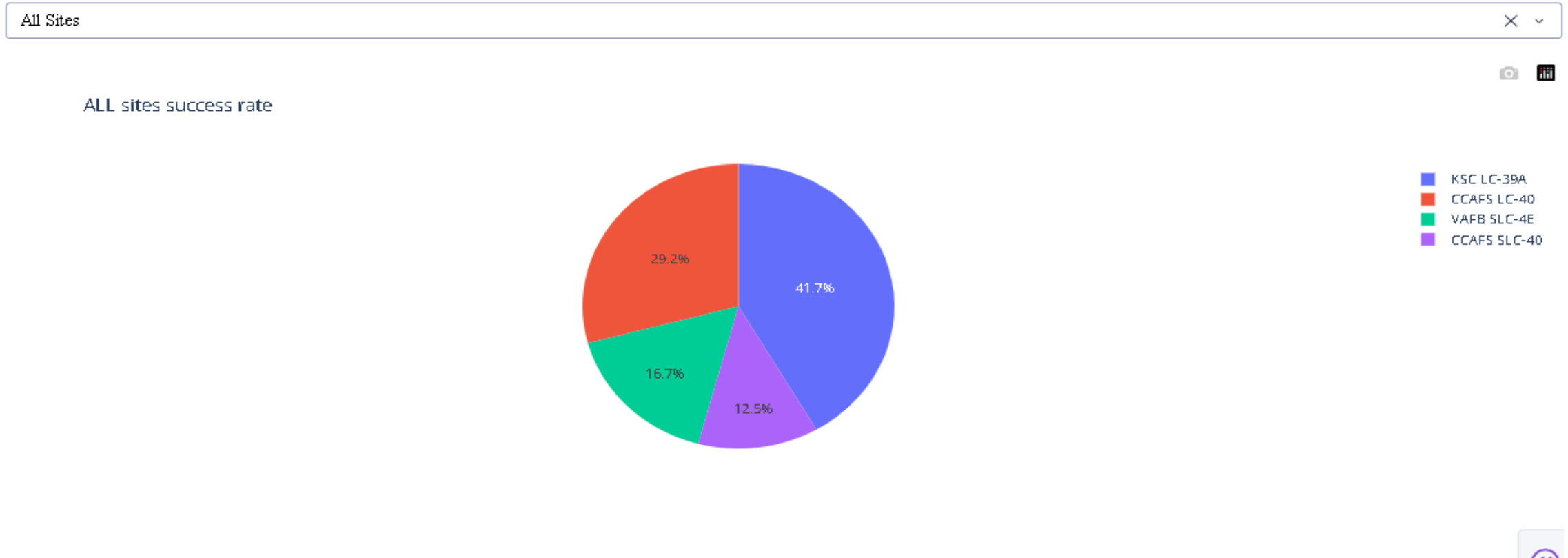


Build a Dashboard with Plotly Dash

- The dashboard features two interactive elements: **LaunchSite** and **PayloadMass** range. The former influences both charts, whereas the latter affects only the final chart.
- The first chart illustrates the proportion of successful outcomes.
- The second chart depicts success as a function of **PayloadMass**, categorised by the booster version. We can visually observe how payload may be correlated with mission outcomes for selected site(s).
- The [Dashboard with Plotly notebook](#) is provided as an external reference for transparency and peer-review purposes

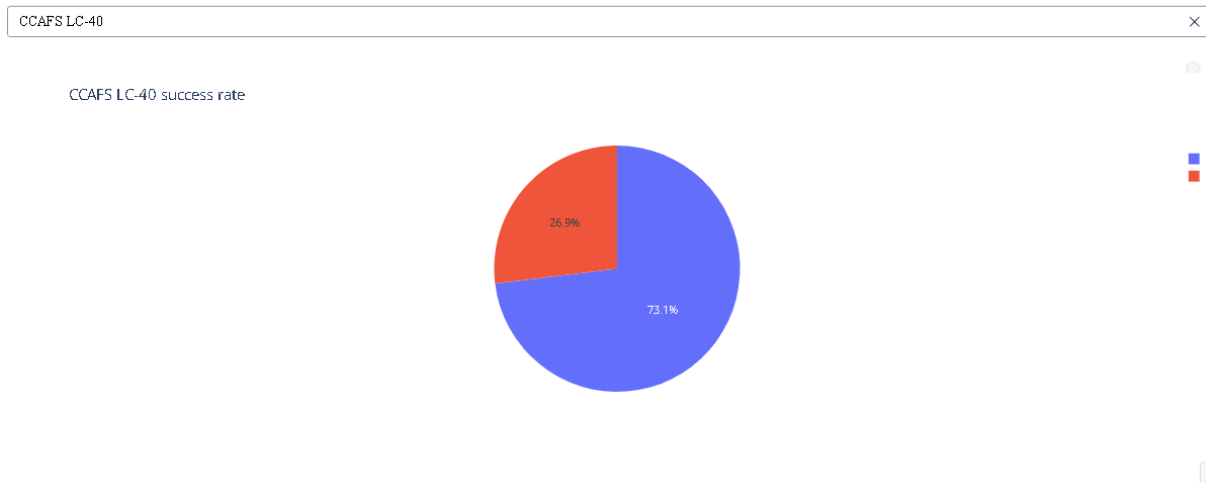
Build a Dashboard with Plotly Dash

SpaceX Launch Records Dashboard

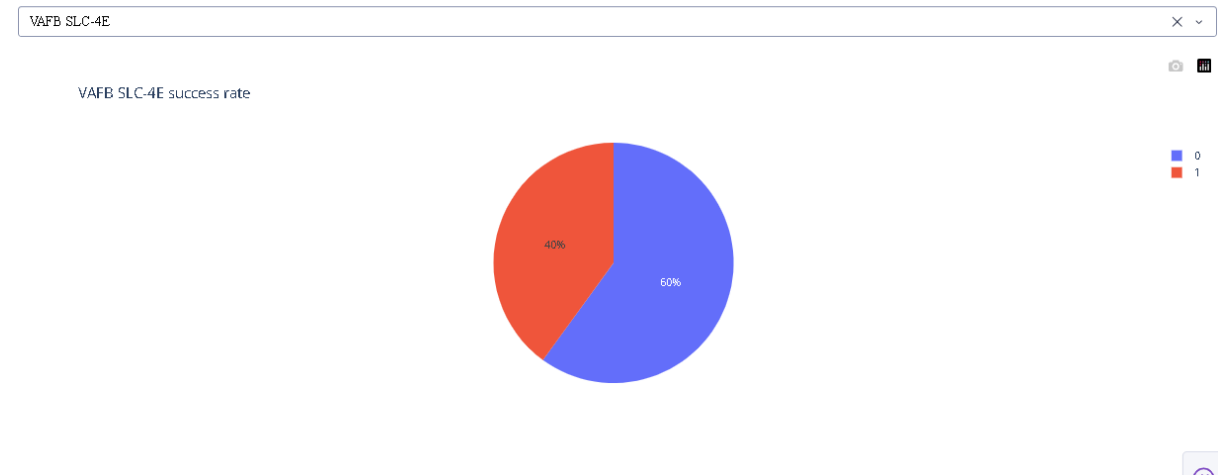


Build a Dashboard with Plotly Dash

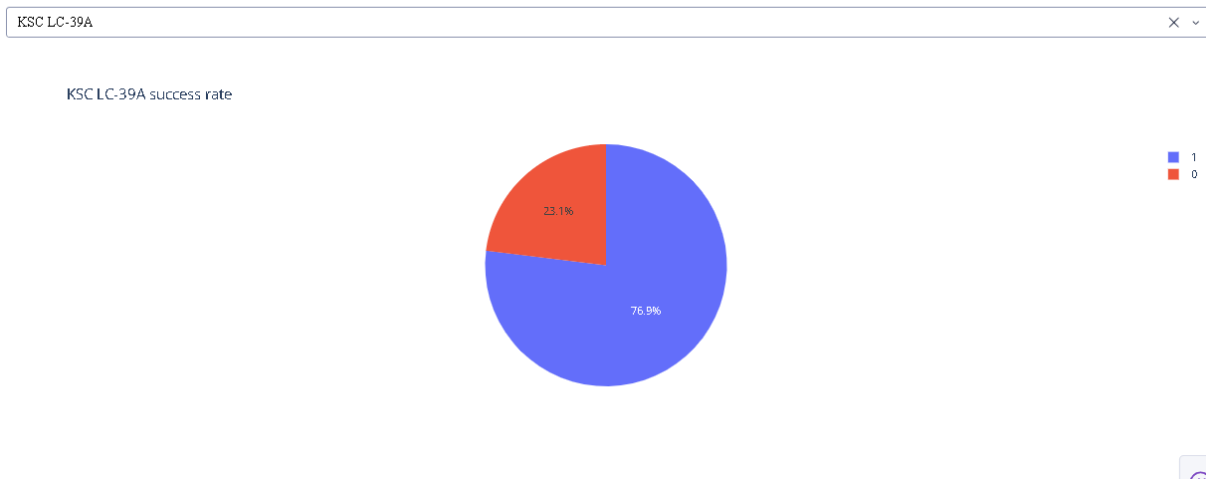
SpaceX Launch Records Dashboard



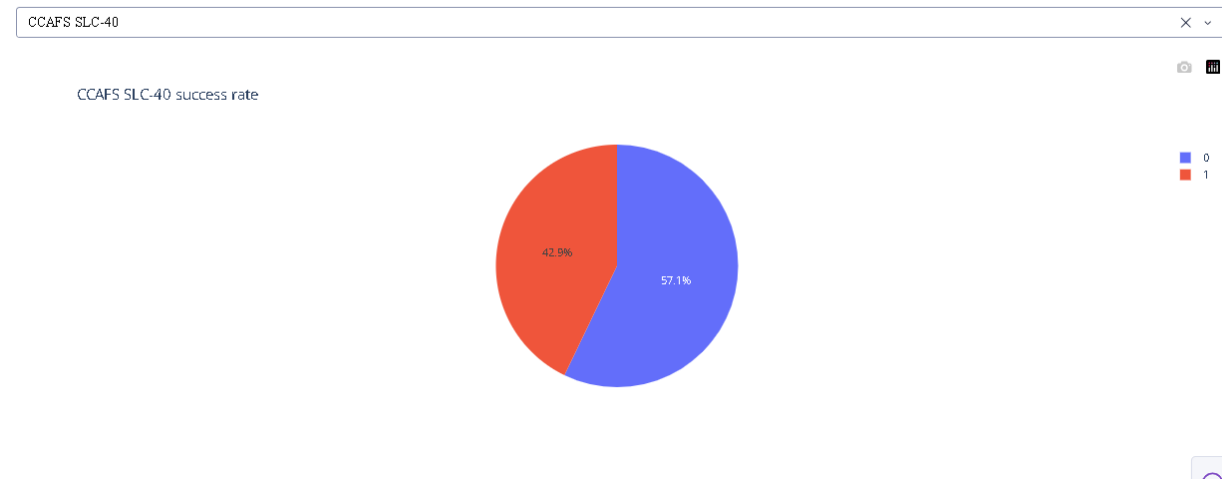
SpaceX Launch Records Dashboard



SpaceX Launch Records Dashboard

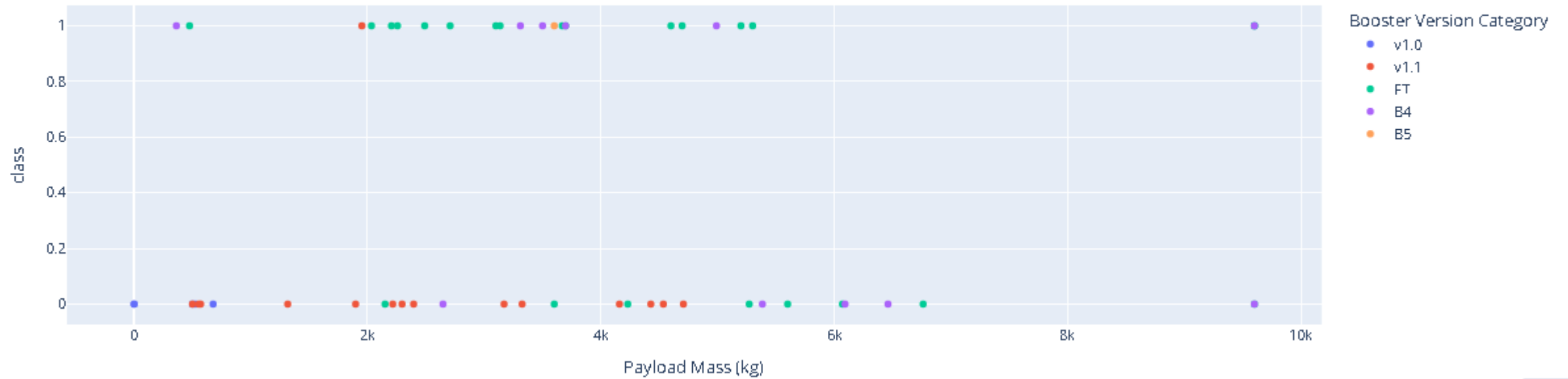


SpaceX Launch Records Dashboard

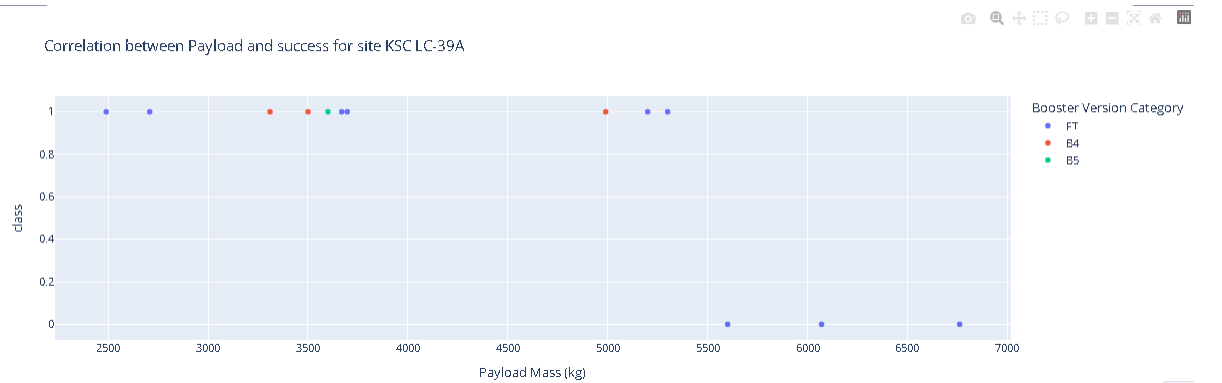
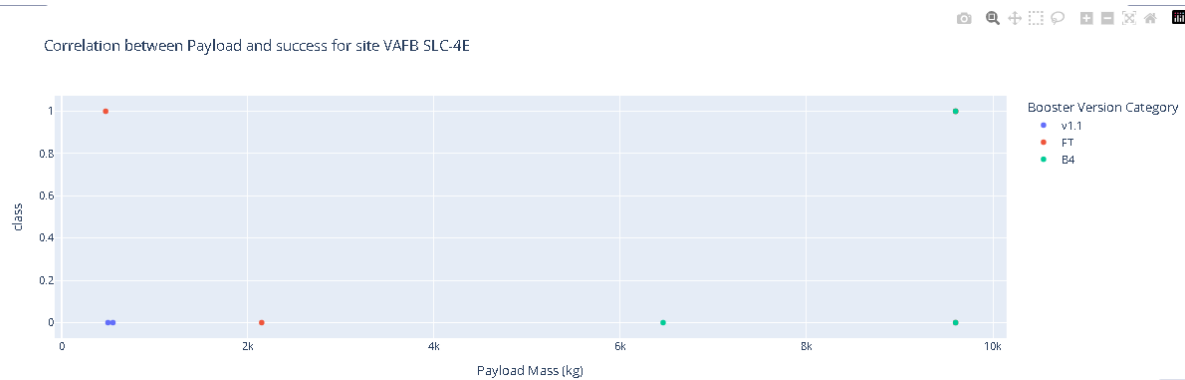
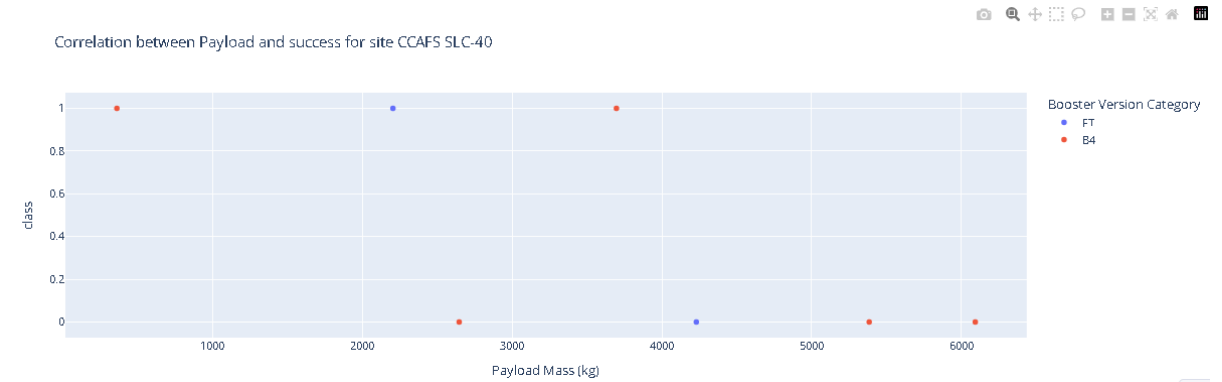
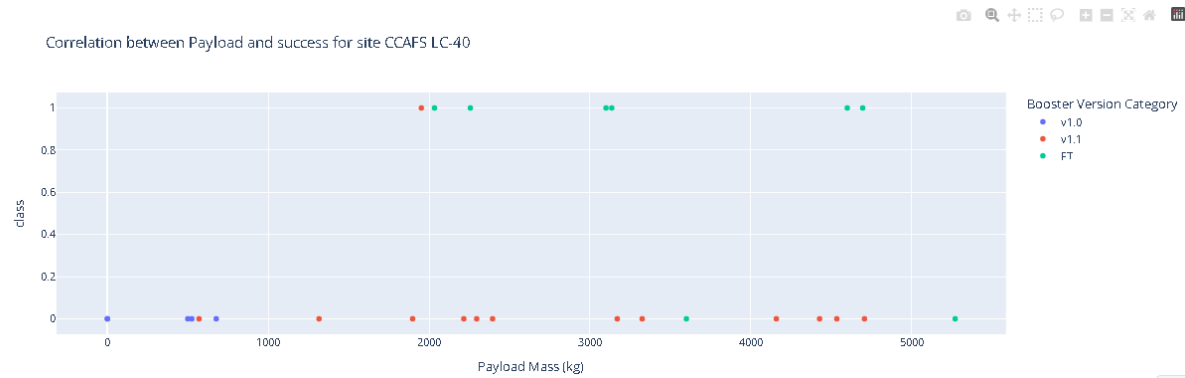


Build a Dashboard with Plotly Dash

Correlation between Payload and success for all sites



Build a Dashboard with Plotly Dash



Predictive Analysis (Classification) (1/9)

- The Predictive Analysis with classification process is presented in the following eight slides.
- [The Predictive Classification notebook](#) is provided as an external reference for transparency and peer-review purposes

Predictive Analysis (Classification) (2/9)

To predict Falcon 9 first stage landing success, four classification algorithms were trained and tuned on a feature matrix of 80 labelled launches: Logistic Regression, Support Vector Machine, Decision Tree, and K-Nearest Neighbours. Each model was optimised using `GridSearchCV` with 10-fold cross-validation, systematically searching across hyperparameter grids — regularisation strength, kernel type, tree depth, and neighbour count — to find the configuration that generalised best beyond the training data. Features were standardised with `StandardScaler` prior to fitting to ensure distance-based models were not distorted by scale differences across variables.

Model performance was evaluated on an 18-sample held-out test set using four complementary metrics — Accuracy, Precision, Recall, and F1 Score — rather than accuracy alone, accounting for the class imbalance present in the dataset. Results were consolidated into a comparative summary table and bar chart to identify trade-offs between precision and recall across models. The best-performing classifier was selected by F1 Score, which balances false positives and false negatives and provides a more robust measure of predictive quality for this binary classification problem.

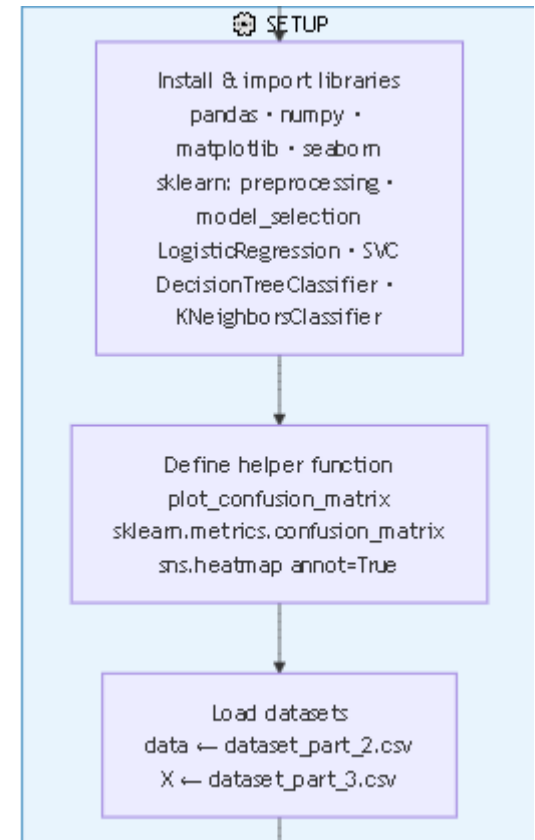
Predictive Analysis (Classification) (3/9)

- Install the required packages, Import the necessary libraries, and load an IBM-curated dataset that enables all students to replicate the results.

https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_2.csv

https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_3.csv

```
def plot_confusion_matrix(y, y_predict):  
    "this function plots the confusion matrix"  
    from sklearn.metrics import confusion_matrix  
  
    cm = confusion_matrix(y, y_predict)  
    ax= plt.subplot()  
    sns.heatmap(cm, annot=True, ax = ax); #annot=True to annotate cells  
    ax.set_xlabel('Predicted labels')  
    ax.set_ylabel('True labels')  
    ax.set_title('Confusion Matrix');  
    ax.xaxis.set_ticklabels(['did not land', 'land']); ax.yaxis.set_ticklabels(['did not land', 'landed'])  
    plt.show()
```



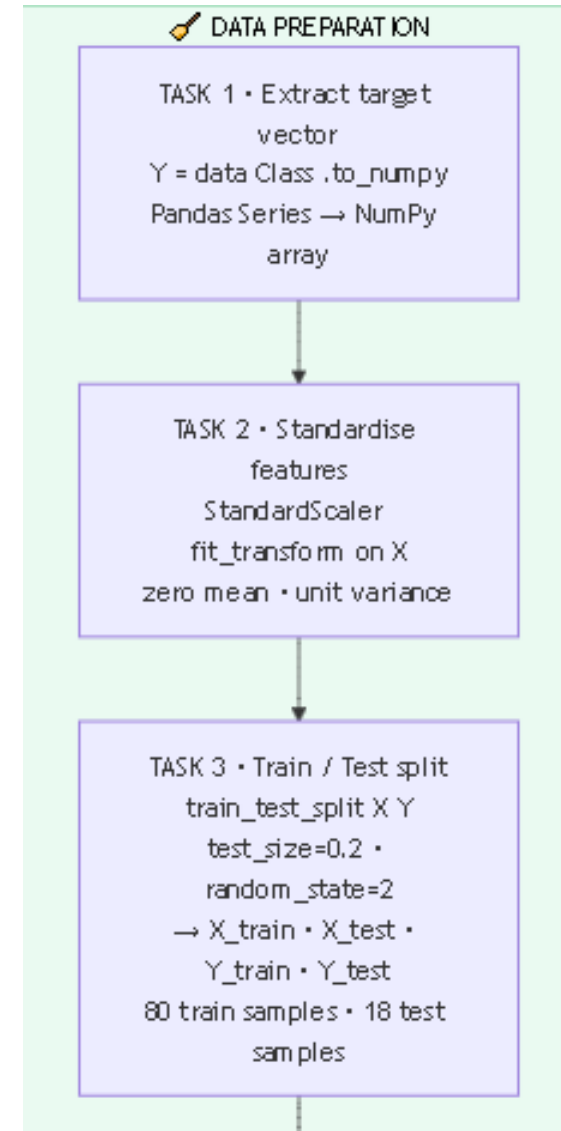
Predictive Analysis (Classification) (4/9)

```
Y = data['Class'].to_numpy()
```

```
array([0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1,  
1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1,  
1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype=int64)
```

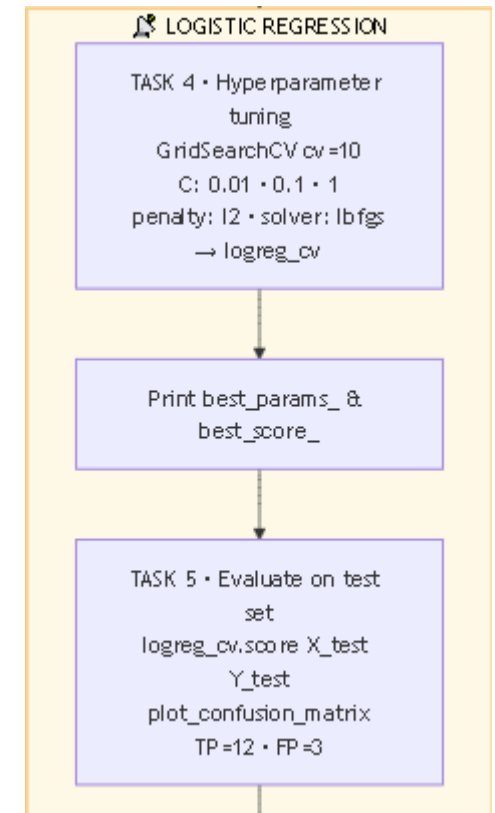
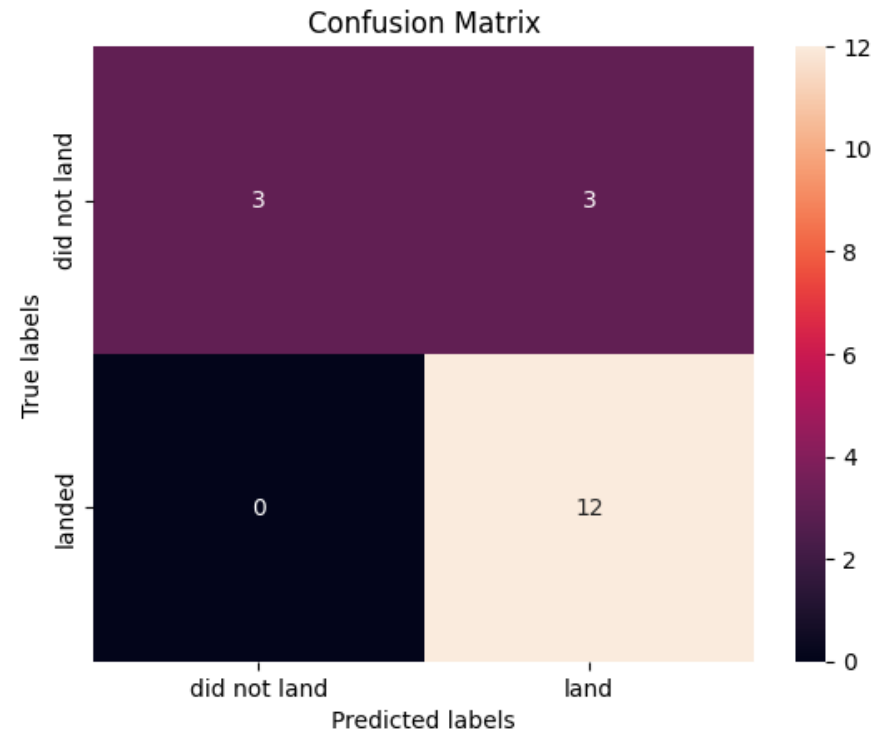
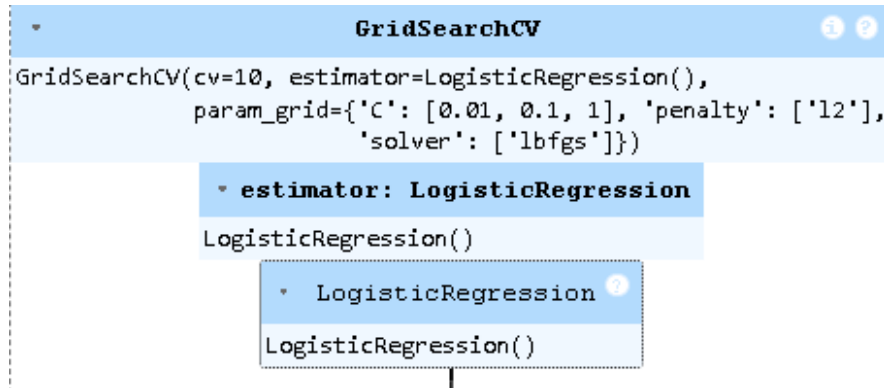
```
scaler = preprocessing.StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, Y_train, Y_test =  
train_test_split(X_scaled, Y, test_size=0.2, random_state=2,  
stratify=Y)
```



Predictive Analysis (Classification) (5/9)

```
parameters = {"C": [0.01, 0.1, 1], 'penalty': ['l2'], 'solver': ['lbfgs']}# 11
lasso l2 ridge
lr=LogisticRegression()
logreg_cv = GridSearchCV(estimator=lr, param_grid=parameters, cv=10)
logreg_cv.fit(X_train, Y_train)
```



tuned hyperparameters :(best parameters) {'C': 0.1, 'penalty': 'l2', 'solver': 'lbfgs'} accuracy : 0.85

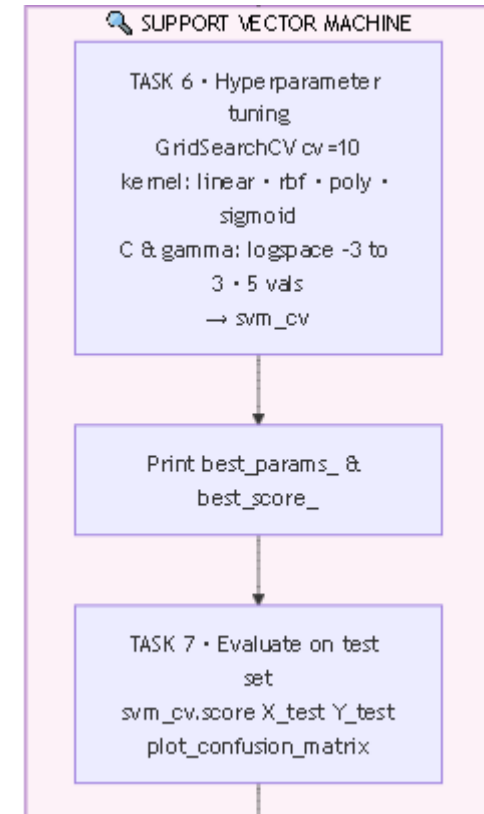
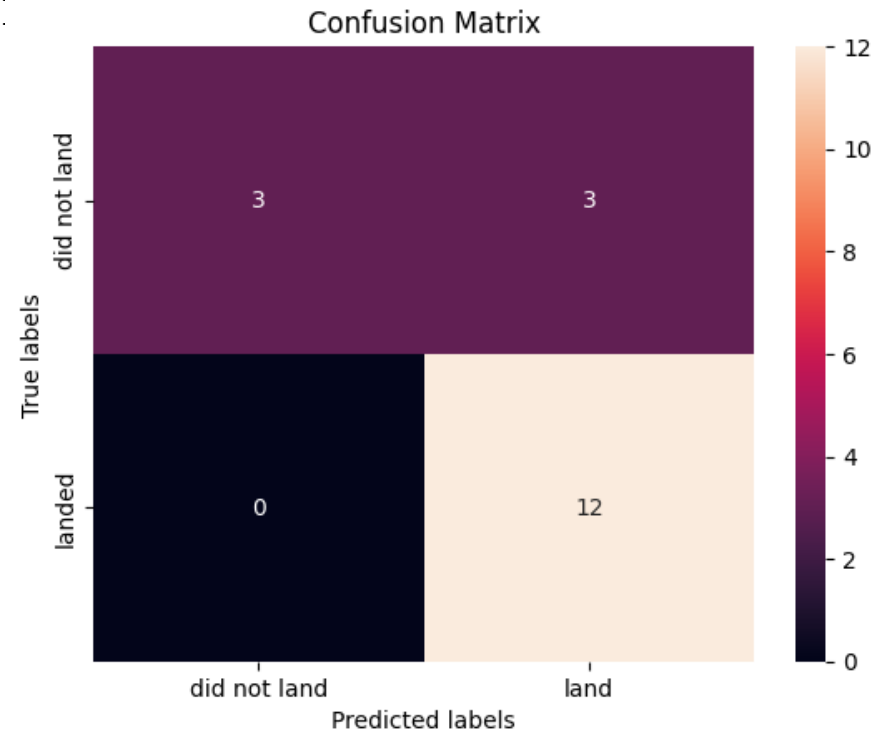
Accuracy on test data: 0.8333333333333334

Predictive Analysis (Classification) (6/9)

```
parameters = {'kernel': ('linear', 'rbf', 'poly', 'rbf', 'sigmoid'),  
              'C': np.logspace(-3, 3, 5),  
              'gamma': np.logspace(-3, 3, 5)}
```

```
svm = SVC()  
svm_cv = GridSearchCV(estimator=svm,  
                      svm_cv.fit(X_train, Y_train)
```

```
GridSearchCV  
GridSearchCV(cv=10, estimator=SVC(),  
             param_grid={'C': array([1.00000000e-03, 3.16227766e-02, 1.00000000e+00, 3.16227766e+01,  
1.00000000e+03]),  
             'gamma': array([1.00000000e-03, 3.16227766e-02, 1.00000000e+00, 3.16227766e+01,  
1.00000000e+03]),  
             'kernel': ('linear', 'rbf', 'poly', 'rbf', 'sigmoid')})  
  * estimator: SVC  
    SVC()  
      * SVC  
        SVC()
```

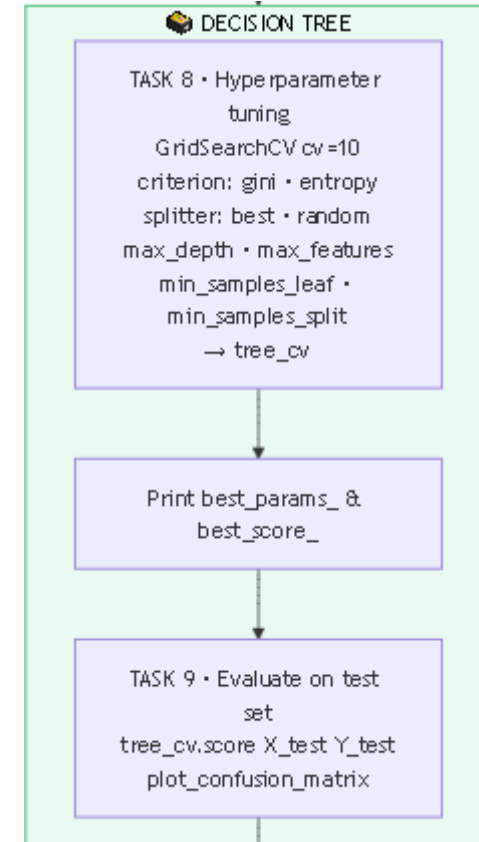
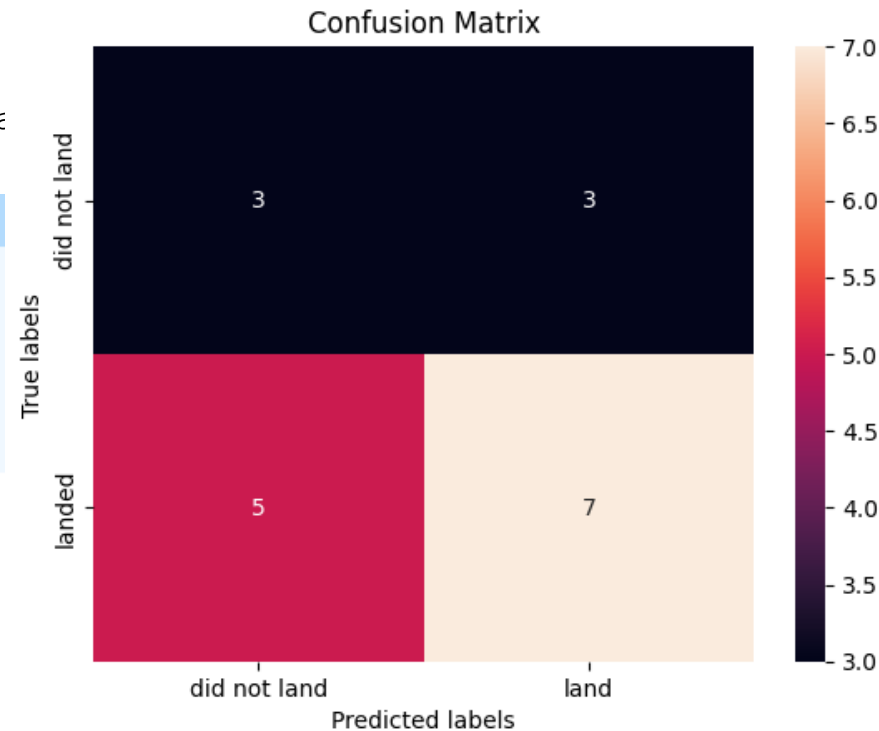
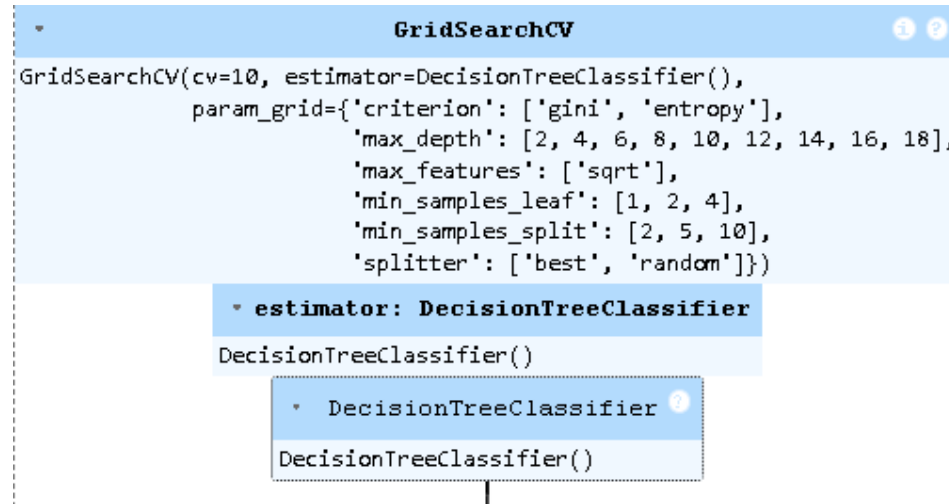


tuned hpyerparameters :(best parameters) {'C': 1.0, 'gamma': 0.03162277660168379, 'kernel': 'sigmoid'}
accuracy : 0.8642857142857144

Accuracy on test data: 0.8333333333333334

Predictive Analysis (Classification) (7/9)

```
parameters = {'criterion': ['gini', 'entropy'],
              'splitter': ['best', 'random'],
              'max_depth': [2*n for n in range(1,10)],
              'max_features': ['sqrt'],
              'min_samples_leaf': [1, 2, 4],
              'min_samples_split': [2, 5, 10]}
tree = DecisionTreeClassifier()
tree_cv = GridSearchCV(estimator=tree, param_grid=parameters)
tree_cv.fit(X_train, Y_train)
```



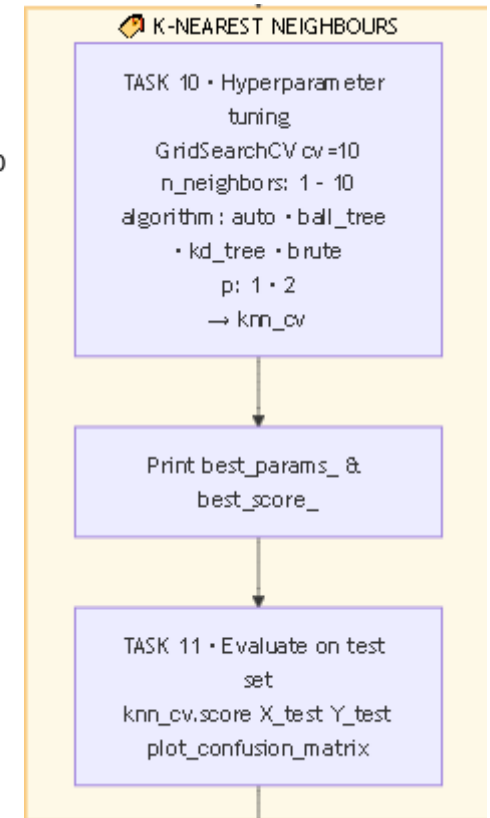
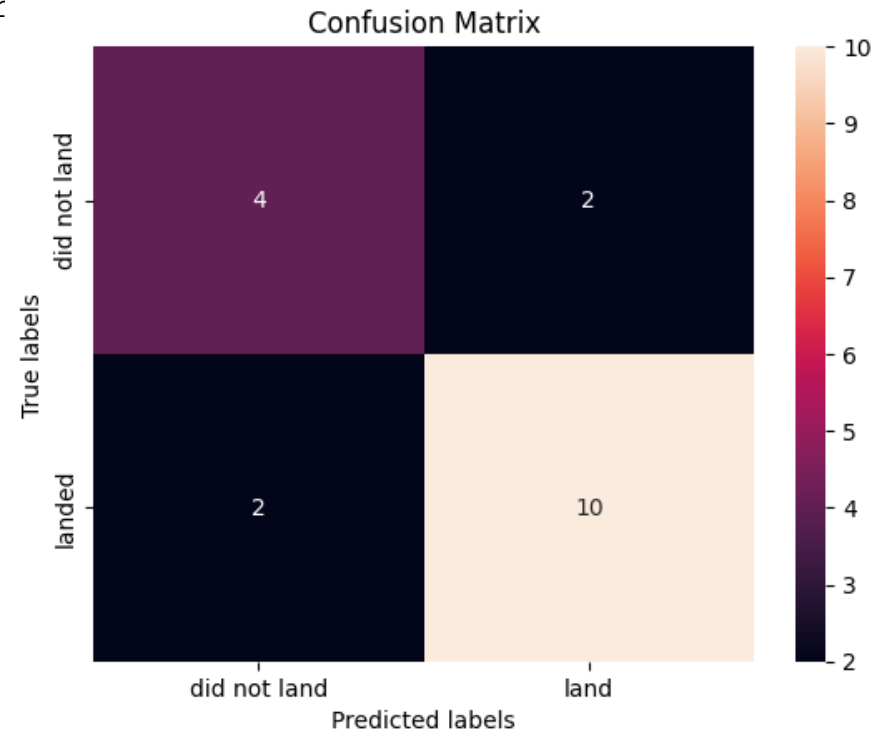
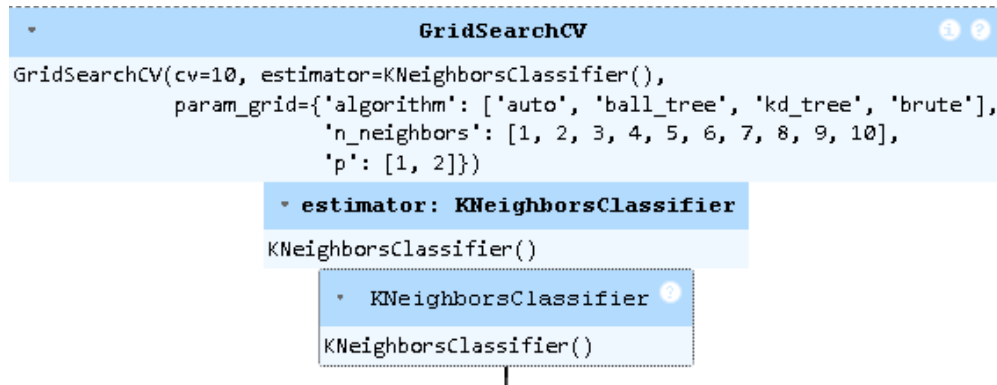
tuned hyperparameters (best parameters) {'criterion': 'gini', 'max_depth': 4, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 5, 'splitter': 'best'}

accuracy : 0.9053571428571429

Accuracy on test data: 0.5555555555555556

Predictive Analysis (Classification) (8/9)

```
parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
              'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
              'p': [1, 2]}
KNN = KNeighborsClassifier()
knn_cv = GridSearchCV(estimator=KNN, param_grid=parameters)
knn_cv.fit(X_train, Y_train)
```



tuned hyperparameters :(best parameters) {'algorithm': 'auto', 'n_neighbors': 4, 'p': 1} accuracy : 0.8642857142857142

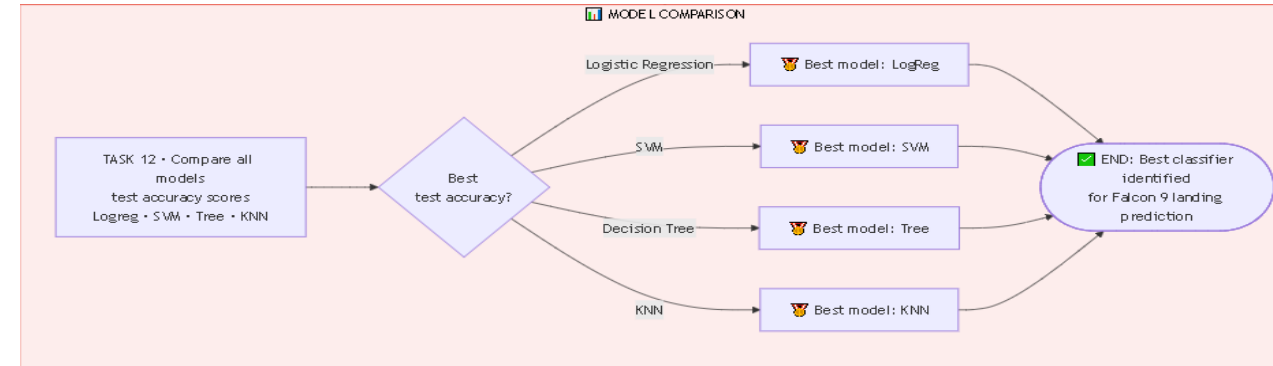
Accuracy on test data: 0.7777777777777778

Predictive Analysis (Classification) (9/9)

```
models = {
    "Logistic Regression": logreg_cv,
    "SVM": svm_cv,
    "Decision Tree": tree_cv,
    "KNN": knn_cv,
}

rows = []
for name, model in models.items():
    yhat = model.predict(X_test)
    rows.append({
        "Model": name,
        "CV Score": round(model.best_score_, 4), # validation (tuning)
        "Test Accuracy": round(model.score(X_test, Y_test), 4), # held-out test
        "Precision": round(precision_score(Y_test, yhat), 4),
        "Recall": round(recall_score(Y_test, yhat), 4),
        "F1 Score": round(f1_score(Y_test, yhat), 4),
    })

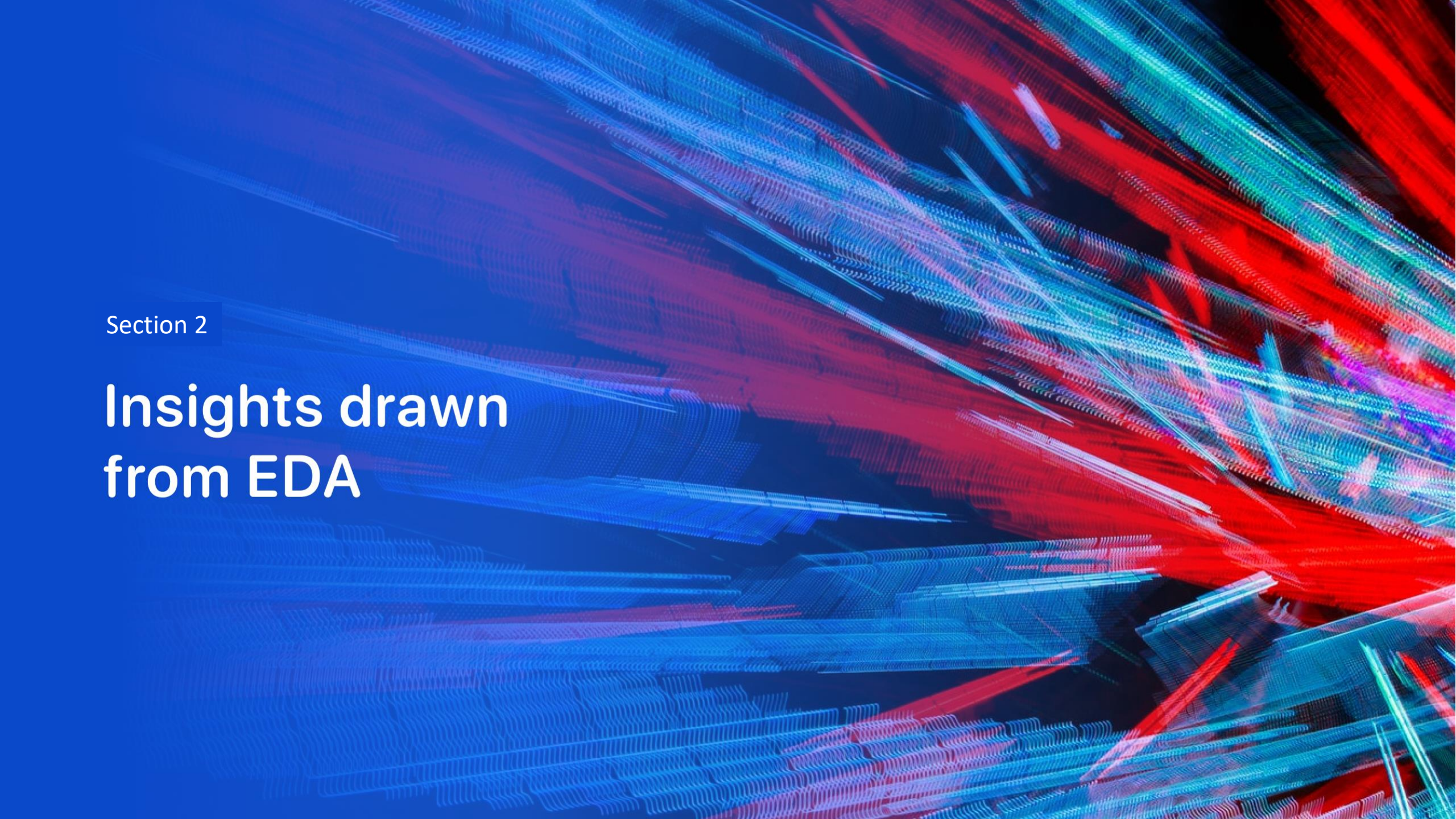
results_df = pd.DataFrame(rows).set_index("Model")
print(results_df.to_string())
```



Model	CV Score	Test Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.8500	0.8333	0.8000	1.0000	0.8889
SVM	0.8643	0.8333	0.8000	1.0000	0.8889
Decision Tree	0.9196	0.8333	0.8462	0.9167	0.8800
KNN	0.8643	0.7778	0.8333	0.8333	0.8333

Results

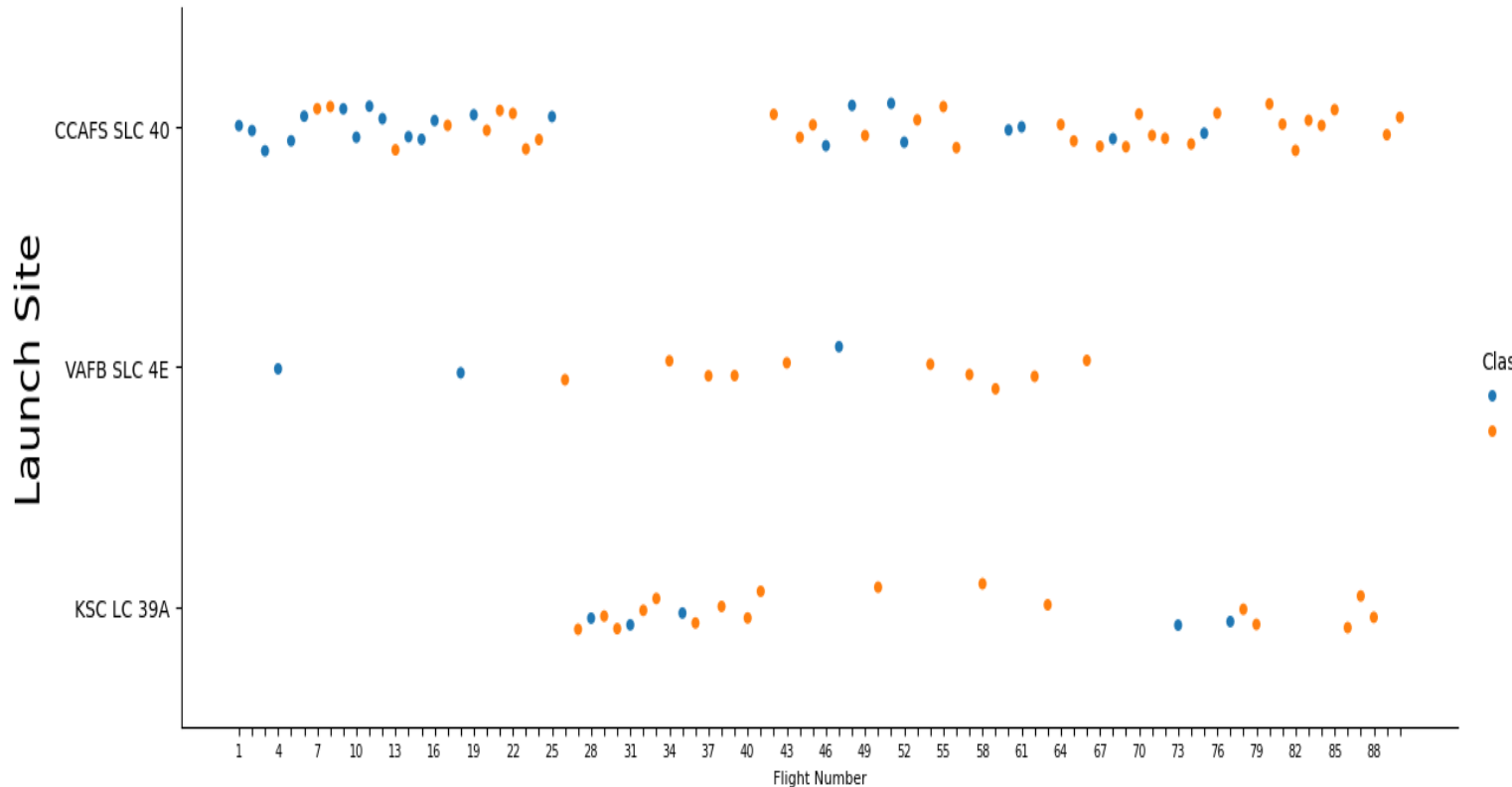
- Average success rate is 66%
- As the flight number increases, the first stage is more likely to land successfully. The payload mass also appears to be a factor; even with more massive payloads, the first stage often returns successfully.
- **CCAFS** was the initial Launch Site. It got initial faults. Concentrates works part of learning curve.
- Top performing orbit types are: **ES-L1, GEO, HEO, and SSO**
- Of all the launches recorded, the **KSC LC 39A** site accounts for the largest proportion of successful missions, representing **41.7%** of the total.
- **KSC LC 39** maintains the highest success rate of any site, with **76.5%** of its launches resulting in a successful outcome.
- The best classification model is Logistic Regression. SVM performs equally but computational cost is higher

The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of red and cyan. A faint, light blue grid pattern is also visible, particularly in the lower-left quadrant. The overall effect is dynamic and technological.

Section 2

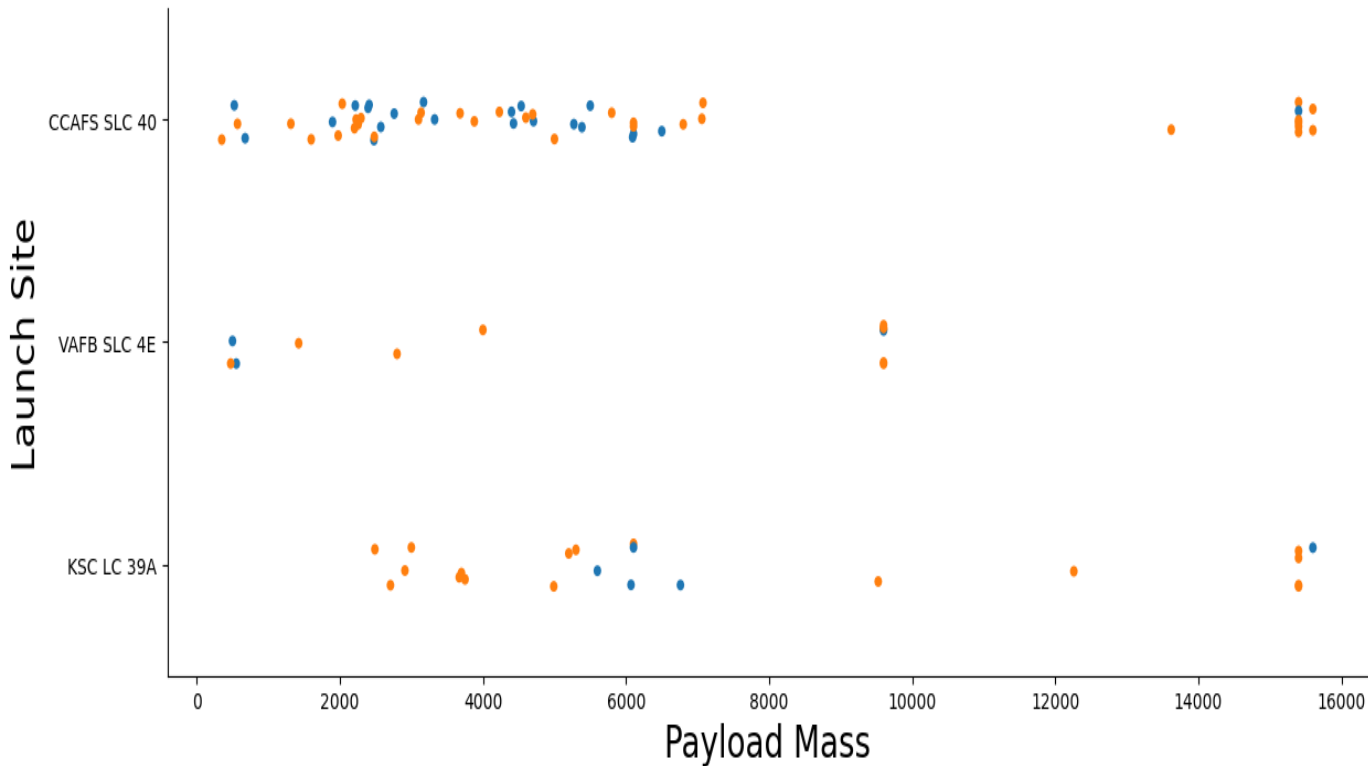
Insights drawn from EDA

Flight Number vs. Launch Site



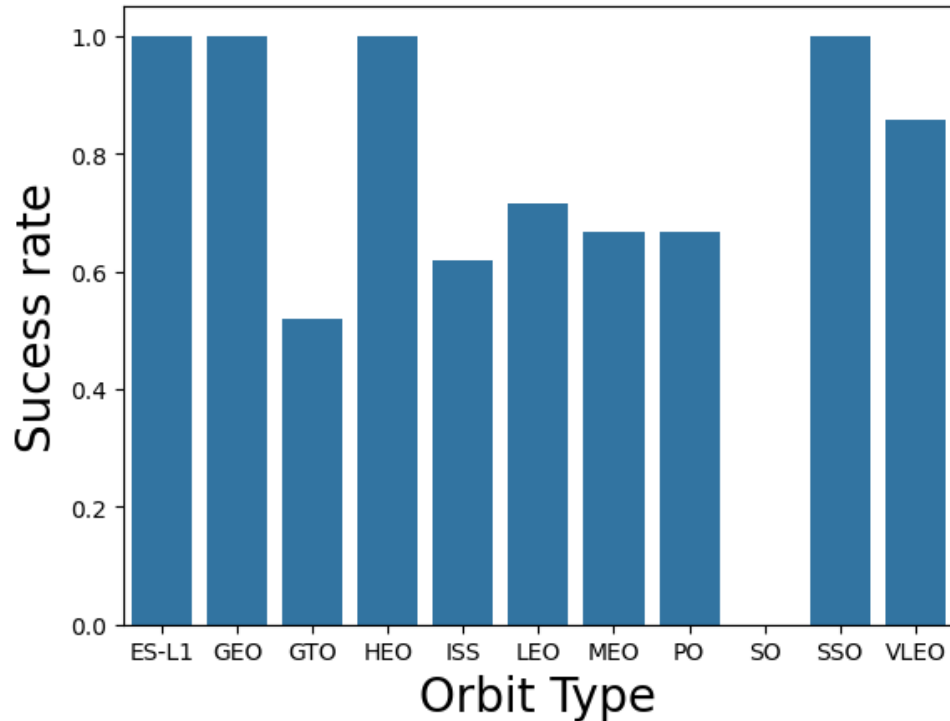
- The colour of the data points indicates a progressive increase in mission success as operational experience matures.
- Initially, launches were conducted exclusively at the **CCAFS** site.
- Following the failure of Flight 24, operations commenced at the other two launch sites, resulting in five consecutive successes at **VAFB** and a higher proportion of successful missions at **KSC**.
- While **KSC** and **VAFB** appear to possess higher success rates than **CCAFS**, the performance across all launch sites becomes remarkably similar if the first 25 launches are regarded as an initial learning curve and excluded from the analysis.

Payload vs. Launch Site



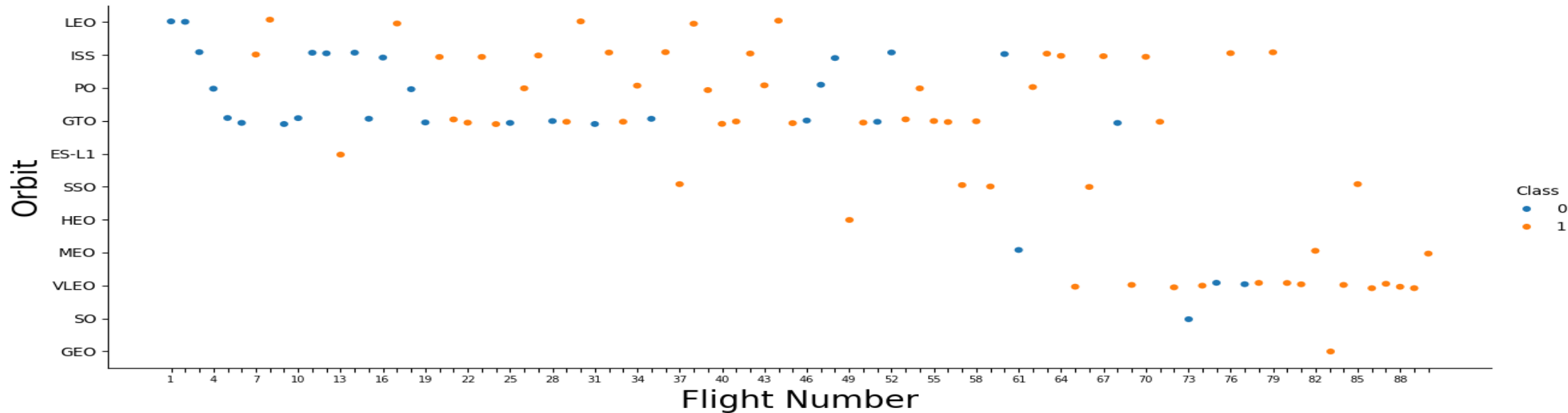
- **CCAFS SLC 40:** This facility accommodates the most extensive array of missions, with a pronounced concentration of payloads spanning the **0 to 7,000 kg** range. Furthermore, it oversees substantial heavy-lift operations surpassing **15,000 kg**.
- **KSC LC 39A:** Operations at this site are principally directed toward mid-to-heavy payloads; one observes a distinct cluster between **2,000 and 6,000 kg**, alongside several missions within the **12,000 to 15,000 kg+** bracket.
- **VAFB SLC 4E:** This site appears to manage a more modest volume of missions, predominantly focused upon the lower end of the mass spectrum—specifically **under 2,000 kg**—as well as select mid-heavy payloads approaching **9,000 kg**.

Success Rate vs. Orbit Type



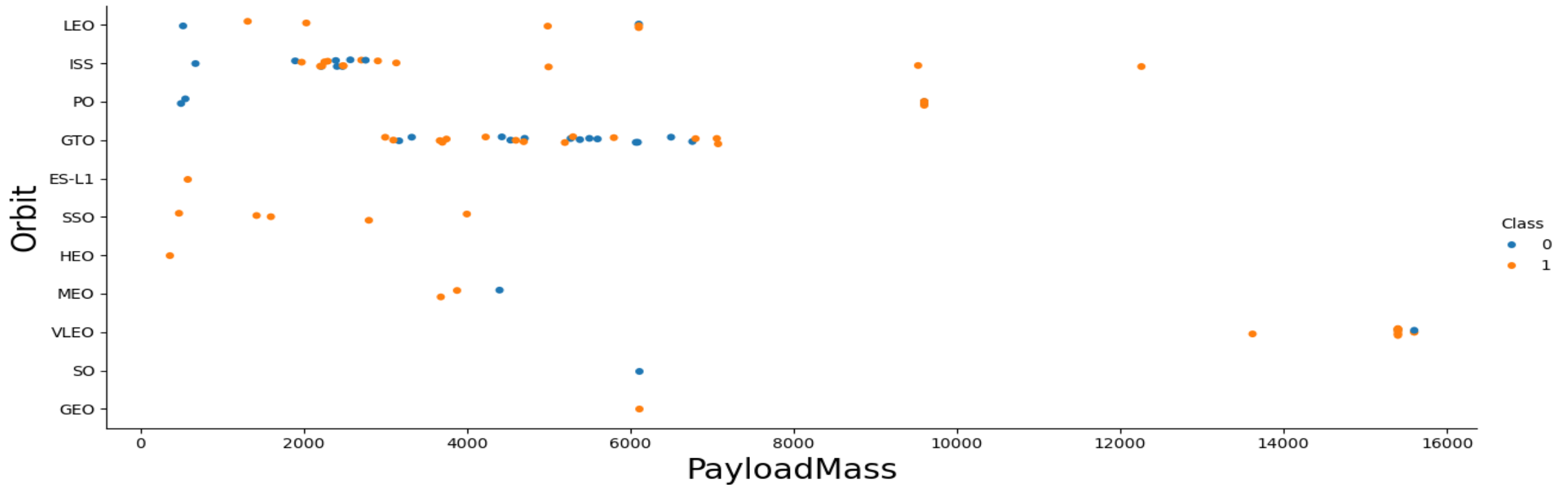
- **1. Top-Performing Orbits (100% Success)**
 - **ES-L1** (Earth-Sun L1 Lagrange point).
 - **GEO** (Geostationary Orbit).
 - **HEO** (High Earth Orbit).
 - **SSO** (Sun-Synchronous Orbit).
- **2. High Reliability Orbits ([70%..85%] Success)**
 - **VLEO** (Very Low Earth Orbit).
 - **LEO** (Low Earth Orbit).
- **3. Moderate to Lower Performance ([50%..70%] Success)**
 - **MEO** (Medium Earth Orbit) and **PO** (Polar Orbit): Both show identical success rates.
 - **ISS** (International Space Station).
 - **GTO** (Geostationary Transfer Orbit).
- **4. Notable Outlier**
 - **SO** (Sun Orbit). This orbit stands out with a success rate of **0.0**, indicating that missions to this specific orbit type have not yet yielded a successful outcome in this dataset.

Flight Number vs. Orbit Type



- Initially, four specific orbit types accounted for the vast majority of flight volume and, consequently, absorbed the impact of the early developmental learning curve. It was only after a consistent success rate was established within these primary orbits that the programme expanded its operational scope to include more specialised orbital profiles."

Payload vs. Orbit Type



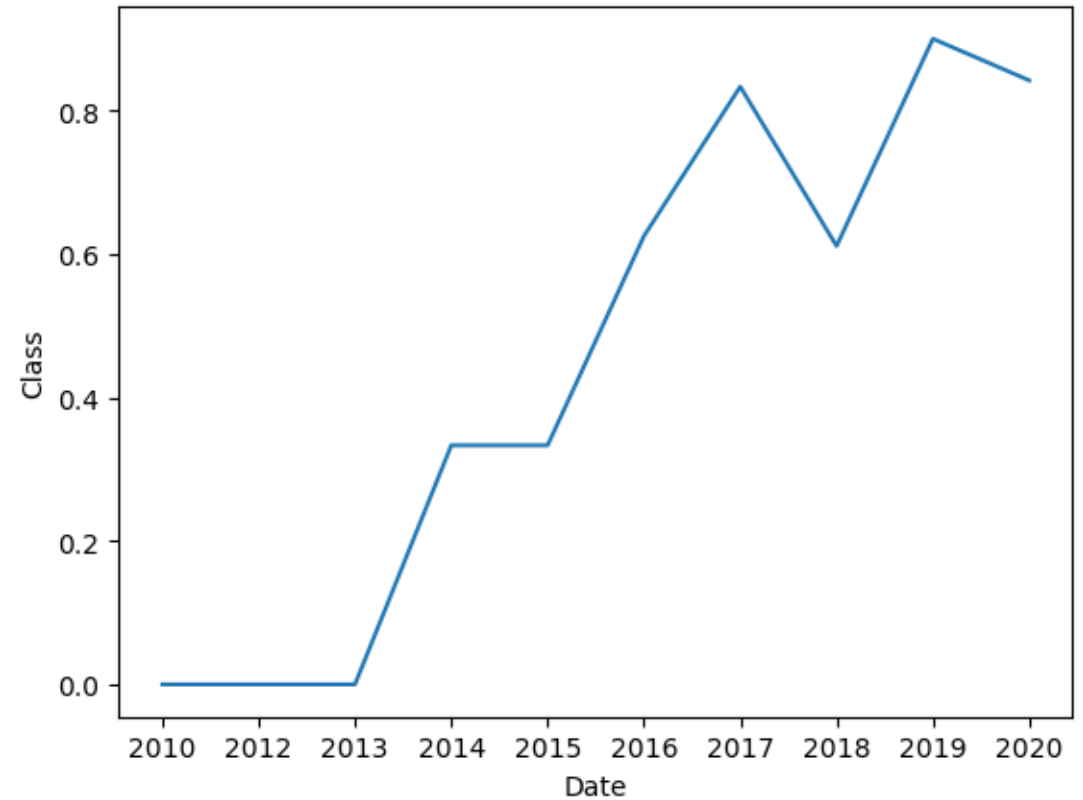
- The data suggests that payload mass does not exert a significant influence on the outcome of a mission across the various orbital profiles. Success is observed consistently across the entire mass spectrum; indeed, the missions carrying the heaviest payloads demonstrate some of the highest reliability recorded in the dataset.“
- **GTO:** have a specific "working range" mostly between **2,500 kg and 7,000 kg**.

Launch Success Yearly Trend

The temporal data illustrates a definitive correlation between the passage of time and mission success.

Following an initial period of developmental challenges between **2010** and **2013**, the programme entered a phase of rapid improvement.

By **2020**, the success rate had stabilised at an impressive level, confirming that cumulative operational experience has successfully mitigated mission risk



All Launch Site Names

Launch_Site

CCAFS LC-40

VAFB SLC-4E

KSC LC-39A

CCAFS SLC-40

- **%sql** select distinct Launch_Site
from SPACEXTABLE

This SQL statement retrieves a comprehensive list of every **unique** launch site stored within the **SPACEXTABLE**, ensuring that each site name is displayed only once, regardless of how many missions were conducted there

Launch Site Names Begin with 'CCA'

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS __KG__	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

- **%sql** select * from SPACEXTABLE WHERE Launch_Site like "CCA%" LIMIT 5
- This specific query is designed to extract a comprehensive set of data for five missions conducted at launch facilities whose names commence with the prefix 'CCA'.
- By employing a wildcard search and a restrictive limit, the command provides a succinct overview of the initial entries associated with those particular sites

Total Payload Mass

Customer	sum(PAYLOAD_MASS__KG_)
NASA (CRS)	45596

- **%sql** select Customer , sum(PAYLOAD_MASS__KG_) from SPACEXTABLE WHERE Customer like "NASA (CRS)" group by Customer
- This query is designed to calculate the aggregate weight of all payloads delivered for the customer designated as '**NASA (CRS)**'.
- By consolidating the individual mass entries associated with this specific client, the command provides a definitive sum of the total cargo transported under their respective missions.

Average Payload Mass by F9 v1.1

Booster_Version	avg(PAYLOAD_MASS__KG_)
F9 v1.1	2928.4

- Calculate the average payload mass carried by booster version F9 v1.1
- **%sql** select Booster_Version , avg(PAYLOAD_MASS__KG_) from SPACEXTABLE WHERE Booster_Version like "F9 v1.1" group by Booster_Version
- The purpose of this query is to determine the mean payload mass transported by the '**F9 v1.1**' booster variant.
- By calculating the arithmetic average of the mass entries associated with this specific model, the command provides an insightful metric regarding the typical lifting capacity utilized during its operational tenure.

First Successful Ground Landing Date

Landing_Outcome	min(Date)
Success (ground pad)	2015-12-22

- **%sql** select Landing_Outcome , min(Date) from SPACEXTABLE where Landing_Outcome like "%ground%" group by Landing_Outcome
- This query is formulated to identify the earliest recorded date for each type of landing success achieved upon solid terrain.
- By isolating landing outcomes that contain the term '**ground**', the command effectively establishes the historical milestone for the first successful touchdown of each relevant category.

Successful Drone Ship Landing with Payload between 4000 and 6000

Booster_Version	PAYLOAD_MASS__KG_	Mission_Outcome	Landing_Outcome
F9 FT B1022	4696	Success	Success (drone ship)
F9 FT B1026	4600	Success	Success (drone ship)
F9 FT B1021.2	5300	Success	Success (drone ship)
F9 FT B1031.2	5200	Success	Success (drone ship)

- **%sql** select Booster_Version, PAYLOAD_MASS__KG_, Mission_Outcome, Landing_outcome from SPACEXTABLE where Landing_Outcome like 'Success (drone ship)' and 4000 < PAYLOAD_MASS__KG_ and PAYLOAD_MASS__KG_ < 6000
- This query is designed to extract a detailed list of missions that achieved a successful landing upon a **drone ship**, specifically where the payload mass fell within the range of **4,000 to 6,000 kilograms**.
- By refining the search parameters to include both a specific recovery method and a precise weight bracket, the command provides a focused overview of the programme's mid-to-heavy lift performance

Total Number of Successful and Failure Mission Outcomes

<code>substr(Mission_outcome,1,7)</code>	<code>count(Mission_Outcome)</code>
Failure	1
Success	100

- **%sql substr** Select `substr(Mission_outcome,1,7)` , `count(Mission_Outcome)` from SPACEXTABLE group by `substr(Mission_Outcome,1,7)`
- This query is employed to categorise and tally the various mission outcomes by isolating the first seven characters of each status entry.
- By truncating the descriptions in this manner, the command ensures that slight variations in terminology are consolidated, thereby providing a clear numerical summary of successes and failures across the entire dataset.

Boosters Carried Maximum Payload

- **%sql** select Booster_Version from SPACEXTABLE WHERE PAYLOAD_MASS__KG_ = (select max(PAYLOAD_MASS__KG_) from SPACEXTABLE) order by Booster_versión
- This query is designed to identify the specific booster versions associated with the heaviest payloads recorded in the dataset.
- By utilising a subquery to determine the absolute maximum payload mass, the command isolates and lists the boosters capable of such a feat, presenting them in a structured alphabetical order.

Booster_Version

F9 B5 B1048.4

F9 B5 B1048.5

F9 B5 B1049.4

F9 B5 B1049.5

F9 B5 B1049.7

F9 B5 B1051.3

F9 B5 B1051.4

F9 B5 B1051.6

F9 B5 B1056.4

F9 B5 B1058.3

F9 B5 B1060.2

F9 B5 B1060.3

2015 Launch Records

Month	Booster_Version	Launch_Site	Landing_Outcome
January	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
April	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)

- **%sql** select CASE substring(Date, 6,2) WHEN '01' THEN 'January' WHEN '02' THEN 'February' WHEN '03' THEN 'March' WHEN '04' THEN 'April' WHEN '05' THEN 'May' WHEN '06' THEN 'June' WHEN '07' THEN 'July' WHEN '08' THEN 'August' WHEN '09' THEN 'September' WHEN '10' THEN 'October' WHEN '11' THEN 'November' WHEN '12' THEN 'December' END as Month, Booster_Version, Launch_site, Landing_Outcome from SPACEXTABLE where substr(Date,0,5)='2015' and Landing_outcome like 'Failure%'
- This query is structured to retrieve a detailed log of all missions that resulted in a landing failure during the calendar year 2015.
- By employing a conditional 'CASE' expression, the command translates numerical month values into their formal English names, thereby enhancing the legibility and presentation of the resulting data

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

Landing_Outcome	events
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

- `%sql select Landing_Outcome, count(Landing_Outcome) as events from SPACEXTABLE where '2010-06-04' <= Date and Date < '2017-03-20' group by Landing_Outcome order by events desc`
- This query is designed to provide a comprehensive tally of mission landing outcomes within a specific historical window, specifically from the 4th of June 2010 to the 20th of March 2017.

By aggregating these results and arranging them in descending order of frequency, the command offers a clear overview of the most prevalent landing events during this period.

A satellite view of Earth from space, showing the curvature of the planet and the glowing lights of cities and continents against the dark background of space. The Earth's surface is a mix of dark blue oceans and lighter blue/white clouds. The lights are concentrated in the lower right quadrant, showing a dense network of urban areas.

Section 3

Launch Sites Proximities Analysis

Launch Sites



VAFB SLC-4E (Vandenberg Space Force Base, California): Situated on the **West Coast**, this facility is ideally positioned for launches into polar and sun-synchronous orbits. Its location allows rockets to ascend over the open Pacific Ocean, ensuring that any debris or primary stages fall safely away from populated areas.

KSC LC-39A (Kennedy Space Center, Florida): Located on the **East Coast**, this historic site is the primary hub for heavy-lift missions and crewed spaceflight. It leverages the Earth's rotational speed to assist launches into equatorial and geostationary orbits.

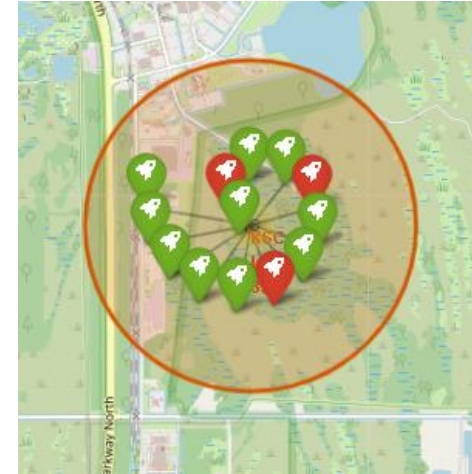
CCAFS SLC-40 (Cape Canaveral Space Force Station, Florida): Adjacent to the Kennedy Space Center, this site serves as a high-frequency operational facility, supporting a significant volume of commercial and satellite deployment missions.

LaunchSite's Launch outcomes

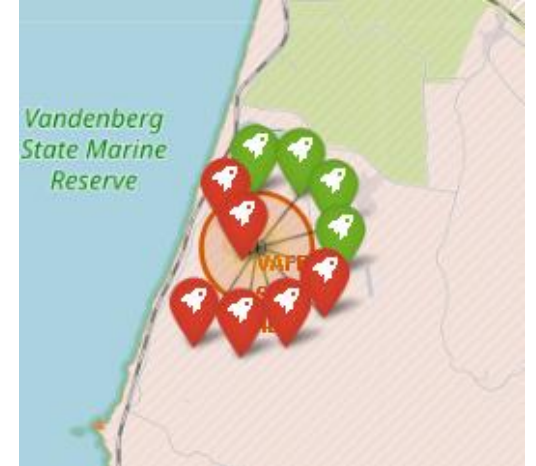
CCAFS SLC-40



KSC LC-39A



VAFB SLC-4E

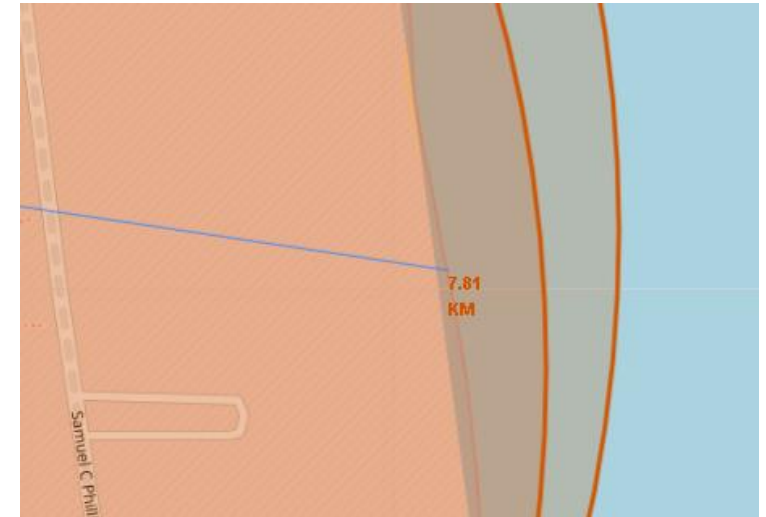
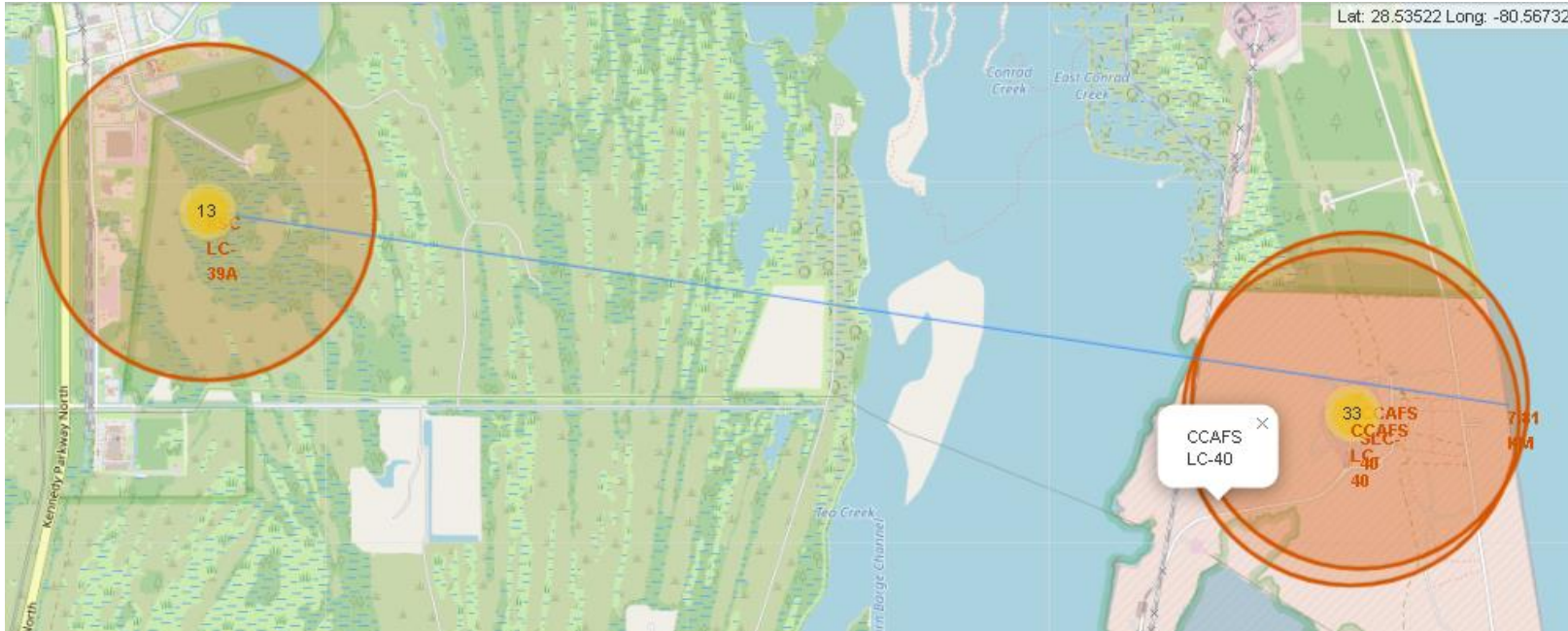


KSC LC-39A Performance: This facility demonstrates a remarkably high success-to-failure ratio. The visual density of green markers suggests that **KSC LC-39A** has matured into the programme's most consistently reliable site for successful mission outcomes.

VAFB SLC-4E Consistency: Although this West Coast site manages a lower volume of flights, it exhibits a commendable level of reliability, with a balanced distribution of successful missions along the California coastline.

CCAFS SLC-40 Challenges: This site presents the most complex operational history. The spiral of red markers (representing failures) clearly illustrates the early "learning curve" the programme endured. However, the subsequent green markers indicate a significant and successful refinement of protocols over time.

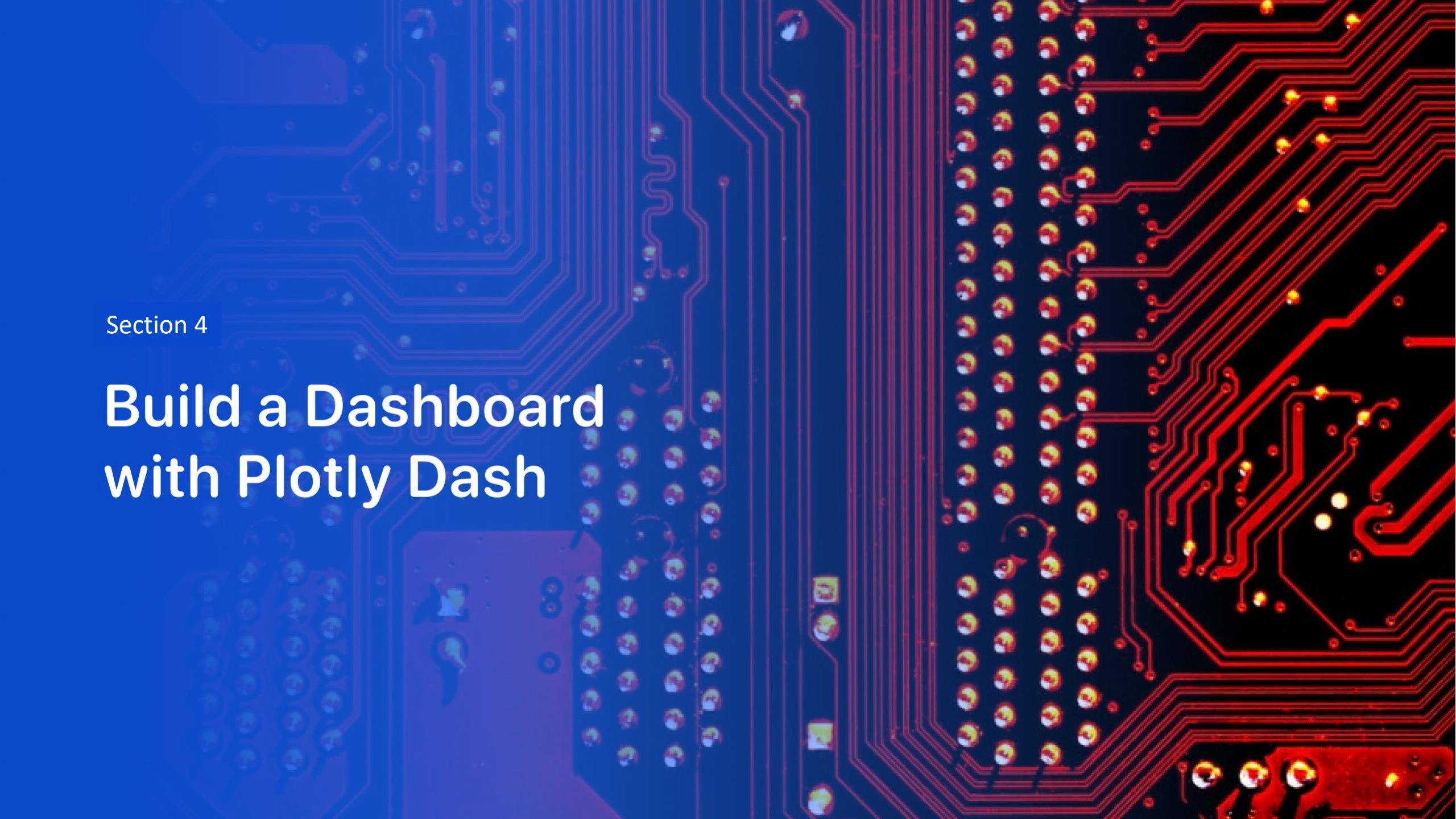
Distance to Coastline of KSC LC-39A



The image illustrates the spatial relationship between the two primary Florida launch facilities, **KSC LC-39A** and **CCAFS LC-40**, and the adjacent Atlantic coastline.

A blue geodesic line has been plotted starting from the central coordinates of the **KSC LC-39A** facility. It terminates at the shoreline of the Atlantic Ocean

The numerical annotation at the eastern terminus of the blue line explicitly confirms that the distance from the launch site to the coastline is **7.81 km**.

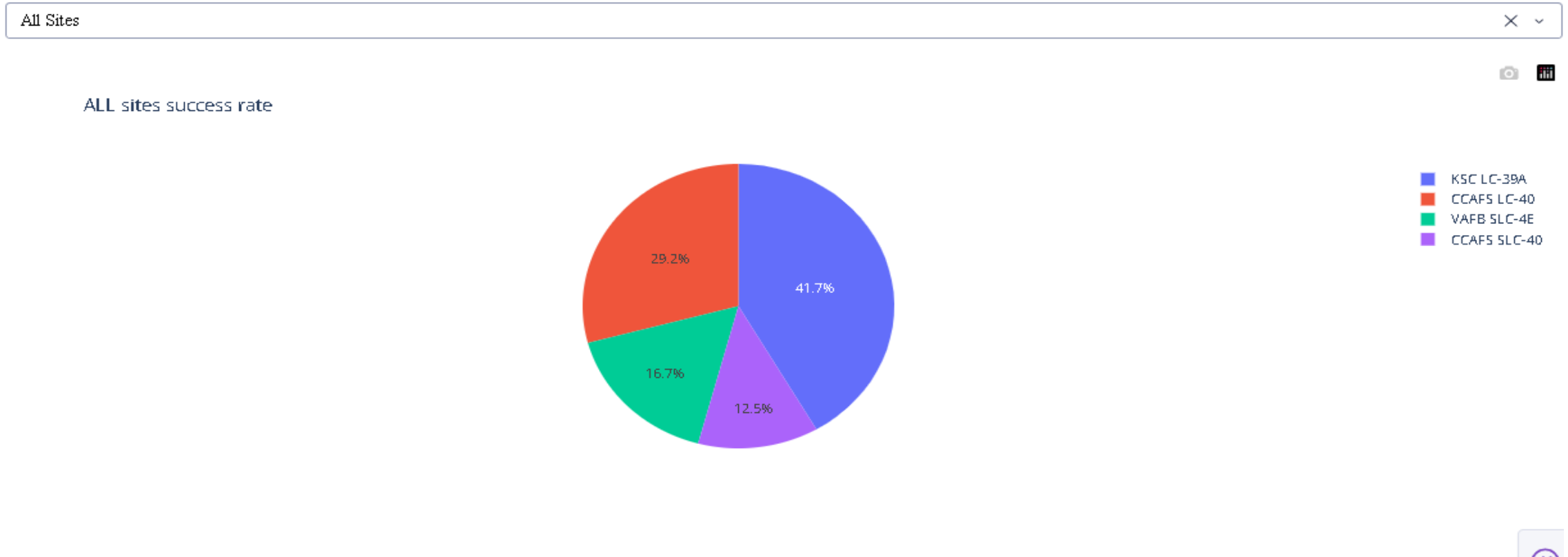


Section 4

Build a Dashboard with Plotly Dash

All Sites Success rate

SpaceX Launch Records Dashboard



- Of all the launches recorded, the **KSC LC 39A** site accounts for the largest proportion of successful missions, representing **41.7%** of the total.

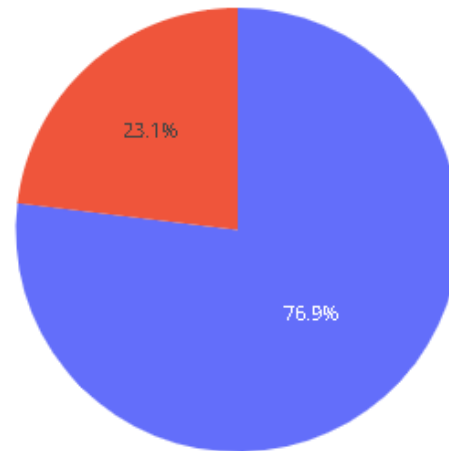
KSC LC 39 Success rate

SpaceX Launch Records Dashboard

KSC LC-39A



KSC LC-39A success rate



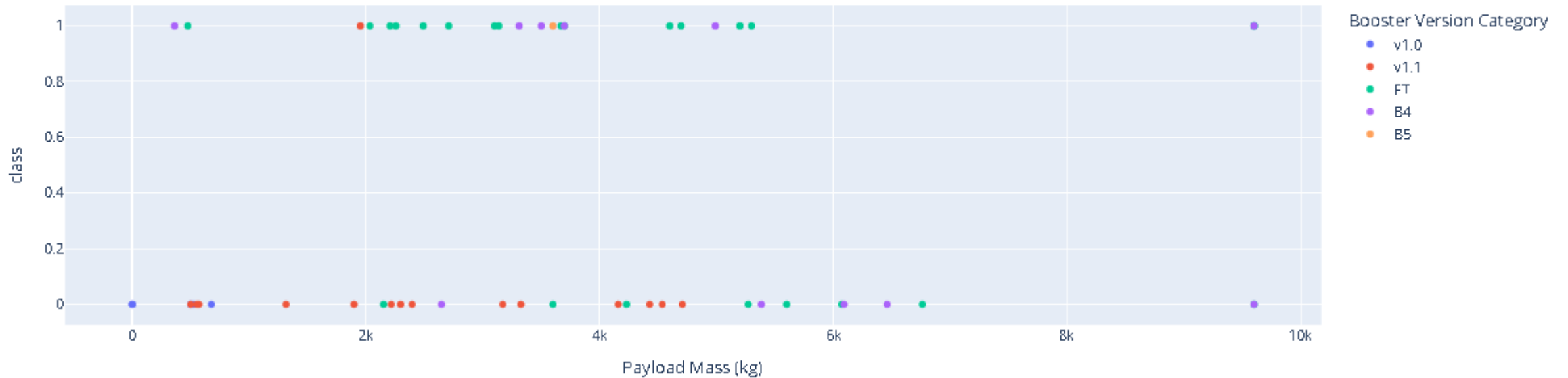
■ 1
■ 0

- **KSC LC 39** maintains the highest success rate of any site, with **76.5%** of its launches resulting in a successful outcome.



Correlation between Booster and success

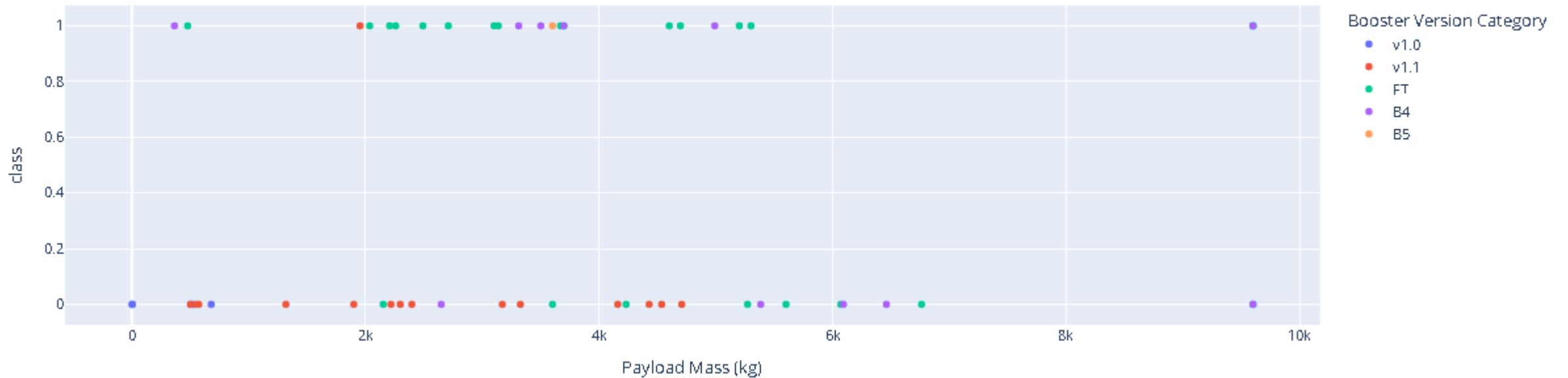
Correlation between Payload and success for all sites



- FT has the highest density of successful launches (Class 1) across a broad range of payload masses, particularly between **2,000 kg and 5,000 kg**. It appears to be the most consistent and reliable. It
- **v1.1** shows a significantly higher failure rate with more data points clustered at the bottom of the chart (Class 0), especially within the **0 to 4,000 kg** range.

Correlation between Payload and success

Correlation between Payload and success for all sites

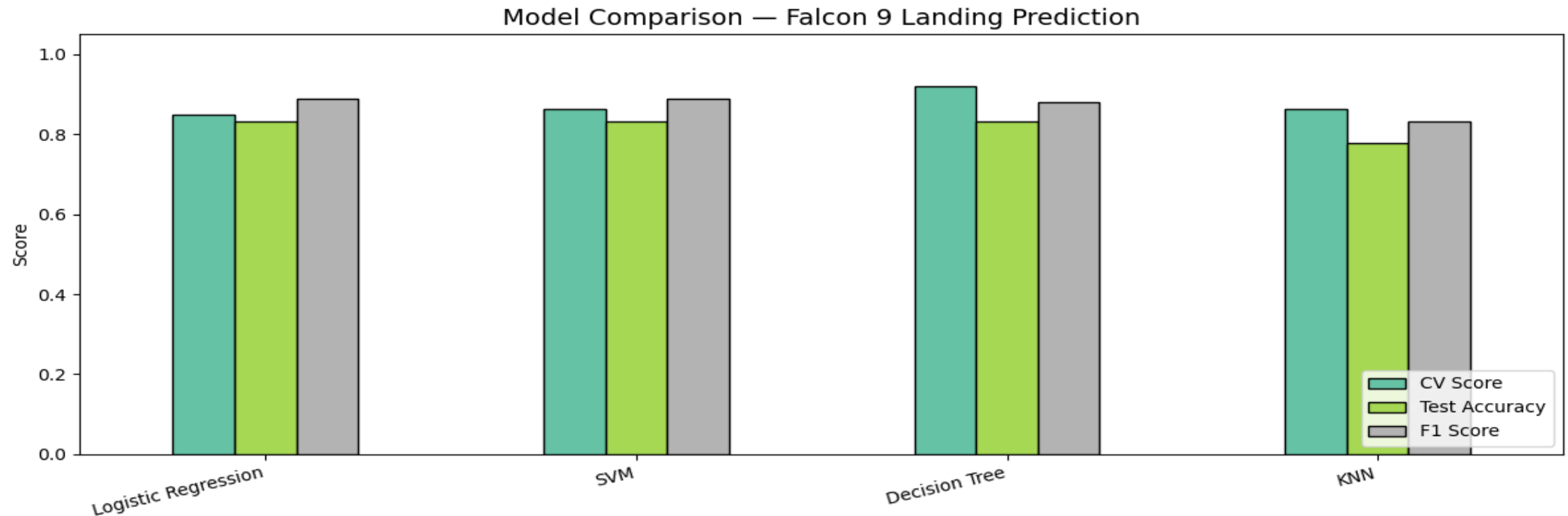


- **Peak performance** is observed within the **3,000 kg to 5,000 kg** payload bracket, characterised by a high density of successful missions.

Section 5

Predictive Analysis (Classification)

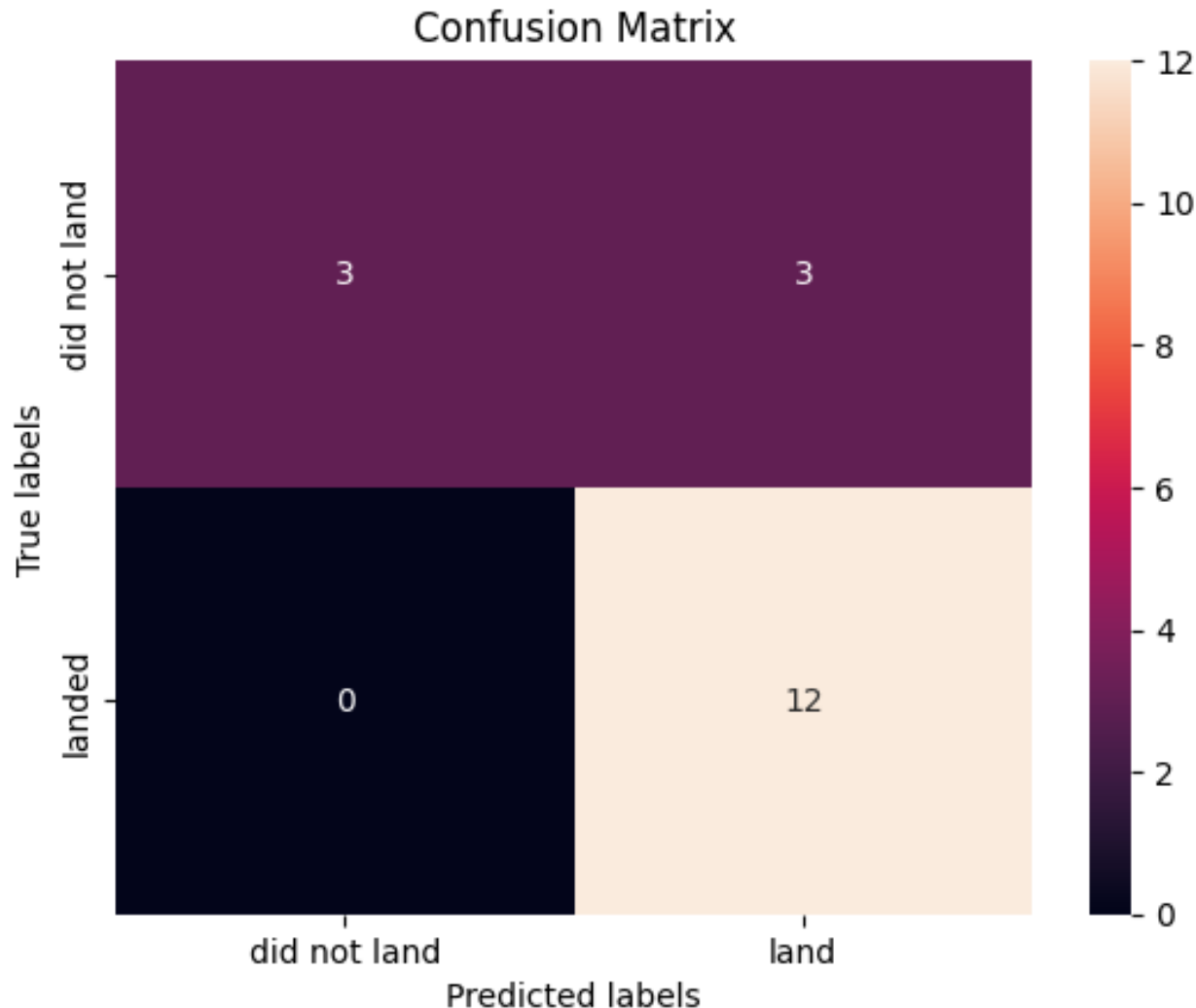
Classification Accuracy



Model	CV Score	Test Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.8500	0.8333	0.8000	1.0000	0.8889
SVM	0.8643	0.8333	0.8000	1.0000	0.8889
Decision Tree	0.9196	0.8333	0.8462	0.9167	0.8800
KNN	0.8643	0.7778	0.8333	0.8333	0.8333

Three models tie at the highest test accuracy of 83.33% — Logistic Regression, SVM, and Decision Tree. In machine learning, when a simpler model matches a complex one, the simpler model wins

Confusion Matrix Logistic Regression



What the model gets right. Of the 12 launches that genuinely landed, the model correctly identified all 12 — a perfect Recall of 1.0 on the positive class. It never predicted a failed landing when the booster actually succeeded. It will never cause a stakeholder to underestimate SpaceX's reuse capability on a mission that was in fact recovered.

Where the model falls short. Of the 6 launches that did not land, the model misclassified 3 as successful landings — the False Positive cells. This means the model is overconfident: when it predicts a landing will fail, it is right only half the time (Precision on the negative class = 0.50). In business terms, these are the cases where a competing bidder might incorrectly assume SpaceX is flying an expendable booster and price their own bid accordingly — only to discover SpaceX was operating on reused hardware and could afford a lower margin.

Overall read. The model's asymmetric error pattern — zero False Negatives, three False Positives — tells a consistent story: it has a strong bias towards predicting successful landings, which reflects the underlying class distribution of the dataset (successful landings dominate the later missions that make up the bulk of the training data). The model has learned the general trend well but has not fully internalised the conditions under which recovery fails. With only 18 test samples, these three misclassifications carry outsized weight, which is why F1 Score (0.889) is more honest measure of performance than the headline accuracy of 83.3%.

Conclusions

- **1. Machine learning can reliably predict Falcon 9 landing success from public data.** A model trained exclusively on publicly available launch records achieved 83.3% test accuracy with zero missed successful landings. This confirms that that proprietary data is not a prerequisite for commercially relevant intelligence about SpaceX's operations.
- **2. Mission experience is the strongest predictor of landing success.** Success rate correlates with accumulated flight experience.
- **3. Simplicity wins when data is scarce.** With only 90 launches available for training and 18 for testing, the most complex models offered no advantage over the simples
- **4. The model's error profile matters more than its accuracy headline.** An 83.3% accuracy shared by three of the four models obscures the most commercially significant finding: the selected model never fails to identify a successful landing (Recall = 1.0), but incorrectly predicts a landing half the time when recovery actually fails. For a competitor building a bid strategy, this asymmetry is critical — the model will never cause an underestimation of SpaceX's reuse capability, but it may occasionally overestimate it. Understanding *where* a model is wrong is more valuable than knowing *how often* it is right.

Thank you!

